Wildbits F256 6809 Reference Manual

Peter Weingartner
6809 Changes by John Federico
11-10-2026

Contents

1 Introduction **7**
- 1.1 The machines . . . 7
- 1.2 The 6809 and the core . . . 7
- 1.3 The parts of the machine . . . 8
- 1.4 Raw programs and NitrOS-9 . . . 8
- 1.5 The examples . . . 9
- 1.6 Notation . . . 9

2 Memory Management **11**
- 2.1 Blocks . . . 11
- 2.2 Slots and LUTs . . . 12
- 2.3 Fixed addresses . . . 13
- 2.4 Device blocks . . . 13
- 2.5 The MMU registers . . . 13
- 2.6 Reset and boot . . . 14
- 2.7 Raw 6809 examples . . . 15
- 2.8 Under NitrOS-9 . . . 16

3 The Text Screen **17**

4 Graphics **26**

5 Sprites **34**

6 Tiles **40**

7 Miscellaneous Features of TinyVicky **47**

8 Sound **54**

9 Interrupt Controller **62**
- 9.1 Groups and registers . . . 62
- 9.2 Reset state . . . 65
- 9.3 NMI from the IEC port . . . 65

9.4 Raw 6809 example: the start-of-frame interrupt 65
9.5 Under NitrOS-9 66

10 Tracking Time **67**

11 Versatile Interface Adapter **73**

12 Game Controllers **76**

13 SD Card Interface **80**

14 Keyboard and Mouse **82**

15 Serial and Wi-Fi Port **86**

16 Direct Memory Access **89**

17 System Control Registers **95**

18 IEC Serial Port **99**

19 Integer Math Coprocessor **100**

20 Using the Debug Port **102**

21 Expansion Connector **106**

22 F256jr I/O Connectors and Jumpers **108**

23 Memory Maps **113**

24 References **117**

25 Copyright **118**

List of Tables

2.1 Physical memory blocks . . . 11
2.2 CPU memory slots . . . 12
2.3 Device blocks . . . 14
2.4 MMU registers . . . 14

3.1 Text Color Lookup Tables . . . 19
3.2 Text Color Lookup Tables Relative to Block $C1 . . . 20
3.3 VICKY Master Control Registers . . . 20
3.4 A sample character . . . 21
3.5 Text Cursor Registers . . . 22
3.6 Text Cursor Flash Rates . . . 22
3.7 MemText Registers . . . 23
3.8 Text Cursor Flash Rates . . . 24
3.9 MemText Foreground/Background LUTS . . . 25
3.10 MemText Registers . . . 25
3.11 MemText Registers . . . 25

4.1 Graphics Color Lookup Tables . . . 27
4.2 Bitmap and Tile Map Layer Registers . . . 30
4.3 Bitmap and Tile Map Layer Codes . . . 30
4.4 Bitmap Registers . . . 31

5.1 Sprite Registers for a Single Sprite . . . 34
5.2 Sprite Sizes . . . 35

6.1 Tile Map Registers . . . 41
6.2 A Tile Map Entry . . . 42
6.3 Arrangement of Tiles in a Tile Set Image . . . 43
6.4 Tile Set 0–3 Registers . . . 44
6.5 Tile Set Registers 4–7 . . . 45

7.1 DIP Switch Register . . . 47
7.2 Border Registers . . . 48

7.3 Background Color Registers 48
7.4 Line Interrupt and Beam Position Registers 49

8.1 CODEC Control Registers 55
8.2 SN76489 Channel Registers 56
8.3 SN76489 Command Formats 56
8.4 SN76489 Noise Frequencies 57
8.5 SID V1 and V2 Registers 59
8.6 SID V3 and Miscellaneous Registers 60
8.7 OPL3 Registers 61

9.1 Interrupt controller registers 63
9.2 Interrupt polarity and edge 64

10.1 Timer Registers 67
10.2 Real Time Clock Registers 69
10.3 RTC Periodic Interrupt Rates 70

11.1 VIA Registers 74
11.2 VIA Control Registers 74

12.1 Joystick Flags 77
12.2 NES/SNES Gamepad Registers 79
12.3 NES/SNES Data Registers 79

13.1 External SD Card Interface Registers 80
13.2 Internal SD Card Interface Registers 80

14.1 PS/2 Port Registers 82
14.2 Mouse Pointer Registers 83
14.3 F256k Keyboard LEDs 85

15.1 UART Registers 86
15.2 UART Data Length 87
15.3 UART Stop Bits 87
15.4 UART Parity 87
15.5 UART RX FIFO Trigger 88
15.6 UART Divisors 88

16.1 DMA Registers (Part 1) 90
16.2 DMA Registers (Part 2) 90

17.1 System Control Registers 95
17.2 LED Flash Rates 96
17.3 System Reset 96
17.4 Random Number Generator 97
17.5 Machine ID and Versions 97
17.6 Machine IDs 98

18.1 IEC Registers 99

19.1 Unsigned Multiplication Registers 100
19.2 Unsigned Division Registers 100
19.3 Unsigned 32-bit Addition Registers 101

20.1 USB Debug Port Command Packet 103
20.2 USB Debug Port Reponse Packet 104
20.3 USB Debug Port Commands 104

23.1 System Memory Map for the F256 113
23.2 CPU Memory Map for the F256 (Banks 0–3) 114
23.3 CPU Memory Map for the F256 (Banks 4–7) 114
23.4 I/O Page 0 Addresses (0xC000–0xD67F) 115
23.5 I/O Page 0 Addresses (0xD680–0xDF13) 115
23.6 Memory Map for I/O Page 1 116

1 Introduction

This manual describes the hardware of the Wildbits Jr2 and K2 running the FNX6809 core: what each device does, which registers control it, and how to program it from 6809 assembly language. Each chapter covers one part of the machine and gives working examples.

The manual doesn't teach 6809 programming. The instruction set is the one used in the Tandy Color Computer, the Dragon and many other machines, and books written for those machines teach it well. The examples are commented so you can follow them if you are still learning.

1.1 The machines

The Wildbits Jr2 and K2 are the main subject of this manual.

Jr2 a single board computer in the mini-ITX format. It needs a PS/2 keyboard, a case and a power supply of your own.

K2 a complete computer with a case and a built-in keyboard. It has a few devices the Jr2 lacks: a second VIA that scans the built-in keyboard, an optical keyboard interface, a WizNet network interface, a supervisor microcontroller (an RP2040) and a small logo display.

Both machines run the same core and share almost all their hardware, so a program written for one runs on the other unless it uses a K2-only device. The text marks every difference.

> **Note:** The same core will be ported to the earlier F256jr and F256k. Most of this manual will apply to them, but they have less memory and fewer devices, so check the chapter for the device you use.

1.2 The 6809 and the core

Most of the machine lives in one FPGA. The core loaded into it, FNX6809, provides a cycle-accurate 6809 CPU along with VICKY, the video controller; the memory management unit (MMU); the sound chips; and most of the I/O devices. Because the CPU is part of the core, you change the CPU by loading a different core; this manual covers the 6809 core only. This edition describes core release RC24 ("Longview").

The CPU is a 6809, not a 6309: the 6309's extra registers and instructions aren't there. It runs at 6.29 MHz. With turbo mode on, cycles that don't use the bus (the internal cycles of an

instruction) run at 12.58 MHz, while every bus cycle keeps its normal timing, so programs run faster without upsetting any device. Turbo mode is on when the first boot-mode DIP switch is on; bit 0 of `$FF90` then reads 0. Check the DIP switch labels and the speed-up on both boards.

The hardware registers are big-endian, like the 6809: a register of two or more bytes keeps its most significant byte at the lowest address. You can load and store 16-bit values with `LDD` and `STD`, and a 24-bit address is a high byte followed by a 16-bit word.

1.3 The parts of the machine

The CPU addresses 64 KB through the MMU, which maps 8 KB blocks of the 2,016 KB of RAM and the device memory into eight slots. Chapter 2 explains the MMU; read it first, because every other chapter depends on it.

The rest of the hardware falls into these groups, each with its own chapter:

Video VICKY draws text, bitmaps, tiles and sprites, with a mouse pointer and line interrupts. It reads its data straight from RAM, without the CPU.

Sound the FPGA holds SID chips, an SN76489 (the PSG) and an OPL3, mixed through the sound CODEC. A VS1053b decodes compressed audio, and a SAM2695 synthesizer plays MIDI.

Timing two timers, the start-of-frame and start-of-line interrupts, and a battery-backed real-time clock.

Input PS/2 keyboard and mouse, two joystick ports through VIA 0, NES and SNES gamepads, and on the K2 the built-in keyboard.

Storage and communication two SD card interfaces, the serial port, the Commodore IEC serial bus, and Wi-Fi through a WizFi360 module.

Helpers a DMA engine for copying and filling memory, integer and floating-point math units, and a random number generator.

Interrupts from all these devices pass through one interrupt controller (chapter 9). The memory map appendix lists every register address.

1.4 Raw programs and NitrOS-9

You can program the machine in two ways, and most chapters give an example of each.

Raw 6809 your program runs on the bare machine, with no operating system. It owns all the hardware: it sets up the MMU, the screen and the interrupts itself. Write this way for games, demos and tests, or to learn how the hardware works.

NitrOS-9 your program runs as a process under NitrOS-9 Level 2, a multitasking operating system with files, a shell and device drivers. The system owns the MMU and the interrupt controller, so a program asks the system to map a device block before it touches a register, and it leaves interrupts to the drivers. Most software for these machines runs under NitrOS-9.

Where NitrOS-9 changes how you use a device, the chapter has a section on it. Use a NitrOS-9 built for the RC24 core: older builds expect the device blocks at different addresses.

1.5 The examples

Complete versions of the examples are in the `examples` folder of the manual's repository, https://github.com/jfed6000/Wildbits-F256-6809-Manual. The folder has three parts:

`examples/raw` raw programs. Start with `skeleton.asm`, which sets up the machine, clears the screen and prints a message; copy it to begin your own program. Its header explains how to build it and load it.

`examples/nos9` NitrOS-9 programs. Start with `hello.asm`. The README explains how to build them and copy them to a NitrOS-9 disk image.

`examples/include` `wildbits_regs.inc`, which defines a name for every register this manual documents. It is generated from the same data as the register tables in this manual, so the names match. NitrOS-9 programs use the names in NitrOS-9's own `wildbits.d` instead.

To load a raw program, connect the debug USB port to another computer and use FoenixMgr. Tell the machine to boot from RAM once, then upload the program:

```
fnxmgr --boot ram
fnxmgr --binary skeleton.bin --address 00E000
```

The upload stops the CPU, writes the program into RAM and restarts the CPU, which takes its reset vector from the program. `fnxmgr --boot flash` returns to the normal boot. Chapter 2 explains what the RAM boot changes, and the debug port chapter describes the protocol FoenixMgr uses.

1.6 Notation

Notes that call out a difference between machines, or something easy to get wrong, have a bar in the margin:

Note: Like this.

Numbers that start with $ are hexadecimal, and numbers that start with % are binary; other numbers are decimal. A range of addresses such as `$FE20`–`$FE2F` includes both ends. A bit is numbered from 0, the least significant bit, and `b7` means bit 7.

Addresses come in two kinds. A CPU address is one of the 64 KB the 6809 sees, written with four digits (`$FFA0`). A physical address is a place in the machine's memory, written with six digits (`$1F0000`), or as a block number and an offset (block `$FC`, offset `$1700`). The MMU turns one into the other.

The examples use the syntax of lwasm, the assembler in lwtools (http://www.lwtools.ca/). Anything that assembles 6809 code with Motorola syntax will need few changes. The parts of lwasm syntax the examples use are:

- `label equ value` gives a name to a number.
- `fcb` stores bytes, `fdb` stores 16-bit words (high byte first), `fcc` stores a string and `rmb` reserves bytes.

- `org` sets the address of the code that follows.
- An expression may use `+ - * /`, `&` (and), `|` (or), `^` (exclusive or) and `~` (complement). For example, `anda #~INT_VKY_SOF` clears the bit named `INT_VKY_SOF`.
- A label that starts with `@` is local: it is known only between the two ordinary labels around it.
- Text after the operand, or after a `*` or `;` at the start of a line, is a comment.

The raw examples build with `lwasm --6809 --format=raw`, which writes a plain binary image. The NitrOS-9 examples build with `--format=os9` and the same options as the NitrOS-9 sources; each example's Makefile has the full command.

2 Memory Management

The Jr2 and K2 have 2 MB of static RAM, 512 KB of flash memory, and a connector for 256 KB of expansion memory. The 6809 can address only 64 KB. To reach the rest, you use the memory management unit (MMU): it divides the CPU's 64 KB into eight slots of 8 KB and lets you choose which 8 KB block of physical memory appears in each slot.

If you write programs for NitrOS-9, the operating system manages the MMU for you; read section 2.8 for what that means for your program. If you write raw 6809 programs, read the whole chapter: your program owns the MMU.

2.1 Blocks

The MMU sees physical memory as 256 blocks of 8 KB, numbered $00 to $FF. A block number is eight bits, and an address within a block is thirteen, so a physical address is 21 bits: the block number times $2000, plus the offset.

Most blocks are RAM. Blocks $40–$9F depend on the FLASHDIS bit in MMU_IO_CTRL ($FFA1): while it is clear, as it is after reset, those blocks are the flash memory and the expansion connector. Set it, and they become RAM too. Blocks $FC–$FF are always device memory: VICKY's registers, fonts and color tables, and the text screen. Table 2.1 shows the layout.

Blocks	Physical addresses	FLASHDIS = 0 (reset)	FLASHDIS = 1
$00–$3F	$000000–$07FFFF	RAM (512 KB)	RAM
$40–$7F	$080000–$0FFFFF	Flash (512 KB)	RAM
$80–$9F	$100000–$13FFFF	Expansion (256 KB)	RAM
$A0–$FB	$140000–$1F7FFF	RAM (736 KB)	RAM
$FC–$FF	$1F8000–$1FFFFF	Devices	Devices

Table 2.1: Physical memory blocks

With FLASHDIS set you have 2,016 KB of RAM in blocks $00–$FB; with it clear you have 1,248 KB, plus the flash and the expansion memory. FLASHDIS changes only what the CPU sees. VICKY and the DMA engine use physical addresses directly, without the MMU, and always reach RAM at every block from $00 to $FB.

> **Note:** Older cores have no FLASHDIS bit; their device memory is at blocks $C0–$C3, and blocks $A0–$FB are not all RAM. Bit 7 of MMU_IO_CTRL always reads 1 on a core with FLASHDIS, so your program can test it before setting the bit.

2.2 Slots and LUTs

The CPU's 64 KB is divided into eight slots (table 2.2). The top three bits of a CPU address select the slot. The MMU looks up the block number for that slot and puts it in front of the remaining 13 bits to make the physical address (figure 2.1, where "bank" means slot).

Slot	A15–A13	CPU addresses
0	000	$0000–$1FFF
1	001	$2000–$3FFF
2	010	$4000–$5FFF
3	011	$6000–$7FFF
4	100	$8000–$9FFF
5	101	$A000–$BFFF
6	110	$C000–$DFFF
7	111	$E000–$FFFF

Table 2.2: CPU memory slots

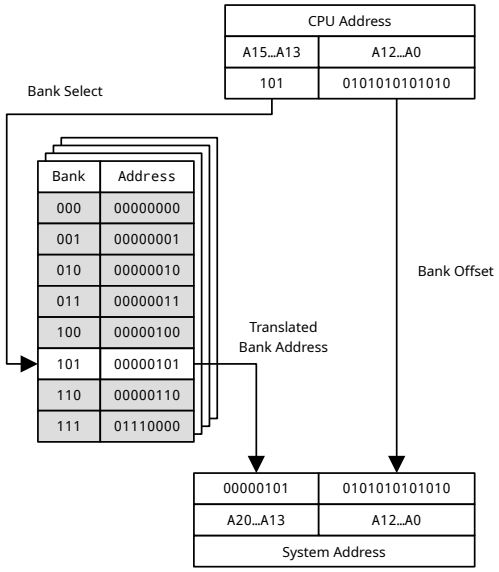


Figure 2.1: MMU address translation

The eight block numbers for the eight slots form a look-up table (LUT). The MMU holds four LUTs, so you can keep four complete memory maps and switch between them by writing one register. The ACT_LUT field of MMU_MEM_CTRL ($FFA0) selects the LUT that translates addresses, and the EDIT_LUT field selects the LUT that the slot registers at $FFA8–$FFAF read and write.

The two fields are independent: you can build a new map in one LUT while another one runs your program. When EDIT_LUT and ACT_LUT are the same, a write to a slot register changes the memory map at once.

Every slot can hold any block, including a device block. Slot 7 is different only because part of it is fixed, as the next section describes.

2.3 Fixed addresses

The top three pages of the CPU's address space behave the same in every LUT:

`$FE00`–`$FFFF` are I/O registers: the interrupt controller, timers, DMA, serial ports, VICKY's control registers and the rest (see the memory map appendix). The MMU registers are at `$FFA0`–`$FFAF`. Reads and writes in this range never reach the RAM block in slot 7, with one exception: `$FFF0`–`$FFFF`.

`$FFF0`–`$FFFF` hold the 6809's interrupt and reset vectors. With VEC_RAM (bit 1 of `MMU_IO_CTRL`) clear, they come from the block in slot 7, like ordinary memory. With VEC_RAM set, 16 bytes of RAM inside the FPGA replace them, so the vectors stay put whatever block you map into slot 7.

`$FD00`–`$FDFF` are ordinary memory in slot 7 while CONST_RAM (bit 0 of `MMU_IO_CTRL`) is clear. Set it, and 256 bytes of RAM inside the FPGA replace them in every LUT. Use this page for code and data that must stay visible whatever the MMU maps: interrupt entry code, or a routine that switches LUTs.

The constant RAM and the vector RAM start with unknown contents. To fill one, set its bit and write to it; with the bit set, your reads and writes go to the internal RAM and the block in slot 7 is untouched.

2.4 Device blocks

Blocks `$FC`–`$FF` hold the larger VICKY memories (table 2.3). To reach one, map it into a free slot, read or write it, and map the slot's usual block back. The registers inside each block are described in the chapters on text, graphics, sprites and tiles.

2.5 The MMU registers

Table 2.4 shows the MMU registers, and the list after it describes each one.

MMU_MEM_CTRL (`$FFA0`, read/write, reset `$00`) Selects the LUT that translates CPU addresses and the LUT that the slot registers edit.

EDIT_LUT (bits 5–4) LUT (0–3) that the slot registers at $FFA8–$FFAF read and write.

ACT_LUT (bits 1–0) LUT (0–3) that translates CPU addresses.

Block	Offsets	Contents
$FC	$0000–$0BFF	Gamma tables (blue, green, red)
	$0C00–$0FFF	Mouse pointer
	$1000–$10FF	Bitmap registers
	$1100–$11FF	Tile registers
	$1200–$12FF	Memory text mode registers
	$1300–$16FF	Sprite registers
	$1700–$173F	Text foreground color LUT
	$1740–$177F	Text background color LUT
$FD	$0000–$0FFF	Fonts (two sets of 2 KB)
	$1000–$1FFF	Graphics color LUTs 0–3 (1 KB each)
$FE	$0000–$12BF	Text screen characters (4,800 bytes)
$FF	$0000–$12BF	Text screen colors (4,800 bytes)

Table 2.3: Device blocks

Address	Name	7	6	5	4	3	2	1	0
$FFA0	MMU_MEM_CTRL	—		EDIT_LUT		—		ACT_LUT	
$FFA1	MMU_IO_CTRL	FLASHDIS_OK	—				FLASHDIS	VEC_RAM	CONST_RAM
$FFA8–$FFAF	MMU_SLOT_0–7	Block number							

Table 2.4: MMU registers

MMU_IO_CTRL ($FFA1, read/write, reset $00) Controls the constant RAM pages and the flash and expansion windows. Change it with a read-modify-write, so that you keep the bits other code has set.

FLASHDIS_OK (bit 7, read-only) Always reads 1 on a core that implements FLASHDIS. Older cores read back the stored 0.

FLASHDIS (bit 2) 1 = blocks $40–$9F are RAM. 0 = blocks $40–$7F are flash and $80–$9F are the expansion connector.

VEC_RAM (bit 1) 1 = 16 bytes of internal RAM replace $FFF0–$FFFF (the 6809 vectors), in every LUT. 0 = the vectors come from the block mapped in slot 7.

CONST_RAM (bit 0) 1 = 256 bytes of internal RAM replace $FD00–$FDFF, in every LUT. 0 = $FD00–$FDFF is ordinary mapped memory.

MMU_SLOT_0–7 ($FFA8–$FFAF, read/write) Slot registers 0–7: the block each 8 KB slot maps, in the LUT selected by EDIT_LUT. At reset LUT 0 maps slots 0–7 to blocks 0–7, except slot 7, which maps flash block $7F (or RAM block 7 after a debug-port RAM boot). Reset doesn't set LUTs 1–3.

2.6 Reset and boot

Power-up and the reset button both reset the MMU:

- MMU_MEM_CTRL and MMU_IO_CTRL are $00: LUT 0 is active and being edited, FLASHDIS is clear, and the constant RAM and vector RAM are off.
- LUT 0 maps slots 0–6 to RAM blocks 0–6, and slot 7 to flash block $7F, the last 8 KB of flash. The CPU fetches its reset vector from $FFFE in that block, so the machine boots from flash.
- LUTs 1–3 keep whatever they held. Set every slot of a LUT before you make it active.

For development you can boot from RAM instead. The FoenixMgr command `--boot ram` sends a command over the USB debug port that makes every later reset of the CPU map RAM block 7 into slot 7 instead of flash block $7F. Each FoenixMgr upload resets the CPU this way, so it also starts your program. The setting lasts until you send `--boot flash`, power the machine off, or press the reset button. Your program then supplies its own vectors at $FFF0–$FFFF of block 7. The raw skeleton in `examples/raw` works this way; its comments describe the steps.

2.7 Raw 6809 examples

These examples use the names from `examples/include/wildbits_regs.inc`, which the manual's register tables are generated from.

Example: Reaching a device block

This fragment maps the text color block ($FF) into slot 2, fills the screen's colors with white on color 0, and puts RAM block 2 back. It assumes that LUT 0 is both active and being edited, as after reset, and that your program and stack aren't in slot 2.

```
            lda   #$FF              text color block
            sta   MMU_SLOT_2        slot 2 ($4000-$5FFF) now shows the colors
            ldx   #$4000
            ldd   #$F0F0            foreground 15, background 0
            ldy   #4800/2           4,800 bytes, two at a time
@fill       std   ,x++
            leay  -1,y
            bne   @fill
            lda   #2                RAM block 2
            sta   MMU_SLOT_2        put it back
```

Example: Building and switching to a second map

This fragment fills LUT 1 while LUT 0 keeps running the program, then switches to LUT 1. LUT 1 maps the same blocks as LUT 0 after a RAM boot (0–7), except that slot 3 shows RAM block $A0. Keep the slot that holds your running code the same in both LUTs, or the CPU fetches its next instruction from somewhere else.

```
            lda   #$10              edit LUT 1, LUT 0 stays active
            sta   MMU_MEM_CTRL
            ldx   #MMU_SLOT_0
```

```
            clrb
@slots      stb   ,x+              slot n = block n
            incb
            cmpb  #8
            bne   @slots
            lda   #$A0
            sta   MMU_SLOT_3       slot 3 = block $A0
            lda   #$01             make LUT 1 active (and edit LUT 0)
            sta   MMU_MEM_CTRL
```

2.8 Under NitrOS-9

NitrOS-9 Level 2 takes over the MMU when it boots:

- It sets FLASHDIS on a core that has it, so all 2,016 KB of RAM is available to the system and to programs.
- It sets CONST_RAM and VEC_RAM and keeps its own code in the constant page at `$FD00` and its vectors in the vector RAM.
- LUT 0 holds the system's map. LUT 1 holds the map of the running process; NitrOS-9 reloads it on every task switch.

Each process gets its own 64 KB address space, built from the blocks NitrOS-9 allocates to it. `$FD00`–`$FFFF` belong to the system in every process; the I/O registers are still at `$FE00`–`$FFFF`, but your program can't use that range as memory.

Don't write the MMU registers from a NitrOS-9 program: the next task switch undoes your changes, and a wrong slot crashes the system. Ask NitrOS-9 instead. `F$MapBlk` maps one or more blocks, such as a device block, into a free part of your address space and returns its address; `F$ClrBlk` unmaps them. `examples/nos9/bgcolor.asm` uses the two calls to reach the text color LUTs in block `$FC`:

```
            pshs  u                F$MapBlk returns the address in U
            ldx   #TEXT_LUT_BLK    block $FC (from wildbits.d)
            ldb   #1               one block
            os9   F$MapBlk
            bcs   fail
            stu   <mapaddr         the data area is still reachable through DP
            puls  u
            ...                    use the block through mapaddr
            ldu   <mapaddr
            ldb   #1
            os9   F$ClrBlk         give the address range back
```

Use the block names that NitrOS-9 defines in `wildbits.d`, such as `VICKY_BLK` and `COLOR_RAM_BLK`, rather than numbers. To get more memory for data, use `F$Mem` to grow your data area, or `F$AllRAM` and `F$MapBlk` to allocate blocks and map them yourself.

3 The Text Screen

The display on the F256 is managed by the video controller of the TinyVicky chip. The display controller provides four different engines for generating images on the screen:

- Text: an old school style text screen where the characters to display are stored in a text matrix, and the shape of those characters comes from font memory. Text mode characters are 8 pixels wide by 8 pixels high.
- Bitmap: a simple pixel image engine that can display a bitmap image covering the screen.
- Sprite: an engine to display small, movable sprites on the screen.
- Tile: an engine to display images on the screen made up of tiles from a tile set.

The bitmap, sprite, and tile engines are considered graphics modes. TinyVicky will let you display either text by itself, a mix of the graphics modes by themselves, or text overlaid on top of the graphics modes.

Text Matrix

The memory for the characters to display on the screen is the text matrix, which is stored in I/O page 2. When this I/O page is swapped into the CPU address space, it appears at 0x18_4000. Each byte of memory corresponds to a single character on the screen in left to right, top to bottom order. The byte at 0x18_4000 (Block $C2 Offset $0000) is the upper left corner of the screen, the byte at 0x18_4001 is the next character to the right, and so on. The number of bytes per line is set by the base resolution of the screen, but is generally 80. When a border is displayed, while that limits the number of characters displayed, the layout in memory remains the same.

The text screen has two core resolutions, tied to the refresh rate of the screen: 80 by 60 at 60 Hz, and 80 by 50 at 70 Hz. Beyond that, the character display may be made double width or double height, or both. This gives the following possible character displays: 80×60, 40×60, 80×30, 40×30, 80×50, 40×50, 80×25, and 40×25.

Example: Print an A to the Screen

```
lda $0001       ; Save the current MMU setting
pha
```

```
lda #$02        ; Swap I/O Page 2 into bank 6
sta $0001

lda #'A'        ; Write 'A' to the upper left corner
sta $C000

pla             ; Restore the old MMU setting
sta $0001
```

Note: this example does not set the font or the color, so depending on how your F256 is initialized, you may not see an actual "A" on the screen.

Text Color LUTs

Characters in TinyVicky text mode have two colors: the foreground and the background. The foreground and background colors are picked for each character out of two different palettes of 16 colors each. The colors in the palettes are picked from the full range of colors F256 can produce, which is more than 16 million colors. This is all managed through two color lookup tables (LUTs) provided by TinyVicky: a text foreground color LUT, and a text background color LUT.

The text LUTs are stored in I/O page 0. The foreground LUT starts at 18x3800, and the background LUT starts at 18x3840 (Block $C1 0x1800 and Block $C1 0x1840).

Each LUT is a list of 16 entries. Each entry is a set of four bytes: blue, green, red, and alpha (in that order). Each byte indicates how much of that primary color is present as a component of the actual color. The values range from 0 (none) to 255 (as much as possible). Currently, the alpha channel is not used and is there for future expansion.

Color Matrix

The way that text color is selected for each character is through the color matrix. This section of memory is in I/O page 3 and starts at 0x18_6000 (Block $C3 Offset $0000) when page 3 is swapped into the CPU's address space. The layout is precisely the same as the text matrix (*e.g.* the character at 0x18_4023 in the text matrix has its color information at 0x18_6023 in the color matrix).

Each byte in the color matrix specifies two colors by providing an index into each of the two text LUTs. The most significant four bits is the number of the foreground color to use. The number of the least significant four bits is the number of the background color to use.

Let's say the color value at 0x18_6023 is 0x45. This means that the foreground color of the character is color 4 from the text foreground LUT, which starts at 0xFF10 (0xFF00 + 4 * 4), and the background color of the character is 5 from the text background LUT, which starts at 0xFF54 (0xFF40 + 4 * 5). If the bytes at 0xFF10 are 0x00, 0x80, 0x80, that means the foreground will be a medium yellow. If the bytes at 0xFF54 are 0xFF, 0x00, 0x00, that means the background will be blue.

Index	R/W	Foreground	Background	0	1	2	3
0	W	18x3800	18x3840	BLUE_0	GREEN_0	RED_0	X
1	W	18x3804	18x3844	BLUE_1	GREEN_1	RED_1	X
2	W	18x3808	18x3848	BLUE_2	GREEN_2	RED_2	X
3	W	18x380C	18x384C	BLUE_3	GREEN_3	RED_3	X
4	W	18x3810	18x3850	BLUE_4	GREEN_4	RED_4	X
5	W	18x3814	18x3854	BLUE_5	GREEN_5	RED_5	X
6	W	18x3818	18x3858	BLUE_6	GREEN_6	RED_6	X
7	W	18x381C	18x385C	BLUE_7	GREEN_7	RED_7	X
8	W	18x3820	18x3860	BLUE_8	GREEN_8	RED_8	X
9	W	18x3834	18x3864	BLUE_9	GREEN_9	RED_9	X
10	W	18x3828	18x3868	BLUE_10	GREEN_10	RED_10	X
11	W	18x382C	18x386C	BLUE_11	GREEN_11	RED_11	X
12	W	18x3830	18x3870	BLUE_12	GREEN_12	RED_12	X
13	W	18x3834	18x3874	BLUE_13	GREEN_13	RED_13	X
14	W	18x3838	18x3878	BLUE_14	GREEN_14	RED_14	X
15	W	18x383C	18x387C	BLUE_15	GREEN_15	RED_15	X

Table 3.1: Text Color Lookup Tables

Example: Make That "A" Yellow on Blue

```
lda $0001       ; Save the MMU state
pha

stz $0001       ; Switch in I/O Page #0

stz $D810       ; Set foreground #4 to medium yellow
lda #$80
sta $D811
sta $D812

lda #$FF        ; Set background #5 to blue
sta $D854
stz $D855
stz $D856

lda #$03        ; Switch to I/O page #3 (color matrix)
sta $0001

lda #$45        ; Color will be foreground=4, background=5
sta $C000

pla             ; Restore the MMU state
sta $0001
```

Index	R/W	Foreground	Background	0	1	2	3
0	W	0x1800	0x1840	BLUE_0	GREEN_0	RED_0	X
1	W	0x1804	0x1844	BLUE_1	GREEN_1	RED_1	X
2	W	0x1808	0x1848	BLUE_2	GREEN_2	RED_2	X
3	W	0x180C	0x184C	BLUE_3	GREEN_3	RED_3	X
4	W	0x1810	0x1850	BLUE_4	GREEN_4	RED_4	X
5	W	0x1814	0x1854	BLUE_5	GREEN_5	RED_5	X
6	W	0x1818	0x1858	BLUE_6	GREEN_6	RED_6	X
7	W	0x181C	0x185C	BLUE_7	GREEN_7	RED_7	X
8	W	0x1820	0x1860	BLUE_8	GREEN_8	RED_8	X
9	W	0x1824	0x1864	BLUE_9	GREEN_9	RED_9	X
10	W	0x1828	0x1868	BLUE_10	GREEN_10	RED_10	X
11	W	0x182C	0x186C	BLUE_11	GREEN_11	RED_11	X
18	W	0x1830	0x1870	BLUE_18	GREEN_18	RED_18	X
18	W	0x1834	0x1874	BLUE_18	GREEN_18	RED_18	X
14	W	0x1838	0x1878	BLUE_14	GREEN_14	RED_14	X
15	W	0x183C	0x187C	BLUE_15	GREEN_15	RED_15	X

Table 3.2: Text Color Lookup Tables Relative to Block $C1

Entering Text Mode

Whether text mode is being displayed (and in what resolution) is controlled by the VICKY Master Control Registers (see table 3.3). For now, we're going to ignore most of the bits, which are used by other display modes. For text mode, we really only care about the TEXT bit, which needs to be set to turn on the text display. The resolution is controlled by DBL_Y, DBL_X, and CLK_70. If we set 0xFFC0 to 0x01 and 0xFFC1 to 0x00, that will put us into text mode at 80×60.

Address	R/W	7	6	5	4	3	2	1	0
0xFFC0	R/W	DIS_VID	GAMMA	SPRITE	TILE	BITMAP	GRAPH	OVRLY	TEXT
0xFFC1	R/W	SHWBG	MTXT_EN	FON_SET	FON_OVLY	MON_SLP	DBL_Y	DBL_X	CLK_70

Table 3.3: VICKY Master Control Registers

TEXT if set (1), text mode display is enabled

OVRLY if set, text will be overlaid on graphics

GRAPH if set, one or more of the graphics modes may be used

BITMAP if set (and GRAPHICS is set), bitmap graphics may be displayed

TILE if set (and GRAPHICS is set), tile graphics may be displayed

SPRITE if set (and GRAPHICS is set), sprite graphics may be displayed

GAMMA if set, gamma correction is enabled

DIS_VID if set, disable video scanning, giving 100% bandwitth to CPU

CLK_70 if set, the video refresh will be set to 70 Hz mode (640x400 text resolution, 320x200 graphics). If clear, the video refresh will be set to 60 Hz (640x480 text resolution, 320x240 graphics).

DBL_X if set, text mode characters will be twice as wide

DBL_Y if set, text mode characters will be twice as high

MON_SLP if set, the monitor SYNC signal will be turned off, putting the monitor to sleep

FON_OVLY if clear (0), only the text foreground color will be displayed when text overlays graphics (all background colors will be completely transparent). If set (1), both foreground and background colors will be displayed, except that background color 0 will be transparent.

FON_SET if set (1), the text font displayed will be font set 1. If clear (0), the text font displayed will be font set 0.

MTXT_EN if set, enable Memtext mode (prioritized over block text)

SHWBG if set, show background in non-overlay mode in MemText mode

Text Fonts

Character shapes (or "glyphs," if you prefer) are defined in font memory, which is in I/O page 1 and starts at 0x18_2000 (Block $C1 Offset $0000). TinyVicky provides for two font sets, and which one is used for text mode is controlled by the FON_SET bit in the Vicky Master Control Register. Only one will be in use at any given time in normal operation. Font set 0 is from 0x18_2000 through 0x1827FF. Font set 1 is from 0x18_2800 through 0x182FFF (Block $C2 Offset $0800).

The F256 treats each character as a square of pixels, 8 pixels on a side. A pixel may be either in the foreground color for the character or in the background color for the character. The way this is managed is that each character has a sequence of eight bytes in the font memory. Each byte represents a row in the character, and each bit represents a pixel in the row (■ for foreground, □ for background).

As an example, let's say we wanted to have a fancy "F" for character 0:

□	□	□	■	■	■	■	■	0x1F
□	□	■	■	□	□	□	□	0x30
□	□	■	■	□	□	□	□	0x30
□	■	■	■	■	■	□	□	0x7C
□	■	■	□	□	□	□	□	0x60
■	■	□	□	□	□	□	□	0xC0
■	■	□	□	□	□	□	□	0xC0

Table 3.4: A sample character

The glyph to display would be defined by the eight byte sequence 0x1F, 0x30, 0x30, 0x7C, 0x60, 0xC0, 0xC0. We would store that sequence starting at 0x18_2000 (0x1F), through 0x18_2007 (0xC0). After that was set, any time the byte 0x00 is written to screen memory, the glyph "F" would be displayed in that position.

Text Cursor

F256 has a text mode cursor. The text mode cursor is implemented as a character which is displayed in a (x, y) position on the screen, visually replacing the character ordinarily at that position. It may be displayed continuously, or it may flash at one of four rates. When flashing, that position in the text screen will alternate between the text cursor and the character at that position in the text matrix. The color for the text cursor comes from the color for the position on the screen as specified in the color matrix. In other words, the text cursor does not have its own color. The text cursor registers are located in I/O bank 0.

Address	R/W	Name	7	6	5	4	3	2	1	0
0xFFD0	R/W	CCR	—				FLASH_DIS	RATE		ENABLE
0xFFD2	R/W	CCH	Cursor character							
0xFFD3	R/W	CCO	Cursor Color							
0xFFD4	R/W	CURX	X15	X14	X13	X12	X11	X10	X9	X8
0xFFD5	R/W		X7	X6	X5	X4	X3	X2	X1	X0
0xFFC6	R/W	CURY	Y15	Y14	Y13	Y12	Y11	Y10	Y9	Y8
0xFFD7	R/W		Y7	Y6	Y5	Y4	Y3	Y2	Y1	Y0

Table 3.5: Text Cursor Registers

ENABLE if this flag is set (1), the cursor is enabled

FLASH_DIS if this flag is set (1), the cursor will not flash. If clear (0), it will flash.

RATE these two bits set the rate at which the cursor flashes (see table 3.6)

CCH the character code for the cursor character to display

CURX the column number (16-bit) for the cursor

CURY the row number (16-bit) for the cursor

RATE1	RATE0	Rate
0	0	1s
0	1	1/2s
1	0	1/4s
1	1	1/5s

Table 3.6: Text Cursor Flash Rates

MemText

MemText mode features include: four 8x8 text fonts or two 8x16 text fonts, two foreground CLUTs and two background CLUTs of 256 colors each, and 256 colors per character. MemText text mode keeps the text and color data in RAM, which allows the use of DMA to copy data and to switch text screen by changing the pointer values to a different value in memory.

MemText Registers located in block $C0

Address	R/W	Name	7	6	5	4	3	2	1	0
18x1200	R/W	MTCR	—						FSIZE	MTXT_EN
18x1201	R/W	CCR	X	CSIZE	TF_1	TF_0	F_ON	CF_1	CF_0	CURS_EN
18x1202	R/W	CURX	X7	X6	X5	X4	X3	X2	X1	X0
18x1203	R/W	CURY	Y7	Y6	Y5	Y4	Y3	Y2	Y1	Y0
18x1204	R/W	N/A	NOT USED							
18x1205	R/W	TXTH	T23	T22	T21	T20	T19	T18	T17	T16
18x1206	R/W	TXTM	T15	T14	T13	T12	T11	T10	T9	T8
18x1207	R/W	TXTK	T7	T6	T5	T4	T3	T2	T1	T0
18x1208	R/W	N/A	NOT USED							
18x1209	R/W	COLH	C23	C22	C21	C20	C19	C18	C17	C16
18x120A	R/W	COLM	C15	C14	C13	C12	C11	C10	C9	C8
18x120B	R/W	COLL	C7	C6	C5	C4	C3	C2	C1	C0
18x120C	R/W	PRE	Pre-Charge (Reserved)							
18x120D	R/W	CCB	Cursor Color B							
18x120E	R/W	CCG	Cursor Color G							
18x120F	R/W	CCR	Cursor Color R							
18x1210	R/W	CG8	Cursor Graphic x8							
18x1211	R/W	CG8	Cursor Graphic x8							
18x1212	R/W	CG8	Cursor Graphic x8							
18x1213	R/W	CG8	Cursor Graphic x8							
18x1214	R/W	CG8	Cursor Graphic x8							
18x1215	R/W	CG8	Cursor Graphic x8							
18x1216	R/W	CG8	Cursor Graphic x8							
18x1217	R/W	CG8	Cursor Graphic x8							
18x1218	R/W	CG16	Cursor Graphic x16							
18x1219	R/W	CG16	Cursor Graphic x16							
18x121A	R/W	CG16	Cursor Graphic x16							
18x121B	R/W	CG16	Cursor Graphic x16							
18x121C	R/W	CG16	Cursor Graphic x16							
18x121D	R/W	CG16	Cursor Graphic x16							
18x121E	R/W	CG16	Cursor Graphic x16							
18x121F	R/W	CG16	Cursor Graphic x16							

Table 3.7: MemText Registers

Screen resolutions are 80x60 for 8x8 fonts and 80x30 for 8x16 fonts. The double pixel registers do not affect MemText mode.

MTCR Memory Text Control Register

MTXT_EN 1=enabled, 0=disabled

FSIZE Font Size: 0=8x8, 1=8x16 (State-Machine)

CCR Cursor Control Register

CURS_EN 1=enabled, 0=disabled

CF_0 Cursor Flashing Rate 0

CF_1 Cursor Flashing Rate 1

F_ON Cursor Flashing on/off

TF_0 Text Flashing Rate 0 (triggered by attributes)

TF_1 Text Flashing Rate 1 (triggered by attributes)

CSIZE Cursor Size: 0=8x8, 1=8x16

RATE1	RATE0	Rate
0	0	1bps
0	1	2bps
1	0	3bps
1	1	5bps

Table 3.8: Text Cursor Flash Rates

CURX Cursor X Register

CURY Cursor Y Register

TXTH Text Address Pointer H

TXTM Text Address Pointer M

TXTL Text Address Pointer L

COLH Color Address Pointer H

COLM Color Address Pointer M

COLL Color Address Pointer L

PRE Pre-Charge (Reserved - defaulted to $0A)

CCB Cursor Color B

CCG Cursor Color R

CCR Cursor color R

CG8 Cursor Graphics x8

CG16 Cursor Graphics x16

MemText Foreground/Background LUTs

The foreground and background LUTs consist of 256 colors in BGRA format. Each LUT is 1K.

Block	Offset	Address Start	Address End	Description
$C5	0x0000	18xA000	18xA3FF	MEMTEXT FOREGROUND - LUT0
$C5	0x0400	18xA400	18xA7FF	MEMTEXT FOREGROUND - LUT0
$C5	0x0800	18xA800	18xABFF	MEMTEXT FOREGROUND - LUT0
$C5	0x0C00	18xAC00	18xAFFF	MEMTEXT FOREGROUND - LUT0

Table 3.9: MemText Foreground/Background LUTS

MemText Fonts

Block	Offset	Address Start	Address End	Description
$C6	0x0000	18xC000	18xDFFF	MEMTEXT FONT (2x8x16),(4x8x8)

Table 3.10: MemText Registers

MemText Attributes

The text memory for MemText is 2 bytes per character. The first byte is the character, the second byte contains the attributes.

Character	Attributes							
7:0	7	6	5	4	3	2	1	0
Character	X	X	FL_EN	INV	BGLUT	FGLUT	FSEL1	FSEL0

Table 3.11: MemText Registers

FSEL1 and FSEL0 selects the font bank. If 8x16 font, then only FSEL0 is used to select one of two font banks. For 8x8, FSEL0 and FSEL1 are used to select one of four font banks

FGLUT select foreground LUT0 or LUT1

INV inverse character

FL_EN 1=enable flashing, 0=disable flashing

4 Graphics

The F256 provides three separate graphics engines, giving programs a choice in how they display information to the user. Those different engines do share certain features, however, and this chapter will cover the common elements. The three graphics engines are bitmaps, tile maps, and sprites. What is common between all these elements is how they decide what colors to display and how to resolve, when two or more objects are in the same place, which object is displayed.

- Bitmaps are simple raster images. They are the size of the screen (320×200 or 320×240) and cannot be moved. The TinyVicky chip used by the F256 allows for three separate bitmaps to be displayed at the same time.
- Tile maps are images made up of tiles. The tiles come in a tile set, which is a raster image like a bitmap but provides 256 tiles. The tile map itself creates its image by indicating which tile is displayed at every position in the tile map. This mapping can be changed on the fly, allowing tile maps to be altered, and tile maps can also be scrolled horizontally and vertically to a limited degree. This allows for possibility for smooth scrolling of a tile map scene. TinyVicky allows for three separate tile maps to be displayed simultaneously.
- Sprites are small, square graphic elements that may be moved to any position on the screen. Sprites are typically used to represent game characters or very mobile UI elements. TinyVicky sprites may be 8, 16, 24, or 32 pixels on a side. There may be as many as 64 sprites active on the screen at once (without using special techniques).

Graphics Colors

The graphics modes use a color lookup system similar to text mode to determine colors. The pixel data for a tile, bitmap, or sprite is composed of bytes, where each byte specifies the color of that pixel. The byte serves as an index into a color lookup table where the red, green, and blue components of the desired color are stored (see figure: 4.1). As with text, the color components are bytes and specify an intensity from 0 (none of that primary color) to 255 (as much of that primary color as possible). Also, as with text, there is a fourth byte that is reserved for future use, meaning that each color takes up four bytes in the CLUT. In short, the byte order of a graphics CLUT entry is exactly the same as for a text CLUT.

However, there is a key difference from text mode. In text mode, there are two colors (foreground and background), and each color is one out of sixteen possibilities. With graphics modes,

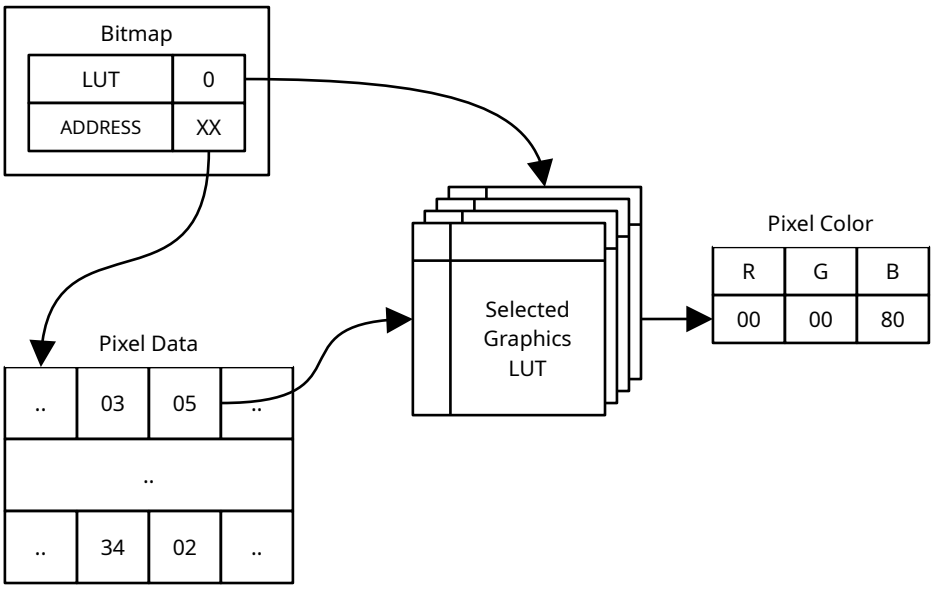


Figure 4.1: Bitmap Data to Pixels

there are 256 possibilities. So a CLUT with only 16 entries will not work. There are therefore separate CLUTs for graphics. TinyVicky provides for four separate graphics CLUTs with 256 entries. Each graphic object on the screen specifies which graphics CLUT it will use for its colors. These CLUTs may be found in I/O page 1 (see table: 4.1).

Block	Offset	Address	R/W	Purpose
$C1	$1000	0x18_3000	R/W	Graphics CLUT 0
$C1	$1400	0x18_3400	R/W	Graphics CLUT 1
$C1	$1800	0x18_3800	R/W	Graphics CLUT 2
$C1	$1C00	0x18_3C00	R/W	Graphics CLUT 3

Table 4.1: Graphics Color Lookup Tables

Example: A Simple Gradient

Let's set up a CLUT so that we have the colors for a gradient fill between red and blue. In this example, `pointer` is a two byte variable down in zero page, which will be used to point to the first byte of the CLUT entry the code is updating. The `Y` register is being used to point to the individual components of the entry.

```
MMU_IO_CTRL = $0001             ; MMU I/O Control Register
VKY_GR_CLUT_0 = $D000           ; Graphics LUT #0

;
; Initialize the LUT to greyscale from (255, 0, 0) to (0, 0, 255)
;
```

```
            lda #$01            ; Set the I/O page to #1
            sta MMU_IO_CTRL

            lda #<VKY_GR_CLUT_0 ; pointer to a particular LUT entry
            sta pointer
            lda #>VKY_GR_CLUT_0
            sta pointer+1

            ldx #0              ; Start with blue = 0

lut_loop:   ldy #0              ; And start at the offset for blue
            txa                 ; Take the current blue color level
            sta (pointer),y     ; Set the blue component
            iny

            lda #0
            sta (pointer),y     ; Set the green component to 0
            iny

            txa                 ; Get the blue component again
            eor #$ff            ; And compute the 2's complement of it
            inc a
            sta (pointer),y     ; Set the red component
            iny

            inx                 ; Go to the next color
            beq lut_done        ; If we are back to black, we're done

            clc                 ; Move pointer to the next LUT entry (+ 4)
            lda pointer
            adc #4
            sta pointer
            lda pointer+1
            adc #0
            sta pointer+1

            bra lut_loop

lut_done:
```

Pixel Data

All three graphics engines arrange their pixel data in the same manner. They all use rectangular raster images as a base, although the width and height of the rectangle can vary. The pixels are placed in memory in sequential order in left-to-right and top-to-bottom order. That is, the

first pixel in the sequence is the upper-left pixel in the image. The next pixel is the pixel to the immediate right and so on. If the image size is $w \times h$, the position of a pixel at (x, y) in the list is $y \times w + x$.

> **Note:** Pixel data for all graphics modes can be stored anywhere in the initial system RAM. That is, graphics data can be stored anywhere in the first 512KB of memory. The graphics engine cannot access graphics data in flash or the expansion memory.

Graphics Layers

Now, what happens if two sprites take up the same position or if a program displays a tile map and a bitmap together? How does TinyVicky determine what color to display at a given position? TinyVicky provides a flexible layering system with several layers. Elements in "near" layers (lower numbers) get displayed on top of elements in "far" layers (higher numbers). If a sprite in layer 0 says a pixel should be blue while a tile in layer 1 says it should be red, the pixel will be blue. Color 0, however, is special. It is always the transparent "color". A pixel that is 0 in an element will be the color of whatever is behind it (or the global background color, if there is nothing behind it) see table 7.3.

TinyVicky provides for seven layers, but they are split up a bit. Three of the layers are for bitmaps and tile maps. Only one bitmap or tile map can be placed in any of those three layers. The other four layers are for sprites only. Any sprite can be assigned to any of the sprite layers, and there can be multiple sprites in a layer. The sprite layers are interleaved with the bitmap and tile map layers (see figure: 4.2).

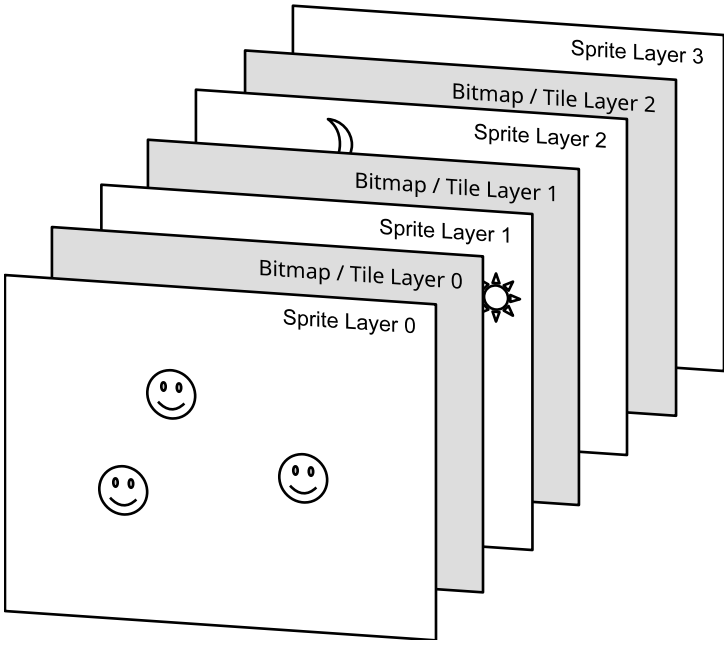


Figure 4.2: TinyVicky Graphic Layers

Bitmaps and tile maps are assigned to their layers using the layer control registers (see table: 4.2). The three fields LAYER0, LAYER1, and LAYER2 in the layer registers are three bit values, which indicate which graphical element to assign to that layer (see table: 4.3).

Address	R/W	7	6	5	4	3	2	1	0
0xFFC2	R/W	—	LAYER1			—	LAYER0		
0xFFC3	R/W	—					LAYER2		

Table 4.2: Bitmap and Tile Map Layer Registers

Code	Layer
0	Bitmap Layer 0
1	Bitmap Layer 1
2	Bitmap Layer 2
4	Tile Map Layer 0
5	Tile Map Layer 1
6	Tile Map Layer 2

Table 4.3: Bitmap and Tile Map Layer Codes

Example: Put Bitmap 0 on Layer 0

As an example of how to use layers, we can set things up for future examples by putting bitmap 0 in the front layer (0), tile map 0 in the next layer (1), and bitmap 1 in the back layer (2).

```
lda #$40             ; Layer 0 = BM 0, Layer 1 = TM 0
sta VKY_LAYER_CTRL_0
lda #$01             ; Layer 2 = BM 1
sta VKY_LAYER_CTRL_1
```

Bitmaps

TinyVicky allows for three full screen bitmaps to be displayed at once. These bitmaps are either 320×200 or 320×240, depending on the value of the CLK_70 bit of the master control register. A bitmap's pixel data contains either 64,000 bytes, or 76,800 bytes of data. In both cases, the pixel data is arranged from left to right and top to bottom. The first 320 bytes are the pixels of the first line (with the first pixel being the left-most). The second 320 bytes are the second line, and so on. Additionally, the bitmaps can independently use any of the four graphics CLUTs to specify the colors for those indexes. TinyVicky provides registers for each bitmap set the CLUT and the address of the bitmap:

ENABLE if set and both graphics and bitmaps are enabled in the Vicky Master Control Register (see table 3.3), then this bitmap will be displayed.

CLUT sets the graphics color lookup table to be used for this bitmap

AD give the address of the first byte of the pixel data within the 512KB system RAM. Note that this address is relative to the system bus of 21 bits and is not based on the CPU's addressing.

To set up and display a bitmap, the following things need to be done. The order is not terribly important, although updates to the bitmap's pixel data after the bitmap is displaying will be visible. That could be desirable, depending on what the program is doing.

Blk	Offset	Address	R/W	BM	7	6	5	4	3	2	1	0
$C0	$1000	0x18_1000	R/W	0	—					CLUT		ENABLE
$C0	$1001	0x18_1001	R/W		—					AD18	AD17	AD16
$C0	$1002	0x18_1002	R/W		AD15	AD14	AD13	AD12	AD11	AD10	AD9	AD8
$C0	$1003	0x18_1003	R/W		AD7	AD6	AD5	AD4	AD3	AD2	AD1	AD0
$C0	$1008	0x18_1008	R/W	1	—					CLUT		ENABLE
$C0	$1009	0x18_1009	R/W		—					AD18	AD17	AD16
$C0	$100A	0x18_100A	R/W		AD15	AD14	AD13	AD12	AD11	AD10	AD9	AD8
$C0	$100B	0x18_100B	R/W		AD7	AD6	AD5	AD4	AD3	AD2	AD1	AD0
$C0	$1010	0x18_1110	R/W	2	—					CLUT		ENABLE
$C0	$1011	0x18_1112	R/W		—					AD18	AD17	AD16
$C0	$1012	0x18_1112	R/W		AD15	AD14	AD13	AD12	AD11	AD10	AD9	AD8
$C0	$1013	0x18_1113	R/W		AD7	AD6	AD5	AD4	AD3	AD2	AD1	AD0

Table 4.4: Bitmap Registers

1. Enable bitmap graphics in the TinyVicky Master Control Register (see table: 3.3). This means you need to set both the GRAPH and BITMAP bits and either clear TEXT or set the OVRLY to display text and bitmap together.
2. Set up the pixel data for the bitmap somewhere in the first 512KB of RAM.
3. Set the address of the bitmap's pixel data in the AD field.
4. Assign the bitmap to a layer using the layer control registers (see table: 4.2).
5. Set the bitmap's CLUT and ENABLE bit in its control register.

Example: Display a Bitmap

This example will build on the previous examples of setting up the CLUT and display a gradient on the screen. First, it needs to turn on the bitmap graphics:

```
MMU_MEM_CTRL = $0000            ; MMU Memory Control Register
MMU_IO_CTRL = $0001             ; MMU I/O Control Register
VKY_MSTR_CTRL_0 = $D000         ; Vicky Master Control Register 0
VKY_MSTR_CTRL_1 = $D001         ; Vicky Master Control Register 1
VKY_BM0_CTRL = $D100            ; Bitmap #0 Control Register
VKY_BM0_ADDR_L = $D101          ; Bitmap #0 Address bits 7..0
VKY_BM0_ADDR_M = $D102          ; Bitmap #0 Address bits 15..8
VKY_BM0_ADDR_H = $D103          ; Bitmap #0 Address bits 17..16

bitmap_base = $10000            ; The base address of our bitmap

stz MMU_IO_CTRL     ; Go back to I/O page #0

lda #$0C            ; enable GRAPHICS and BITMAP. Disable TEXT
```

```
            sta VKY_MSTR_CTRL_0 ; Save that to VICKY master control register 0
            stz VKY_MSTR_CTRL_1 ; Make sure we're just in 320x240 mode
```

Next, it needs to set up the bitmap: setting the address, CLUT, and enabling the bitmap:

```
            ;
            ; Turn on bitmap #0
            ;

            stz VKY_BM1_CTRL    ; Make sure bitmap 1 is turned off

            lda #$01            ; Use graphics LUT #0, and enable bitmap
            sta VKY_BM0_CTRL

            lda #<bitmap_base   ; Set the low byte of the bitmap's address
            sta VKY_BM0_ADDR_L
            lda #>bitmap_base   ; Set the middle byte of the bitmap's address
            sta VKY_BM0_ADDR_M
            lda #`bitmap_base   ; Set the upper two bits of the address
            and #$03
            sta VKY_BM0_ADDR_H
```

Now, the code needs to create the pixel data for the gradient in memory. This is a bit tricky on the F256, because the program is using the larger 320×240 screen, which requires more than 64 KB of memory. In order to write to the entire bitmap, the program will have to work with the MMU to switch memory banks to access the whole bitmap. The program will use bank 1 (0x2000 – 0x3FFF) as its window into the bitmap, which will start at 0x10000. It will walk through the memory byte-by-byte, setting each pixel's color based on what line it is on (tracked in a `line` variable). Once it has written a bank's worth of pixels (8 KB), it will increment the bank number and update the MMU register. Once it has written 240 lines, it will finish.

In the following code, `bm_bank` and `line` are byte variables, and `pointer` and `column` are two-byte variables in zero page (although really only `pointer` has to be there).

```
            ; Set the line number to 0
            stz line

            ; Calculate the bank number for the bitmap
            lda #(bitmap_base >> 13)
            sta bm_bank

bank_loop:  stz pointer         ; Set the pointer to start of the current bank
            lda #$20
            sta pointer+1

            ; Set the column to 0
            stz column
            stz column+1
```

```
            ; Alter the LUT entries for $2000 -> $bfff

            lda #$80            ; Turn on editing of MMU LUT #0, and use #0
            sta MMU_MEM_CTRL

            lda bm_bank
            sta MMU_MEM_BANK_1  ; Set the bank we will map to $2000 - $3fff

            stz MMU_MEM_CTRL    ; Turn off editing of MMU LUT #0

            ; Fill the line with the color..

loop2:      lda line            ; The line number is the color of the line
            sta (pointer)

inc_column: inc column          ; Increment the column number
            bne chk_col
            inc column+1

chk_col:    lda column          ; Check to see if we have finished the row
            cmp #<320
            bne inc_point
            lda column+1
            cmp #>320
            bne inc_point

            lda line            ; If so, increment the line number
            inc a
            sta line
            cmp #240            ; If line = 240, we're done
            beq done

            stz column          ; Set the column to 0
            stz column+1

inc_point:  inc pointer         ; Increment pointer
            bne loop2           ; If < $4000, keep looping
            inc pointer+1
            lda pointer+1
            cmp #$40
            bne loop2

            inc bm_bank         ; Move to the next bank
            bra bank_loop       ; And start filling it

done:       nop                 ; Lock up here
            bra done
```

5 Sprites

In addition to bitmaps and tiles, the F256 provides support for sprites, which are mobile graphical objects that can appear anywhere on the screen. F256 sprites are similar to the sprites on the Commodore 64 or player-missile graphics on the 8-bit Atari computers, but they are more flexible than either of those. A sprite is essentially a little bitmap that can be positioned anywhere on the screen. Each one can come in one of four sizes: 8×8, 16×16, 24×24, or 32×32. Each one can display up to 256 colors, picked from one of the four graphics color lookup tables.

A program for the F256 can use up to 64 sprites, each one of which is controlled by a block of sprite control registers. The sprite control registers are in I/O page 0, and start at 0x18_1300 (Block $C0 offset $1300). Each sprite takes up 8 bytes, so sprite 0 starts at 0x18_1300, sprite 1 starts at 0x18_1308, sprite 2 at 0x18_1310, and so on. The registers for each sprite are arranged within that block of 8 bytes as shown in table 5.1.

Offset	R/W	7	6	5	4	3	2	1	0
0	W	—	SIZE		LAYER		LUT		ENABLE
1	W	—					AD18	AD17	AD16
2	W	AD15	AD14	AD13	AD12	AD11	AD10	AD9	AD8
3	W	AD7	AD6	AD5	AD4	AD3	AD2	AD1	AD0
4	W	X15	X14	X13	X12	X11	X10	X9	X8
5	W	X7	X6	X5	X4	X3	X2	X1	X0
6	W	Y15	Y14	Y13	Y12	Y11	Y10	Y9	Y8
7	W	Y7	Y6	Y5	Y4	Y3	Y2	Y1	Y0

Table 5.1: Sprite Registers for a Single Sprite

These registers manage seven fields:

ENABLE if set, this particular sprite will be displayed (assuming the graphics and sprite engines are enabled in the Vicky Master Control Register).

LUT selects the graphics color lookup table to use in assigning colors to pixels

LAYER selects which sprite layer the sprite will be displayed on

SIZE selects the size of the sprite (see table 5.2)

AD the address of the bitmap (must be within the first 512KB of RAM). The address is based on the 21-bit system bus, not the CPU's address space.

X the X coordinate where the sprite will be displayed (corresponds to the sprite's upper-left corner)

Y the Y coordinate where the sprite will be displayed (corresponds to the sprite's upper-left corner)

SIZE		Meaning
0	0	32×32
0	1	24×24
1	0	16×16
1	1	8×8

Table 5.2: Sprite Sizes

Sprite Layers and Display Priority

While a sprite can be assigned to any of four layers, this layer is only used for determining how the sprite interacts with bitmap or tile map graphics and not how sprites layer with each other. When sprites "collide," a built-in sprite priority order is used to determine which sprite determines a pixel's color. When two sprites are both trying to set the color of a pixel on the screen, the sprite with the lowest number is the one that determines the color. For example, if sprite 0 and sprite 5 are in the same location, it is sprite 0 that will display in the foreground. The sprite layers *cannot* be used to change this.

The best practice for assigning sprites is to place the images that need to be on top in the first sprites and those that need to be in the back in the higher numbered sprites. Use the LAYER field for the sprites to control how the sprites layer with the tile maps and bitmaps.

Sprite Positioning

The coordinate system for sprites is similar to that for bitmap graphics, but it is offset by 32 pixels in both the horizontal and vertical directions. There is a sort of margin area around the entire displayed screen that a sprite can be in and be either partially or completely hidden from view. The horizontal coordinate for a sprite ranges from 0 to 352. The vertical coordinate can range from 0 to 232 or 272, depending on the vertical resolution. For a sprite to have its top-left corner in the top-left of the screen, its position would need to be $(32, 32)$. This coordinate system is the same for all sprites, regardless of their size. Figure: 5.1 shows how the coordinate system is arranged.

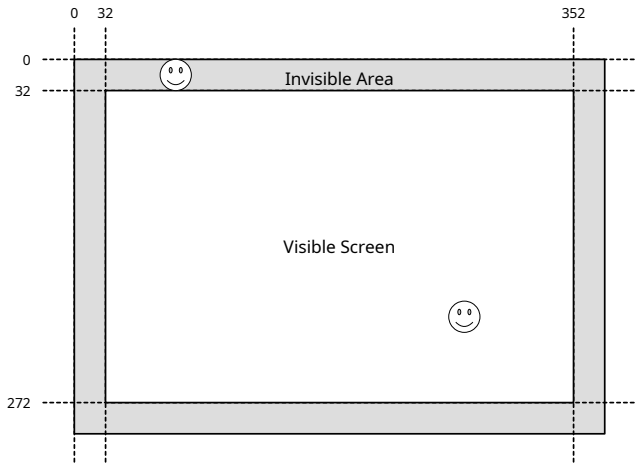


Figure 5.1: Sprite Positions

Example: Displaying a Sprite

In this example, we'll just put a ball on the screen. First, the program needs to set up TinyVicky to be in sprite mode with no border and a light purple background:

```
MMU_IO_CTRL = $0001             ; MMU I/O Control Register

VKY_MSTR_CTRL_0 = $D000         ; Vicky Master Control Register 0
VKY_MSTR_CTRL_1 = $D001         ; Vicky Master Control Register 1
VKY_BRDR_CTRL = $D004           ; Vicky Border Control Register
VKY_BKG_COL_B = $D00D           ; Vicky Graphics Background Color Blue
VKY_BKG_COL_G = $D00E           ; Vicky Graphics Background Color Green
VKY_BKG_COL_R = $D00F           ; Vicky Graphics Background Color Red

VKY_SP0_CTRL = $D900            ; Sprite #0's control register
VKY_SP0_AD_L = $D901            ; Sprite #0's pixel data address register
VKY_SP0_AD_M = $D902
VKY_SP0_AD_H = $D903
VKY_SP0_POS_X_L = $D904         ; Sprite #0's X position register
VKY_SP0_POS_X_H = $D905
VKY_SP0_POS_Y_L = $D906         ; Sprite #0's Y position register
VKY_SP0_POS_Y_H = $D907

VKY_GR_CLUT_0 = $D000           ; Graphics LUT #0 (in I/O page #1)

ptr_src = $0080                 ; A pointer to data to read
ptr_dst = $0082                 ; A pointer to data to write

            ;
            ; Set up TinyVicky to display sprites
            ;
            lda #$24                    ; Graphics & Sprite engines enabled
```

```
            sta VKY_MSTR_CTRL_0
            stz VKY_MSTR_CTRL_1         ; 320x240 @ 60Hz

            stz VKY_BRDR_CTRL           ; No border

            lda #$96                    ; Background: lavender
            sta VKY_BKG_COL_R
            lda #$7B
            sta VKY_BKG_COL_G
            lda #$B6
            sta VKY_BKG_COL_B
```

Next, the program loads the sprite's colors into the CLUT (`ptr_src` and `ptr_dst` are 16-bit storage locations in zero page and are used as pointers):

```
            ;
            ; Load the sprite LUT into memory
            ;

            lda #$01                    ; Switch to I/O Page #1
            sta MMU_IO_CTRL

            lda #<balls_clut_start      ; Set the source pointer to the palette
            sta ptr_src
            lda #>balls_clut_start
            sta ptr_src+1

            lda #<VKY_GR_CLUT_0         ; Set the destination to Graphics CLUT
            sta ptr_dst
            lda #>VKY_GR_CLUT_0
            sta ptr_dst+1

            ldx #0                      ; X is the number of colors copied
color_loop: ldy #0                      ; Y points to the color component
comp_loop:  lda (ptr_src),y             ; Read a byte from the code
            sta (ptr_dst),y             ; And write it to the CLUT
            iny                         ; Move to the next byte
            cpy #4
            bne comp_loop               ; Continue until 4 bytes copied

            inx                         ; Move to the next color
            cpx #16
            beq done_lut                ; Until we have copied all 16

            clc                         ; Move ptr_src to the next source color
            lda ptr_src
            adc #4
            sta ptr_src
```

```
            lda ptr_src+1
            adc #0
            sta ptr_src+1

            clc                         ; Move ptr_dst to the next destination
            lda ptr_dst
            adc #4
            sta ptr_dst
            lda ptr_dst+1
            adc #0
            sta ptr_dst+1

            bra color_loop              ; And start copying that new color

done_lut:   stz MMU_IO_CTRL             ; Go back to I/O Page 0
```

Finally, we point sprite 0 to the pixel data (which is included in the assembly code below), set its location on the screen (which will be the upper left corner of the screen), and then we turn on the sprite setting its LUT and LAYER in the process:

```
            ;
            ; Set up sprite #0
            ;
init_sp0:   lda #<balls_img_start       ; Address = balls_img_start
            sta VKY_SP0_AD_L
            lda #>balls_img_start
            sta VKY_SP0_AD_M
            stz VKY_SP0_AD_H

            lda #32
            sta VKY_SP0_POS_X_L         ; (x, y) = (32, 32)... should be
            stz VKY_SP0_POS_X_H         ; upper-left corner of the screen

            lda #32
            sta VKY_SP0_POS_Y_L
            stz VKY_SP0_POS_Y_H

            lda #$41                    ; Size=16x16, Layer=0, LUT=0, Enabled
            sta VKY_SP0_CTRL
```

Here is the pixel data for the sprite:

```
balls_img_start:
.byte $0, $0, $0, $0, $0, $0, $3, $2, $2, $1, $0, $0, $0, $0, $0, $0
.byte $0, $0, $0, $0, $5, $5, $4, $3, $3, $3, $3, $2, $0, $0, $0, $0
.byte $0, $0, $0, $7, $7, $7, $6, $5, $4, $4, $3, $3, $1, $0, $0, $0
.byte $0, $0, $7, $9, $A, $B, $A, $8, $6, $5, $4, $3, $2, $1, $0, $0
.byte $0, $5, $7, $A, $D, $E, $D, $A, $7, $5, $5, $4, $3, $1, $1, $0
```

```
.byte $0, $5, $7, $B, $E, $E, $E, $C, $7, $5, $5, $4, $3, $1, $1, $0
.byte $3, $4, $6, $A, $D, $E, $D, $A, $7, $5, $5, $4, $3, $2, $1, $1
.byte $2, $3, $5, $8, $A, $C, $A, $8, $6, $5, $5, $4, $3, $2, $1, $1
.byte $2, $3, $4, $6, $7, $7, $7, $6, $5, $5, $5, $4, $3, $1, $1, $1
.byte $1, $3, $4, $5, $5, $5, $5, $5, $5, $5, $5, $3, $3, $1, $1, $1
.byte $0, $3, $3, $4, $5, $5, $5, $5, $5, $5, $4, $3, $2, $1, $1, $0
.byte $0, $2, $3, $3, $4, $4, $4, $4, $4, $3, $3, $2, $1, $1, $1, $0
.byte $0, $0, $1, $2, $3, $3, $3, $3, $3, $3, $2, $1, $1, $1, $0, $0
.byte $0, $0, $0, $1, $1, $1, $2, $2, $1, $1, $1, $1, $1, $0, $0, $0
.byte $0, $0, $0, $0, $1, $1, $1, $1, $1, $1, $1, $1, $0, $0, $0, $0
.byte $0, $0, $0, $0, $0, $0, $1, $1, $1, $1, $0, $0, $0, $0, $0, $0
```

Here are the colors for the sprite (note that this example is using only 15 colors, to make the example more understandable in print):

```
balls_clut_start:
.byte $00, $00, $00, $00
.byte $88, $00, $00, $00
.byte $7C, $18, $00, $00
.byte $9C, $20, $1C, $00
.byte $90, $38, $1C, $00
.byte $B0, $40, $38, $00
.byte $A8, $54, $38, $00
.byte $C0, $5C, $50, $00
.byte $BC, $70, $50, $00
.byte $D0, $74, $68, $00
.byte $CC, $88, $68, $00
.byte $E0, $8C, $7C, $00
.byte $DC, $9C, $7C, $00
.byte $EC, $A4, $90, $00
.byte $EC, $B4, $90, $00
```

6 Tiles

The third graphics engine TinyVicky provides is the tile map system. The tile map system might seem a bit confusing at first, but really it is very similar to text mode, just made more flexible. In text mode, we have characters (256 of them). The shapes of the characters are defined in the font. What character is shown in a particular spot on the screen is set in the text matrix, which is a rectangular array of bytes in memory. In the same way, with the tile system we have tiles (256 of those, too). What those tiles look like are defined in a "tile set." What tile is shown in a particular spot on the screen is set in the "tile map." So there is an analogy:

$$\begin{aligned} \text{character} &\approx \text{tile} \\ \text{font} &\approx \text{tile set} \\ \text{text matrix} &\approx \text{tile map} \end{aligned}$$

There are several differences with tile maps, however:

- A tile map may use tiles that are either 8×8 pixels or 16×16 pixels.
- A tile map can be scrolled smoothly horizontally or vertically.
- A tile may use 256 colors in its pixels as opposed to text mode's two-color characters. This means that a tile set uses one byte per pixel, with that byte's value being an index into a CLUT (as with bitmaps and sprites), where text mode fonts are one *bit* per pixel choosing between a foreground and background color.
- The tile map system allows for up to eight different tile sets to be used at the same time, where text mode has a single font.
- Up to three different tile maps can be displayed at one time, where text mode can only display one text matrix.
- A tile map can be placed on any one of three display layers, where text mode is always on top.

Tile Maps

There are three tile maps supported by TinyVicky, each of which has 12 bytes worth of registers (see table: 6.1). Tile map 0 starts at 0x18_1100 (Block $C0 Offset $1100). Tile map 1 starts at 0x18_110C (Block $C0 Offset $110C). Tile map 2 starts at 0x18_1118 (Block $C0 Offset $111B).

Offset	R/W	7	6	5	4	3	2	1	0
0	W	—			TILE_SIZE	—			ENABLE
1	W	—					AD18	AD17	AD16
2	W	AD15	AD14	AD13	AD12	AD11	AD10	AD9	AD8
3	W	AD7	AD6	AD5	AD4	AD3	AD2	AD1	AD0
4	W	MAP_SIZE_X							
5	W	RESERVED							
6	W	MAP_SIZE_Y							
7	W	RESERVED							
8	W	—		X9	X8	X7	X6	X5	X4
9	W	X3	X2	X1	X0	SSX3	SSX2	SSX1	SSX0
10	W	—				Y7	Y6	Y5	Y4
11	W	Y3	Y2	Y1	Y0	SSY3	SSY2	SSY1	SSY0

Table 6.1: Tile Map Registers

TILE_SIZE if 1, tiles are 8 pixels wide by 8 tall. If 0, tiles are 16 pixels wide by 16 pixels tall

ENABLE if set, the tile map will be displayed (if GRAPH and TILES are set in TinyVicky's Master Control Register)

AD the address of the tile map data in RAM

MAP_SIZE_X the width of the tile map in tiles (*i.e.* the number of columns)

MAP_SIZE_Y the height of the tile map in tiles (*i.e.* the number of rows)

X horizontal scroll in tile widths

SSX horizontal scroll in pixels. How these bits are used varies with the size. If tiles are 16 pixels wide, then flags SSX[3..0] are used. If tiles are only 8 pixels wide, then only SSX[3..1] are used.

Y vertical scroll in tile heights

SSY vertical scroll in pixels. How these bits are used varies with the size. If tiles are 16 pixels wide, then flags SSY[3..0] are used. If tiles are only 8 pixels wide, then only SSY[3..1] are used.

One way tile maps get their flexibility is that, where text mode uses 8-bit bytes for the text matrix, a tile map uses 16-bit integers in memory. A tile map entry is divided up into two pieces: the first byte is the number of the tile to display in that position (much like the character code in text mode), but the upper byte contains attribute bits (see table: 6.2) and has two fields:

SET is the number of the tile set to use for this tile's appearance

CLUT is the number of the graphics CLUT to use in setting the colors

This attribute system makes tiles very powerful. Effectively, a single tile map can display 1,024 completely unique shapes at one time by using all eight tile sets. Also, since the CLUT is set for each tile in the attributes, the number of tiles needed can be reduced by clever use of recoloring.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
—			CLUT		SET			TILE NUMBER							

Table 6.2: A Tile Map Entry

Scrolling

Tile maps can scroll across the screen both horizontally and vertically. The position of the tile map on the screen is controlled through the registers at offsets 8, 9, and 10. The horizontal position is controlled by X, and SSX. The vertical position is controlled by Y, and SSY. The bits X and Y set the position in units of tiles. That is, the number in X[9..0] specifies how many complete tile columns the tile map is moved left. Likewise, Y[9..0] specifies how many tile rows the tile map is moved up. The SSX and SSY bits are used to specify how many rows of pixels within a tile the tile map is to move. SSX and SSY are therefore "smooth scroll" registers. They have a small trick to their use, however:

If the tile map uses tiles 16 pixels on a side, SSX[3..0] is used to specify the number of pixels to shift the tile map left: from 0 to 15. If, on the other hand, the tile map uses tiles 8 pixels on a side, only SSX[3..1] are used to specify the number of pixels to move: from 0 to 7. Note that SSX[0] is not used at all in this case. The SSY bits work in exactly the same way for smooth scrolling in the vertical direction.

To make sure that scrolling will work properly, the tile map needs to be at least as big as the full screen (even if it is largely "empty"), and there should be blank columns to the left and the right and blank rows above and below. That is, it is best to leave an empty margin all the way around your working tile map.

Tile Sets

Essentially, a tile set is just a bitmap, but of a smaller size and arranged in a specific pattern. A tile set can be either a linear arrangement of tiles or a square arrangement of tiles. In the linear arrangement, the image is one tile wide by 256 tiles high. So for 8×8 tiles, the tile set is 8 pixels wide by 2,048 pixels high. For 16×16 tiles, the tile set is 16 pixels wide by 4,096 pixels high. The tiles are arranged vertically, so the first 8 or 16 (depending on tile size) rows are tile 0, the second set of rows are tile 1, and so on. For the square arrangement, the tile set is either 128 pixels wide by 128 pixels high (for 8×8 tiles), or it is 256 pixels wide by 256 pixels high (for 16×16 tiles). In both cases, the tiles are laid out left to right and top to bottom in a grid that is 16 tiles wide by 16 tiles high (see table 6.3).

As with bitmaps and sprites, the pixels of the tiles are each an individual byte. The contents of the byte (0 – 255) serving as an index into a color lookup table. The pixels are also laid out in left-to-right and top-to-bottom order, just as with bitmaps and individual sprites.

	128 or 256 pixels															
128 or 256 pixels	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	46
	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63
	64	65	66	67	68	69	60	71	72	73	74	75	76	77	78	79
	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95
	96	97	98	99	100	101	102	103	104	105	106	107	108	109	110	111
	112	113	114	115	116	117	118	119	120	121	122	123	124	125	126	127
	128	129	130	131	132	133	134	135	136	137	138	139	140	141	142	143
	144	145	146	147	148	149	150	151	152	153	154	155	156	157	158	159
	160	161	162	163	164	165	166	167	168	169	170	171	172	173	174	175
	176	177	178	179	180	181	182	183	184	185	186	187	188	189	190	191
	192	193	194	195	196	197	198	199	200	201	202	203	204	205	206	207
	208	209	210	211	212	213	214	215	216	217	218	219	220	221	222	223
	224	225	226	227	228	229	230	231	232	233	234	235	236	237	238	239
	240	241	242	243	244	245	246	247	248	249	250	251	252	253	254	255

Table 6.3: Arrangement of Tiles in a Tile Set Image

TinyVicky supports eight separate tile sets. Each one has a single three byte address register, which provides the address to the tile set pixel data, and a configuration register (see table: 6.4). To use them, a program simply stores the address of the pixel data to use into the appropriate address register. The configuration register contains a single SQUARE flag, which indicates the layout of the tile set image. If SQUARE is set (1), the tile set image is square (128×128 pixels for 8×8 tiles or 256×256 pixels for 16×16 tiles). If SQUARE is clear (0), the tile set image is vertical ($8 \times 2,048$ pixels for 8×8 tiles, or $16 \times 4,096$ pixels for 16×16 tiles).

Example: A Simple Tile Map

```
;
; Set up TinyVicky to display tiles
;
lda #$14                    ; Graphics and Tile engines enabled
sta VKY_MSTR_CTRL_0
stz VKY_MSTR_CTRL_1         ; 320x240 @ 60Hz

lda #$40                    ; Layer 0 = Bitmap 0, Layer 1 = Tile map 0
sta VKY_LAYER_CTRL_0
lda #$15                    ; Layer 2 = Tile Map 1
sta VKY_LAYER_CTRL_1

stz VKY_BRDR_CTRL           ; No border
```

Address	R/W	Tile Set	7	6	5	4	3	2	1	0
0x18_1180	W	0	—					AD18	AD17	AD16
0x18_1181	W		AD15	AD14	AD13	AD12	AD11	AD10	AD9	AD8
0x18_1182	W		AD7	AD6	AD5	AD4	AD3	AD2	AD1	AD0
0x18_1183	W		—				SQUARE	—		
0x18_1184	W	1	—					AD18	AD17	AD16
0x18_1185	W		AD15	AD14	AD13	AD12	AD11	AD10	AD9	AD8
0x18_1186	W		AD7	AD6	AD5	AD4	AD3	AD2	AD1	AD0
0x18_1187	W		—				SQUARE	—		
0x18_1188	W	2	—					AD18	AD17	AD16
0x18_1189	W		AD15	AD14	AD13	AD12	AD11	AD10	AD9	AD8
0x18_118A	W		AD7	AD6	AD5	AD4	AD3	AD2	AD1	AD0
0x18_118B	W		—				SQUARE	—		
0x18_118C	W	3	—					AD18	AD17	AD16
0x18_118D	W		AD15	AD14	AD13	AD12	AD11	AD10	AD9	AD8
0x18_118E	W		AD7	AD6	AD5	AD4	AD3	AD2	AD1	AD0
0x18_118F	W		—				SQUARE	—		

Table 6.4: Tile Set 0–3 Registers

```
lda #$19                    ; Background: midnight blue
sta VKY_BKG_COL_R
lda #$19
sta VKY_BKG_COL_G
lda #$70
sta VKY_BKG_COL_B
```

To define the tile set, all we really need to do is to set the address register for the tile set to point to the actual pixel data. In this particular case, the code is just going to use tile set 0.

```
;
; Set tile set #0 to our image
;

lda #<tiles_img_start
sta VKY_TS0_ADDR_L
lda #>tiles_img_start
sta VKY_TS0_ADDR_M
lda #‘tiles_img_start
sta VKY_TS0_ADDR_H
```

Finally, the code sets up the tile map itself, setting the size of the tiles, the size of the tile map, setting the position of the screen in the tile map, and pointing to the tile map data.

```
;
```

Address	R/W	Tile Set	7	6	5	4	3	2	1	0
0x18_1190	W	4	—					AD18	AD17	AD16
0x18_1191	W		AD15	AD14	AD13	AD12	AD11	AD10	AD9	AD8
0x18_1192	W		AD7	AD6	AD5	AD4	AD3	AD2	AD1	AD0
0x18_1193	W		—				SQUARE	—		
0x18_1194	W	5	—					AD18	AD17	AD16
0x18_1195	W		AD15	AD14	AD13	AD12	AD11	AD10	AD9	AD8
0x18_1196	W		AD7	AD6	AD5	AD4	AD3	AD2	AD1	AD0
0x18_1197	W		—				SQUARE	—		
0x18_1198	W	6	—					AD18	AD17	AD16
0x18_1199	W		AD15	AD14	AD13	AD12	AD11	AD10	AD9	AD8
0x18_119A	W		AD7	AD6	AD5	AD4	AD3	AD2	AD1	AD0
0x18_119B	W		—				SQUARE	—		
0x18_119C	W	7	—					AD18	AD17	AD16
0x18_119D	W		AD15	AD14	AD13	AD12	AD11	AD10	AD9	AD8
0x18_119E	W		AD7	AD6	AD5	AD4	AD3	AD2	AD1	AD0
0x18_119F	W		—				SQUARE	—		

Table 6.5: Tile Set Registers 4–7

```
; Set tile map #0
;

lda #$01                    ; 16x16 tiles, enable
sta VKY_TM0_CTRL

lda #22                     ; Our tile map is 20x15
sta VKY_TM0_SIZE_X
lda #16
sta VKY_TM0_SIZE_Y

lda #<tile_map              ; Point to the tile map
sta VKY_TM0_ADDR_L
lda #>tile_map
sta VKY_TM0_ADDR_M
lda #‘tile_map
sta VKY_TM0_ADDR_H

lda #$0F                    ; Set scrolling (15, 0)
sta VKY_TM0_POS_X_L
lda #$00
sta VKY_TM0_POS_X_H

stz VKY_TM0_POS_Y_L
stz VKY_TM0_POS_Y_H
```

The tile map itself. In this case, we just define it in-line. The data is formatted to match the dimensions of the tile map for ease of reading. Note that the left-most and right-most columns are essentially blank, providing some buffer space to allow for scrolling. Similarly, there is a spare row on the bottom. This data is formatted as single hexadecimal digits, to make it easier to format this data on the page, but the data is actually stored as 16-bit values. This is taking advantage of the fact that the code is using CLUT 0 and LAYER 0 for the tiles and that there are no more than 16 tiles in the tile set.

```
tile_map:
.word $4,$1,$0,$1,$0,$0,$0,$0,$0,$0,$0,$0,$0,$0,$0,$0,$0,$0,$4,$0,$4,$0
.word $0,$0,$1,$0,$0,$0,$0,$0,$0,$0,$0,$0,$0,$0,$0,$0,$0,$0,$0,$4,$0,$0
.word $0,$1,$0,$1,$0,$0,$6,$7,$7,$7,$7,$7,$7,$7,$7,$8,$0,$0,$4,$0,$4,$0
.word $0,$0,$0,$0,$0,$0,$9,$1,$2,$3,$4,$5,$0,$0,$0,$A,$0,$0,$0,$0,$0,$0
.word $0,$0,$0,$0,$0,$0,$9,$2,$1,$2,$3,$4,$5,$0,$0,$A,$0,$0,$0,$0,$0,$0
.word $0,$0,$0,$0,$0,$0,$9,$3,$2,$1,$2,$3,$4,$5,$0,$A,$0,$0,$0,$0,$0,$0
.word $0,$0,$0,$0,$0,$0,$9,$4,$3,$2,$1,$2,$3,$4,$5,$A,$0,$0,$0,$0,$0,$0
.word $0,$0,$0,$0,$0,$0,$9,$5,$4,$3,$2,$1,$2,$3,$4,$A,$0,$0,$0,$0,$0,$0
.word $0,$0,$0,$0,$0,$0,$9,$0,$5,$4,$3,$2,$1,$2,$3,$A,$0,$0,$0,$0,$0,$0
.word $0,$0,$0,$0,$0,$0,$9,$0,$0,$5,$4,$3,$2,$1,$2,$A,$0,$0,$0,$0,$0,$0
.word $0,$0,$0,$0,$0,$0,$9,$0,$0,$0,$5,$4,$3,$2,$1,$A,$0,$0,$0,$0,$0,$0
.word $0,$0,$0,$0,$0,$0,$B,$C,$C,$C,$C,$C,$C,$C,$C,$D,$0,$0,$0,$0,$0,$0
.word $0,$3,$0,$3,$0,$0,$0,$0,$0,$0,$0,$0,$0,$0,$0,$0,$0,$0,$2,$0,$2,$0
.word $0,$0,$3,$0,$0,$0,$0,$0,$0,$0,$0,$0,$0,$0,$0,$0,$0,$0,$0,$2,$0,$0
.word $0,$3,$0,$3,$0,$0,$0,$0,$0,$0,$0,$0,$0,$0,$0,$0,$0,$0,$2,$0,$2,$0
.word $0,$0,$0,$0,$0,$0,$0,$0,$0,$0,$0,$0,$0,$0,$0,$0,$0,$0,$2,$0,$0,$4
```

7 Miscellaneous Features of TinyVicky

DIP Switches

F256 has eight DIP switches on the board, which can be used to configure various options. The DIP switches are present on a single register (see table: 7.1). The DIP switches ground their signals when placed in their "on" positions. So a true or asserted value is 0, while the false or de-asserted value is 1.

Address	R/W	7	6	5	4	3	2	1	0
0xFF90	R	GAMMA	USER2	USER1	USER0	BOOT			

Table 7.1: DIP Switch Register

There are five fields of switches:

GAMMA this is a dedicated switch to indicate if gamma correction should be turned on (0) or not (1)

USER0, USER1, USER2 these three switches are reserved for use by the operating system or programs. On is 0, off is 1.

BOOT these four switches provide information to the operating system for boot options.

The Border

The F256's display can have a border, which overlays all the other display elements. The border can have any color which TinyVicky can display, and can have a width from 0 to 31 pixels. The border can also be turned off, leaving the full display for graphics or text.

When using graphics modes, the border simply hides the graphics elements underneath it. For text mode, things are a little different. The text display will be shifted so that the character at $(0, 0)$ is still the upper-left character. The layout of the text and color matrixes do not change, however. Cells that are under the right side or bottom of the border will still be in the matrixes but will not be displayed. Another way to put it is that, if the text resolution is 80 characters wide, it will remain 80 characters per line even if the border is on and only 76 characters are displayed.

Address	R/W	Name	7	6	5	4	3	2	1	0
0xFFC4	R/W	BRDR_CTRL	—	SCROLL_X			—			ENABLE
0xFFC5	R/W	BRDR_BLUE	Blue component of border color							
0xFFC6	R/W	BRDR_GREEN	Green component of border color							
0xFFC7	R/W	BRDR_RED	Red component of border color							
0xFFC8	R/W	BRDR_WIDTH	—			SIZE_X				
0xFFC9	R/W	BRDR_HEIGHT	—			SIZE_Y				

Table 7.2: Border Registers

ENABLE when set (1), the border will be displayed

SCROLL_X the number of pixels the border should be shifted in the horizontal direction

BRDR_BLUE the amount of blue in the border (0 = none, 255 = maximum amount)

BRDR_GREEN the amount of green in the border (0 = none, 255 = maximum amount)

BRDR_RED the amount of red in the border (0 = none, 255 = maximum amount)

SIZE_X the width of the left and right sides of the border in pixels (from 0 to 31)

SIZE_Y the height of top and bottom of the border in pixels (from 0 to 31)

Background Color

In text mode, the background color is determined by the color matrix and the text color LUTs. For the graphics modes, however, a background color is specified separately. There are three registers to specify the background color's red, green, and blue components (see table: 7.3). This is the color that will be displayed in graphics modes, if all the layers specify that a given pixel has the color 0 (which is always the transparent pixel color).

Address	R/W	Name	7	6	5	4	3	2	1	0
0xFFCD	R/W	BGND_BLUE	Blue component of background color							
0xFFCE	R/W	BGND_GREEN	Green component of background color							
0xFFCF	R/W	BGND_RED	Red component of background color							

Table 7.3: Background Color Registers

Line Interrupt and Beam Position

TinyVicky can trigger a SOL interrupt (see table: 9.1) when the display has reached a given raster line. This can be useful for split-screen style effects or other programming tricks that work off of partitioning the screen into separate areas. To use this feature, a program would enable the line interrupt and set a register to the number of the line on the screen when the interrupt should

be triggered. In addition to setting a line interrupt, there are two 12-bit registers that allow the program to see what line and column is TinyVicky is currently drawing. The addresses for all these registers overlap. The line interrupt registers are write-only, and the current beam position registers are read only (see table: 7.4)

Address	R/W	Name	7	6	5	4	3	2	1	0
0xFFD8	W	LINT_CTRL	—							ENABLE
0xFFDA	W	LINT_L	—				L11	L10	L9	L8
0xFFD9	W		L7	L6	L5	L4	L3	L2	L1	L0
0xFFDB	W	—	Reserved							
0xFFD9	R	RAST_COL	—				X11	X10	X9	X8
0xFFD8	R		X7	X6	X5	X4	X3	X2	X1	X0
0xFFDB	R	RAST_ROW	—				Y11	Y10	Y9	Y8
0xFFDA	R		Y7	Y6	Y5	Y4	Y3	Y2	Y1	Y0

Table 7.4: Line Interrupt and Beam Position Registers

ENABLE if set (1), TinyVicky will trigger line interrupts (write only)

LINT_L the line number (12 bits) on which to trigger the next line interrupt (write only). The top of the display is line 0, and the bottom of the screen is 400 for 320×200 mode, and 480 for 320×240 mode.

RAST_COL the number (12 bits) of the current pixel being drawn (read only)

RAST_ROW the number (12 bits) of the current line being drawn (read only)

Example: Changing Border with the Line

In this example, we will play with a split-screen style effect changing the color of the border so that the top and bottom borders are blue while the left and right borders are red. To do this, we will use the line interrupt twice for each frame: once when we are on the line just below the last line of the top border, and once when we are on the first line of the bottom border.

To make this work, the example has a single `state` variable, which will track which color border is being rendered. It will enable the line interrupt, and then set the line number to wait for. When that interrupt comes in, it will check the state variable, setting the border color and new line number based on `state`. It will also flip `state` to the other value (0 or 1).

```
INT_PEND_0 = $D660              ; Pending register for interrupts 0 - 7
INT_PEND_1 = $D661              ; Pending register for interrupts 8 - 15
INT_MASK_0 = $D66C     ; Mask register for interrupts 0 - 7
INT_MASK_1 = $D66D     ; Mask register for interrupts 8 - 15
INT01_VKY_SOL = $02

MMU_IO_CTRL = $0001             ; MMU I/O Control Register

VKY_MSTR_CTRL_0 = $D000         ; Vicky Master Control Register 0
```

```
VKY_MSTR_CTRL_1 = $D001         ; Vicky Master Control Register 1
VKY_BRDR_CTRL = $D004           ; Vicky Border Control Register
VKY_BRDR_COL_B = $D005          ; Vicky Border Color -- Blue
VKY_BRDR_COL_G = $D006          ; Vicky Border Color -- Green
VKY_BRDR_COL_R = $D007          ; Vicky Border Color -- Red
VKY_BRDR_HORI = $D008           ; Vicky Border Horizontal Thickness in pixels
VKY_BRDR_VERT = $D009           ; Vicky Border vertical thickness in pixels

VIRQ = $FFFE

LINE0 = 16                      ; Start at line 16 (first line on the text display)
LINE1 = 480 - 16                ; End on line 464 (last line of text display)

;
; Variables
;
* = $0080

state       .byte ?             ; Variable to track which color we should use

;
; Code
;
* = $e000

start:      ; Disable IRQ handling
            sei

            ; Go back to I/O page 0
            stz MMU_IO_CTRL

            ; Load my IRQ handler into the IRQ vector
            ; NOTE: this code just takes over IRQs completely. It could save
            ;       the pointer to the old handler and chain to it when it has
            ;       handled its interrupt. But what is proper really depends on
            ;       what the program is trying to do.
            lda #<my_handler
            sta VIRQ
            lda #>my_handler
            sta VIRQ+1

            ; Mask off all but the SOL interrupt
            lda #$ff
            sta INT_MASK_1
            and #~INT01_VKY_SOL
            sta INT_MASK_0
```

```
            ; Clear all pending interrupts
            lda #$ff
            sta INT_PEND_0
            sta INT_PEND_1

            ; Make sure we're in text mode
            lda #$01                ; enable TEXT
            sta VKY_MSTR_CTRL_0     ; Save to VICKY master control register 0
            stz VKY_MSTR_CTRL_1

            ; Set the border
            lda #$01                ; Enable the border
            sta VKY_BRDR_CTRL

            lda #16                 ; Make it 16 pixels wide
            sta VKY_BRDR_VERT
            sta VKY_BRDR_HORI

            lda #$80                ; Make it cyan to start with
            sta VKY_BRDR_COL_B
            sta VKY_BRDR_COL_G
            stz VKY_BRDR_COL_R

            lda #$01                ; Turn on the line interrupt
            sta VKY_LINE_CTRL

            lda #<LINE0             ; set the line to interrupt on
            sta VKY_LINE_NBR_L
            lda #>LINE0
            sta VKY_LINE_NBR_H

            stz state               ; Start in state 0

            ; Re-enable IRQ handling
            cli

loop:       ; Just loop forever... a real program will do stuff here
            nop
            bra loop

;
; A simple interrupt handler
;
my_handler: .proc
            pha

            ; Save the system control register
```

```
            lda MMU_IO_CTRL
            pha

            ; Switch to I/O page 0
            stz MMU_IO_CTRL

            ; Check for SOL flag
            lda #INT01_VKY_SOL
            bit INT_PEND_0
            beq return              ; If it's zero, exit the handler

            ; Yes: clear the flag for SOL
            sta INT_PEND_0

            lda state               ; Check the state
            beq is_zero

            stz state               ; If state 1: Set the state to 0

            lda #<LINE0             ; Set the line to interrupt on
            sta VKY_LINE_NBR_L
            lda #>LINE0
            sta VKY_LINE_NBR_H

            lda #$80                ; Make the border blue
            sta VKY_BRDR_COL_B
            stz VKY_BRDR_COL_G
            stz VKY_BRDR_COL_R
            bra return

is_zero:    lda #$01                ; Set the state to 1
            sta state

            lda #<LINE1             ; set the line to interrupt on
            sta VKY_LINE_NBR_L
            lda #>LINE1
            sta VKY_LINE_NBR_H

            lda #$80                ; Make the border red
            sta VKY_BRDR_COL_R
            stz VKY_BRDR_COL_G
            stz VKY_BRDR_COL_B

            ; Restore the system control register
return:     pla
            sta MMU_IO_CTRL
```

```
; Return to the original code
pla
rti
.pend
```

Gamma Correction

TinyVicky has the ability to apply gamma correction to the video signal. This allows users to adjust their images to match their monitors. Activating gamma correction is done by setting the GAMMA flag in the Vicky master control register (see table: 3.3). When enabled, colors will be adjusted through the gamma look up tables. There are three tables: blue is at 0xC000, green is at 0xC400, and red is at 0xC800.

The way that the gamma look up tables work is very straight forward. When drawing a pixel, the separate color components are used as indexes into their respective gamma LUTs, and the value in the LUT is used as the new component value. For instance, if a pixel's color is (r, g, b), then the new color is:

```
r_corrected = gamma_red[r]
g_corrected = gamma_green[g]
b_corrected = gamma_blue[b]
```

On power up, TinyVicky sets up a default gamma correction of 1.8, although software (either the user's program or the operating system) has to turn on gamma correction to use it.

8 Sound

The F256 line has a couple of sound chips, which chips are present depends upon the model. All of the machines have built-in the SN76489 (called the "PSG" here), which was used by many vintage machines including the TI99/4A, the BBC Micro, the IBM PCjr, and the Tandy 1000. The PSG chips on the F256 are actually implemented as part of the TinyVicky FPGA. The F256jr also has two sockets on the board that may be populated with SID chips (either the original 6581, the later 8580, or any of the new FPGA replacements). The F256k implements these SID chips in the FPGA but also includes a physical OPL3 chip on the board.

CODEC

The F256 (and indeed all the C256 and F256 family computers up to this point) makes use of a WM8776 CODEC chip. You can think of the CODEC as the central switchboard for audio on the F256. The CODEC chip has inputs for several audio channels (both analog and digital), and each audio device on the F256 is routed to an input on the CODEC. The CODEC then has outputs for audio line level and headphones. The CODEC will convert analog inputs to digital, mix all the audio inputs according to its settings, and then convert the resulting digital audio to analog and drive the outputs. With the CODEC, you can turn on and off the various input channels, control the volume, and mute or enable the different outputs.

The CODEC is a rather complex chip with many features, and the full details are really beyond the scope of this document. Most programs for the F256 will not need to use it or will only use it in very specific ways. Therefore, this document will really just show how to access it and initialize it and then leave a reference to the data sheet for the chip that has the complete data on the chip.

Raw access to the CODEC chip is fairly complex. Fortunately, the FPGA on the F256 provides three registers to simply access for programs. The FPGA takes care of the actual timing of transmitting data to the CODEC, serializing the data correctly, and so on. All the program needs to know about are the correct format for the 16-bit command words that are sent to the CODEC, and then a status register to monitor.

The CODEC commands are based around a number of registers. Each command is really just writing values to those registers. The command words are 16-bits wide, with the 7 most significant bits being the number of the register to write, and the 9 least significant bits being the data to write. For instance, there is a register to enable and disable the headphone output. Bit 0 of the register controls whether or not the headphone output is enabled (0 = enabled, 1 = disabled). The register number is 13. So, to disable the output on the headphones, we would need

to write 000000001 to register 13. The register number in binary is 0001101, So the command word we would need to send is 0001101000000001 or 0x1A01.

The registers for the CODEC on the F256 are shown in table 8.1.

Address	R/W	7	6	5	4	3	2	1	0	Purpose
0xD620	W	D7	D6	D5	D4	D3	D2	D1	D0	Command Low
0xD621	W	R6	R5	R4	R3	R2	R1	R0	D8	Command High
0xD622	R	X							BUSY	Status
0xD622	W	X							START	Control

Table 8.1: CODEC Control Registers

Bit 0 of the status/control register both triggers sending the command (on a write) and indicates if the CODEC is busy receiving a command (writing a 1 triggers the sending of the command, reading a 1 indicates that the CODEC is busy).

So to mute the headphones, we would issue the following:

```
wait:  lda $D622     ; Wait for the CODEC to be ready
       and #$01
       cmp #$01
       beq wait      ; Bit 0 = 1, CODEC is still busy... keep waiting

       lda #$01      ; Set command to %0001101000000001, or R13 <- 000000001
       sta $D620
       lda #$1A
       sta $D621

       lda #$01      ; Trigger the transmission of the command to the CODEC
       sta $D622
```

Using the PSGs

The F256 has support for dual SN76489 (PSG) sound chips, emulated in the FPGA. The SN76489 was used in several vintage machines, including the TI-99/4A, BBC Micro, IBM PCjr, and Tandy 1000. The chip provides three independent square-wave tone generators and a single noise generator. Each tone generator can produce tones of several frequencies in 16 different volume levels. The noise generator can produce two different types of noise in three different tones at 16 different volume levels.

Access to each PSG is through a single memory address, but that single address allows the CPU to write a value to eight different internal registers. For each tone generator, there is a ten bit frequency (which takes two bytes to set), and a four bit “attenuation” or volume level. For the noise generator, there is a noise control register and a noise attenuation register.

There are four basic formats of bytes that can be written to the port, as shown in table 8.3.

Note: there is a PSG sound device for the left stereo channel and one for the right. The left channel PSG can be accessed at 0xD600, and the right channel at 0xD610. Both are in I/O page 0. There is also a sound “device” for managing the left and right PSGs together, which starts at

R2	R1	R0	Channel	Purpose
0	0	0	Tone 1	Frequency
0	0	1	Tone 1	Attenuation
0	1	0	Tone 2	Frequency
0	1	1	Tone 2	Attenuation
1	0	0	Tone 3	Frequency
1	0	1	Tone 3	Attenuation
1	1	0	Noise	Control
1	1	1	Noise	Attenuation

Table 8.2: SN76489 Channel Registers

D7	D6	D5	D4	D3	D2	D1	D0	Purpose
1	R2	R1	R0	F3	F2	F1	F0	Set the low four bits of the frequency
0	X	F9	F8	F7	F6	F5	F4	Set the high six bits of the frequency
1	1	1	0	X	FB	F1	F0	Set the type and frequency of the noise generator
1	R2	R1	R0	A3	A2	A1	A0	Set the attenuation (four bits)

Table 8.3: SN76489 Command Formats

0xD608. The combined registers work in the same way as the left and right PSGs. Writing to the combined registers is equivalent to writing to the left and right channel registers simultaneously.

The PSGs can be used with their outputs mixed in one of two modes. They can either be used as independent 4 voice stereo sound (one PSG on the left and one on the right), or they can be used as 8 voice monaural sound (both PSGs are routed to both left and right sound channels). This is controlled by the PSG_ST flag in the system control registers (see page 95).

Attenuation

All the channels support attenuation or volume control. The PSG expresses the loudness of the sound with how much it is attenuated or dampened. Therefore, an attenuation of 0 will be the loudest sound, while an attenuation of 15 will make the channel silent.

Tones

Each of the three sound channels generates simple square waves. The frequency generated depends upon the system clock driving the chip and the number provided in the frequency register. The relationship is:

$$f = \frac{C}{32n}$$

where f is the frequency produced, C is the system clock, and n is the number provided in the register. Expressed a different way, the value we need to produce a given frequency can be computed as:

$$n = \frac{C}{32f}$$

For the F256 the system clock is 3.57 MHz, which means:

$$n = \frac{111,563}{f}$$

So, let us say we want channel 1 to produce a concert A, which is 440Hz at maximum volume. The value we need to set for the frequency code is $111,320/440 = 253$ or 0xFE. We can do that with this code:

```
lda #$90        ; %10010000 = Channel 1 attenuation = 0
sta $D600       ; Send it to left PSG
sta $D610       ; Send it to right PSG

lda #$8E        ; %10001100 = Set the low 4 bits of the frequency code
sta $D600       ; Send it to left PSG
sta $D610       ; Send it to right PSG

lda #$0F        ; %00001111 = Set the high 6 bits of the frequency
sta $D600       ; Send it to left PSG
sta $D610       ; Send it to right PSG
```

To turn it off later, we just need to write:

```
lda #$9F        ; %10011111 = Channel 1 attenuation = 15 (silence)
sta $D600       ; Send it to left PSG
sta $D610       ; Send it to right PSG
```

Noise

Noise works differently from tones, since it is random. The noise generator on the PSG can produce two styles of noise determined by the FB bit: white noise (FB = 1), and periodic (FB = 0). The noise has a sort of frequency, based on either the system clock or the current output of tone 3. This frequency is set using the F1 and F0 bits:

F1	F0	Frequency
0	0	$C/512$
0	1	$C/1024$
1	0	$C/2048$
1	1	Tone 3 output

Table 8.4: SN76489 Noise Frequencies

As an example, to set white noise of the highest frequency ($C/512$ or around 6 kHz), we could use the code:

```
lda #$F0        ; %10010000 = Channel 3 attenuation = 0
sta $D600       ; Send it to left PSG
sta $D610       ; Send it to right PSG
```

```
lda #$E4        ; %11100100 = white noise, f = C/512
sta $D600       ; Send it to left PSG
sta $D610       ; Send it to right PSG
```

To turn it off later, we just need to write:

```
lda #$FF        ; %1ff11111 = Channel 3 attenuation = 15 (silence)
sta $D600       ; Send it to left PSG
sta $D610       ; Send it to right PSG
```

Using the SIDs

The SID is a full-featured analog sound synthesizer, and a full explanation of how to use it is really beyond the scope of this document. In this document, I will provide just an introduction to the chip and list the register addresses for the SID chips (see table 8.5), which are expressed as offsets from a base address. Since there are two SID chips, there are two base addresses: the left channel starts at `0xD400`, and the right channel starts at `0xD500`. For the F256jr, the SID chips are optional. The board comes with two unpopulated sockets into which either genuine SID chips or the various FPGA replacements may be installed. For the F256k, there is no socket for SIDs, but the two SID chips are provided by the built-in FPGAs.

The SID chip provides three independent voices (so it can play three notes at once). The three voices are almost identical in their features, with voice 3 being the only one different. Each voice can produce one of four basic sound wave forms: randomized noise, square waves, saw tooth waves, and triangle waves. These waves can be generated over a range of frequencies, and for the square waves, the width of the pulse (*i.e.* duty cycle) may be adjusted.

The type of wave form produced by a voice is controlled by the NOISE, PULSE, SAW, and TRI bits. If NOISE is set to 1, the output is random noise. If PULSE is set, a square wave is produced. If SAW is set, a saw tooth wave is produced. If TRI is set, the voice produces a triangle wave. If PULSE is set, the duty cycle of the square wave (or pulse width, if you prefer) is set by the PW bits according to the formula PW/40.95 (expressed as a percent).

The frequency of the waveform is set by the bits `F[15..0]`. This number sets the actual frequency according the the formula:

$$f_{\text{out}} = \frac{FC}{16777216}$$

where: f_{out} is the output frequency, F is the number set in the registers, and C is the system clock driving the SIDs. For the F256, C is 1.022714 MHz, so the formula for the F256 is:

$$f_{\text{out}} = \frac{F}{16.405}$$

or:

$$F = 16.404 f_{\text{out}}$$

For example: concert A, which is 440 Hz, would be: $F = 16.405 \times 440 \approx 7218$. So, to play a concert A, you would set the frequency to 7218, or 0x1C32.

Each of the three voices has a sound “envelope” which changes the volume of the sound during the duration of the note. There are four phases to the sound envelope: attack, decay,

Voice	Offset	R/W	7	6	5	4	3	2	1	0
V1	0	W	F7	F6	F5	F4	F3	F2	F1	F0
	1	W	F15	F14	F13	F12	F11	F10	F9	F8
	2	W	PW7	PW6	PW5	PW4	PW3	PW2	PW1	PW0
	3	W	X				PW11	PW10	PW9	PW8
	4	W	NOISE	PULSE	SAW	TRI	TEST	RING	SYNC	GATE
	5	W	ATK3	ATK2	ATK1	ATK0	DLY3	DLY2	DLY1	DLY0
	6	W	STN3	STN2	STN1	STN0	RLS3	RLS2	RLS1	RLS0
V2	7	W	F7	F6	F5	F4	F3	F2	F1	F0
	8	W	F15	F14	F13	F12	F11	F10	F9	F8
	9	W	PW7	PW6	PW5	PW4	PW3	PW2	PW1	PW0
	10	W	X				PW11	PW10	PW9	PW8
	11	W	NOISE	PULSE	SAW	TRI	TEST	RING	SYNC	GATE
	12	W	ATK3	ATK2	ATK1	ATK0	DLY3	DLY2	DLY1	DLY0
	13	W	STN3	STN2	STN1	STN0	RLS3	RLS2	RLS1	RLS0

Table 8.5: SID V1 and V2 Registers

sustain, and release (ADSR). When the note first starts playing (that is, the GATE bit for the voice is set to 1), it starts at the attack phase when the volume starts at zero and goes up to the current maximum volume (which is controlled by VOL3-0). How fast this happens is determined by the attack rate (ATK3-0 in the registers). Once the volume reaches the maximum, the volume goes down again to the sustain volume. This phase is called decay, and the speed at which the volume drops is determined by the DCY3-0 register values. Next, the envelope enters the sustain phase, where the volume is held steady at the sustain level (STN3-0). It stays here until the note is to stop playing (GATE is set to 0). At this point, the envelope enters the release stage, where the volume drops back to zero at the release rate (RLS3-0).

The ADSR envelope allows the SID chip to mimic the qualities of various musical instruments or shape various sound effects. For instance, a pipe organ's notes are typically either on or off, so the attack, decay, and release rates would be set to be instantaneous, and the sustain level would be set to full. A piano, on the other hand tends to have a sharp, somewhat percussive sound at the beginning with the note holding a long time on release if not dampened.

While the different voices are independent, they can be set to alter one another through two different effects: synchronization, and ring modulation. With these features, the voices can interact with each other in the following pairs:

- Voice 1 → Voice 2
- Voice 2 → Voice 3
- Voice 3 → Voice 1

Ring Modulation

If a voice's RING bit is set and the voice is set to use the triangle wave form (TRI is set), then the triangle wave will be replaced by the combination of the two voice's frequencies. So if the RING

Voice	Offset	R/W	7	6	5	4	3	2	1	0
V3	14	W	F7	F6	F5	F4	F3	F2	F1	F0
	15	W	F15	F14	F13	F12	F11	F10	F9	F8
	16	W	PW7	PW6	PW5	PW4	PW3	PW2	PW1	PW0
	17	W	X				PW11	PW10	PW9	PW8
	18	W	NOISE	PULSE	SAW	TRI	TEST	RING	SYNC	GATE
	19	W	ATK3	ATK2	ATK1	ATK0	DLY3	DLY2	DLY1	DLY0
	20	W	STN3	STN2	STN1	STN0	RLS3	RLS2	RLS1	RLS0
	21	W	X					FC2	FC1	FC0
	22	W	FC10	FC9	FC8	FC7	FC6	FC5	FC4	FC3
	23	W	RES3	RES2	RES1	RES0	EXT	FILTV3	FILTV2	FILTV1
	24	W	MUTEV3	HIGH	BAND	LOW	VOL3	VOL2	VOL1	VOL0

Table 8.6: SID V3 and Miscellaneous Registers

bit of voice 1 is set, the result will be the ring modulation of voice 1 and voice 3. Ring modulation tends to produce harmonics and overtones and can be used for bell like sounds.

Synchronization

If a voice's SYNC bit is set, the frequency it produces will be synchronized to the controlling voice. So if voice 1's SYNC bit is set, its frequency will be synchronized to voice 3.

NOTE: Voice 3 can be muted by setting MUTEV3. This is useful to have the wave forms generated by voice 3 be used for ring modulation and synchronization without having voice 3's wave forms being actually audible.

Filtering

The SID chip can apply a filter to the audio before sending it out for amplification. The filter works at an adjustable frequency and may be used as either a high-pass filter (if HIGH is set), a low-pass filter (if LOW is set), or as a band-pass filter (if BAND is set). The filter frequency is set by the bits FC0-10. The filter may be applied or not to each voice independently. Bits FILTV1, FILTV2, and FILTV3 control whether the filter is applied to voices 1, 2, and 3 respectively. Finally, a resonance effect may be tuned on the filter using the RES0-3 bits: 0 indicates no resonance, and 15 indicates maximum resonance.

OPL3

Note: This section is relevant to the F256k only. It does not apply to any of the F256 revisions.

The F256k includes a physical OPL3 sound chip, specifically the Yamaha YMF262 sound chip. The OPL3 provides for complex, FM synthesized sound, which allows numerous oscillators to be combined in various ways to generate musical tones. An explanation of the various registers

and functions provided by the OPL3 device is well outside the scope of this manual as it deserves its own book. Only how to access those ports will be covered here.

The OPL3 provides many registers or ports for setting the various parameters. These ports are arranged in an address space of 9 bits (0x000–0x1FF). To access these ports, the CPU must first write the address of the port desired into one of two address registers. It then must write the data to be written to that port into the data register. To maintain compatibility with older versions of the Yamaha FM synthesizer chips, these ports are accessed through two different sets of address registers. For ports 0x000 through 0x0FF, the first address register is used. For ports 0x100 through 0x1FF, the second address register is used. See table 8.7.

Address	R/W	Purpose
0xD580	W	Address pointer register for ports 0x000–0x0FF
0xD581	W	Data register for all ports
0xD582	W	Address pointer register for ports 0x100–0x1FF

Table 8.7: OPL3 Registers

9 Interrupt Controller

The 6809 has three interrupt inputs: IRQ, FIRQ and NMI. On the Jr2 and K2 nearly every device interrupts through IRQ. FIRQ isn't connected, and NMI comes only from the IEC port, when you ask for it (section 9.3).

Many devices share the one IRQ line, so an interrupt controller sits between them and the CPU. It records which devices have raised an interrupt, lets you choose which of them reach the CPU, and lets your handler find the cause without polling every device.

If you write programs for NitrOS-9, the system and its drivers own the interrupt controller; read section 9.5. If you write raw 6809 programs, you set it up yourself.

9.1 Groups and registers

The controller handles up to 32 sources in four groups of eight. Each group has four registers, and each source has the same bit position in all four:

PENDING reads 1 for each source that has raised an interrupt. Write 1 to a bit to clear it; writing 0 leaves it alone, so you can clear one source without touching the others.

MASK decides which pending flags interrupt the CPU. A masked source (1) still sets its pending flag, but doesn't interrupt; an unmasked source (0) does.

EDGE and **POLARITY** decide what sets the pending flag (table 9.2).

The controller holds IRQ low while any pending flag is set and unmasked. Your handler must clear the pending flags it services, or the CPU takes the interrupt again as soon as it returns.

Table 9.1 shows the registers and the source of each bit, and the list after it describes them. The K2 has three sources the Jr2 lacks: VIA 1 for the built-in keyboard, the optical keyboard and the WizNet FIFO. On the Jr2 those bits never become pending.

INT_PENDING_0 ($FE20, read/write, reset $00) Group 0 pending flags. A bit reads 1 when its source has raised an interrupt. Write 1 to a bit to clear it; writing 0 leaves it alone.

INT_CARTRIDGE (bit 7) Interrupt line on the cartridge connector.

INT_DMA (bit 6) DMA transfer finished (when enabled in the DMA control register).

INT_TIMER_1 (bit 5) Timer 1 reached its target value.

Address	Name	7	6	5	4	3	2	1	0
$FE20	INT_PENDING_0	INT_CARTRIDGE	INT_DMA	INT_TIMER_1	INT_TIMER_0	INT_PS2_MOUSE	INT_PS2_KBD	INT_VKY_SOL	INT_VKY_SOF
$FE21	INT_PENDING_1	INT_SDC_INS	INT_VIA1	INT_VIA0	INT_RTC	—			INT_UART
$FE22	INT_PENDING_2	—	INT_HDMI_PIN	INT_WIFI_PIN	INT_NET_PIN	IEC_SREQ_i	IEC_ATN_i	IEC_CLK_i	IEC_DATA_i
$FE23	INT_PENDING_3	—		INT_WIZFI_TX	—	INT_WIZNET	INT_OPT_KBD	INT_MIDI_RX	INT_WIZFI_RX
$FE24–$FE27	INT_POLARITY_0–3	One bit per source							
$FE28–$FE2B	INT_EDGE_0–3	One bit per source							
$FE2C–$FE2F	INT_MASK_0–3	One bit per source							

Table 9.1: Interrupt controller registers

INT_TIMER_0 (bit 4) Timer 0 reached its target value.

INT_PS2_MOUSE (bit 3) PS/2 mouse data.

INT_PS2_KBD (bit 2) PS/2 keyboard data.

INT_VKY_SOL (bit 1) VICKY start of line: the display reached the line set in the line interrupt registers.

INT_VKY_SOF (bit 0) VICKY start of frame.

INT_PENDING_1 ($FE21, read/write, reset $00) Group 1 pending flags. Bits 3–1 have no source.

INT_SDC_INS (bit 7) An SD card was inserted.

INT_VIA1 (bit 6) VIA 1, which scans the K2's built-in keyboard. Not connected on the Jr2.

INT_VIA0 (bit 5) VIA 0 (the joystick ports).

INT_RTC (bit 4) Real-time clock.

INT_UART (bit 0) The serial port (UART).

INT_PENDING_2 ($FE22, read/write, reset $00) Group 2 pending flags: the IEC bus lines and the interrupt pins of the network, Wi-Fi and HDMI parts. Bit 7 has no source. The IEC bits can also raise an NMI (see the text).

INT_HDMI_PIN (bit 6) HDMI interface interrupt pin.

INT_WIFI_PIN (bit 5) Wi-Fi module interrupt pin.

INT_NET_PIN (bit 4) Network module interrupt pin.

IEC_SREQ_i (bit 3) IEC SREQ line.

IEC_ATN_i (bit 2) IEC ATN line.

IEC_CLK_i (bit 1) IEC clock line.

IEC_DATA_i (bit 0) IEC data line.

INT_PENDING_3 ($FE23, read/write, reset $00) Group 3 pending flags: receive and transmit FIFOs. Bit 4 (INT_MIDI_VS_RX in NitrOS-9) and bits 7–6 have no source on either board.

INT_WIZFI_TX (bit 5) The Wi-Fi transmit FIFO became empty.

INT_WIZNET (bit 3) The WizNet FIFO has data. K2 only.

INT_OPT_KBD (bit 2) The optical keyboard FIFO has data. K2 only.

INT_MIDI_RX (bit 1) The MIDI receive FIFO has data.

INT_WIZFI_RX (bit 0) The Wi-Fi receive FIFO has data.

INT_POLARITY_0–3 ($FE24–$FE27, read/write, reset $00) Polarity for groups 0–3 ($FE24–$FE27), one bit per source in the same positions as the pending flags. With EDGE, chooses the condition that sets the pending flag.

INT_EDGE_0–3 ($FE28–$FE2B, read/write, reset $FF) Edge select for groups 0–3 ($FE28–$FE2B). 1 = the flag is set by a change of the input; 0 = it is set while the input is at a level.

INT_MASK_0–3 ($FE2C–$FE2F, read/write, reset $FF) Masks for groups 0–3 ($FE2C–$FE2F). 1 = the source can set its pending flag but doesn't interrupt the CPU; 0 = a pending flag interrupts the CPU.

> **Note:** Some devices have an interrupt enable of their own as well, such as the VIAs, the DMA engine and the line interrupt. Such a device interrupts the CPU only when you enable it in the device and unmask it in the controller.

Edge and polarity

EDGE	POLARITY	The pending flag is set
0	0	while the input is low
0	1	while the input is high
1	0	when the input goes from high to low
1	1	when the input goes from low to high

Table 9.2: Interrupt polarity and edge

After reset every source is edge-triggered on a falling edge (EDGE $FF, POLARITY $00). That suits all the device sources, and NitrOS-9 never changes it. With a level setting, the flag is set again as long as the input stays at that level, even straight after you clear it. The edge and polarity registers matter mainly for the IEC lines in group 2, where you may want to catch a particular change on the bus.

Start of frame and start of line

VICKY raises the start-of-frame interrupt (SOF) once per frame, when it has drawn the last visible line, which makes it the right moment to move sprites or change colors without tearing. It also makes a steady clock: 60 times a second in the 640×480 modes.

The start-of-line interrupt (SOL) happens when the display reaches a line you choose. Set the line and enable the interrupt in VICKY's line interrupt registers (table 7.4); then unmask INT_VKY_SOL.

9.2 Reset state

At power-up and reset the controller masks every source, clears every pending flag, and sets every source to falling-edge triggering. The 6809 itself starts with IRQ and FIRQ masked in its condition code register.

Unmask a source only after you have installed its handler and cleared its pending flag; a flag left over from before can otherwise interrupt at once.

9.3 NMI from the IEC port

Bit 5 of the IEC output register (`$FE81`) routes the IEC interrupts to NMI as well. While the bit is set, any unmasked pending IEC flag (group 2, bits 0–3) pulls NMI low as well as IRQ. While it is clear, the IEC flags use IRQ only. No other source can raise NMI. The bit is clear after reset.

The 6809 ignores NMI until your program has loaded S for the first time. NMI can't be masked in the CPU, so mask the IEC sources in INT_MASK_2 when you don't want it.

9.4 Raw 6809 example: the start-of-frame interrupt

This example counts frames in the top left character of the screen. It builds on the raw skeleton in `examples/raw/skeleton.asm`, which already masks and clears every interrupt and puts the 6809 vectors at the end of its image. It uses the names in `examples/include/wildbits_regs.inc`, plus `INT_VKY_SOF equ $01`.

The handler maps the text screen (block `$FE`) into slot 5 to change the character, then puts slot 5 back. The 6809 saves every register when it takes an IRQ, so the handler needs to save only the slot it changes.

```
IrqHandler  lda   #INT_VKY_SOF
            bita  INT_PENDING_0     is SOF pending?
            beq   @done
            sta   INT_PENDING_0     yes: clear it
            lda   MMU_SLOT_5        save slot 5 ($A000-$BFFF)
            pshs  a
            lda   #$FE              text screen block
            sta   MMU_SLOT_5
            inc   $A000             the character at row 0, column 0
            puls  a
            sta   MMU_SLOT_5        put slot 5 back
@done       rti
```

Point the IRQ vector at the handler in the skeleton's vector table (`fdb IrqHandler` at `$FFF8`), and turn the interrupt on in your setup code:

```
            orcc  #$10              hold off IRQs while we set up
            lda   #INT_VKY_SOF
            sta   INT_PENDING_0     clear any old SOF
            lda   INT_MASK_0
```

```
            anda  #~INT_VKY_SOF     unmask SOF only
            sta   INT_MASK_0
            andcc #~$10             let IRQs in
```

The handler reads `MMU_SLOT_5` to save it; that read returns the LUT selected by EDIT_LUT, so this example assumes EDIT_LUT and ACT_LUT are the same, as after reset.

9.5 Under NitrOS-9

When NitrOS-9 starts, it masks every source and clears every pending flag. Each driver then unmasks the sources it handles, changing only its own bits with a read-modify-write.

Interrupts belong to device drivers. A driver installs its handler with `F$IRQ`, giving the address of the pending register (`INT_PENDING_0` to `INT_PENDING_3`) and a packet with a flip byte, a mask byte for its source, and a priority. When an IRQ arrives, NitrOS-9 checks each installed handler's pending register against its mask and calls the handlers whose bits are set. The handler clears its pending flag and returns. This is how the start-of-line driver installs its handler:

```
            ldd   #INT_PENDING_0    the register to check
            leax  SOL_Pckt,pcr      flip, mask and priority
            leay  SOL_IRQSvc,pcr    the handler
            os9   F$IRQ
            ...
            lda   INT_MASK_0        unmask SOL, keeping the other bits
            anda  #^INT_VKY_SOL
            sta   INT_MASK_0

SOL_Pckt    fcb   %00000000         flip byte
            fcb   INT_VKY_SOL       mask byte
            fcb   $F1               priority
```

A program doesn't handle interrupts itself. It asks a driver to send it a signal when the event happens, and handles the signal with an intercept routine (`F$Icpt`). For example, the `SS.SOLIRQ` SetStat asks the start-of-line driver to signal your process when the display reaches a given line. Don't write the interrupt controller's registers from a program: you would take sources away from the drivers that own them.

10

Tracking Time

Interval Timers

The F256 provides two 24-bit timers. The two timers work on different clocks: timer 0 works off the video dot clock (25.175 MHz), while timer 1 works off the start-of-frame timing (either 60 Hz or 70 Hz, depending on the resolution). The timers have a few features in how they time things:

- they can count up from 0 or down from a starting value
- they can be set to trigger an interrupt on a specific value
- they can either reload a start value or reset the value to 0 on reaching the target value

Address	R/W	Name	7	6	5	4	3	2	1	0
0xFE30	W	T0_CTR	—	—			UP	LD	CLR	EN
0xFE30	R	T0_STAT	—							EQ
0xFE33	R/W	T0_VAL	V23	V22	V21	V20	V19	V18	V17	V16
0xFE32	R/W		V15	V14	V13	V12	V11	V10	V9	V8
0xFE31	R/W		V7	V6	V5	V4	V3	V2	V1	V0
0xFE34	R/W	T0_CMP_CTR	—						RELD	RECLR
0xFE37	R/W	T0_CMP	C23	C22	C21	C20	C19	C18	C17	C16
0xFE36	R/W		C15	C14	C13	C12	C11	C10	C9	C8
0xFE35	R/W		C7	C6	C5	C4	C3	C2	C1	C0
0xFE38	W	T1_CTR	INT_EN	—			UP	LD	CLR	EN
0xFE38	R	T1_STAT	—							EQ
0xFE3B	R/W	T1_VAL	V23	V22	V21	V20	V19	V18	V17	V16
0xFE3A	R/W		V15	V14	V13	V12	V11	V10	V9	V8
0xFE39	R/W		V7	V6	V5	V4	V3	V2	V1	V0
0xFE3C	R/W	T1_CMP_CTR	—						RELD	RECLR
0xFE3F	R/W	T1_CMP	C23	C22	C21	C20	C19	C18	C17	C16
0xFE3E	R/W		C15	C14	C13	C12	C11	C10	C9	C8
0xFE3D	R/W		C7	C6	C5	C4	C3	C2	C1	C0

Table 10.1: Timer Registers

There are five registers for each timer:

CTR the master control register for the timer. There are five flags:

UP if set, the timer will count up. If clear, it will count down.

CLR if set, the timer will reset to 0

LD if set, the timer will be set to the last value written to VAL

EN if set, the timer will count clock ticks

STAT this register (read on the same address as CTR) has just one flag EQ, which indicates if the timer has reached the target value

VAL when read, gives the current value of the timer. When written, sets the value to use when loading the timer.

CMP_CTR this register contains two flags to control what happens when the target value is reached. When RECLR is set, the timer will return to 0 on reaching the target value. When RELD is set, the timer will be set to the last value written to VAL.

CMP this register contains the target value for comparison

Real Time Clock

For programs needing to keep track of time, F256 provides a real time clock chip (RTC), the bq4802. This chip, keeps track of the year (including century), month, day, hour (in 12 or 24 hour mode), minute, and second. The coin cell battery on the F256 motherboard is to provide power to the RTC so it can continue tracking time even when the F256 is turned off or unplugged. Additionally, the RTC can send interrupts to the CPU, either periodically or at a specific time.

The RTC is relatively straightforward to use, but one potentially tricky thing to keep in mind is that there is a specific procedure to follow when reading or writing the date-time. As well as the registers the CPU can access, the RTC has internal registers which are constantly updating as time progresses. Normally, the internal registers update their external counterparts, but this should not be allowed to happen while the CPU is getting or setting the externally facing registers. So, to access the external registers, the program must first disable the automatic updates to the external registers. Then it can read or write the external registers. Then it can re-enable the automatic updates. If the program has changed the registers, when updates are re-enabled the data in the external registers will be sent to the internal registers in one action. This keeps the time information consistent.

There are 16 registers for the RTC (see table 10.2). There is a register each for century, year, month, day of the week (*i.e.* Sunday-Saturday), day, hour, minute, and second. Each one is expressed in binary-coded-decimal, meaning the lower four bits are the ones digit (0-9), and the upper bits are the 10s digit. In most cases, the upper digit is limited (*e.g.* seconds and minutes can only have 0-5 as the tens digit). For seconds, minutes, hours, and day there is a separate alarm register, which will be described later. Finally, there are the four registers for rates, enabled, flags, and control:

The Enables register has four separate enable bits:

Address	R/W	Name	7	6	5	4	3	2	1	0
0xFE40	R/W	Seconds	0	second 10s digit			second 1s digit			
0xFE41	R/W	Seconds Alarm	0	second 10s digit			second 1s digit			
0xFE42	R/W	Minutes	0	minute 10s digit			minute 1s digit			
0xFE43	R/W	Minutes Alarm	0	minute 10s digit			minute 1s digit			
0xFE44	R/W	Hours	AM/PM	0	hour 10s digit		hour 1s digit			
0xFE45	R/W	Hours Alarm	AM/PM	0	hour 10s digit		hour 1s digit			
0xFE46	R/W	Days	0	0	day 10s digit		day 1s digit			
0xFE47	R/W	Days Alarm	0	0	day 10s digit		day 1s digit			
0xFE48	R/W	Day of Week	0	0	0	0	0	day of week digit		
0xFE49	R/W	Month	0	0	0	month 10s digit	month 1s digit			
0xFE4A	R/W	Year	year 10s digit				year 1s digit			
0xFE4B	R/W	Rates	0	WD			RS			
0xFE4C	R/W	Enables	0	0	0	0	AIE	PIE	PWRIE	ABE
0xFE4D	R/W	Flags	0	0	0	0	AF	PF	PWRF	BVF
0xFE4E	R/W	Control	0	0	0	0	UTI	STOP	12/24	DSE
0xFE4F	R/W	Century	century 10s digit				century 1s digit			

Table 10.2: Real Time Clock Registers

AIE if set (1), the alarm interrupt will be enabled. The RTC will raise an interrupt when the current time matches the time specified in the alarm registers.

PIE if set (1), the RTC will raise an interrupt periodically, where the period is specified by the RS field.

PWRIE if set (1), the RTC will raise an interrupt on a power failure (not relevant to the F256).

ABE if set (1), the RTC will allow alarm interrupts when on battery backup (not relevant to the F256).

The Flags register has four separate flags, which generally reflect why an interrupt was raised:

AF if set (1), the alarm was triggered

PF if set (1), the periodic interrupt was triggered

PWRF if set (1), the power failure interrupt was triggered

BVF if set (1), the battery voltage is within safe range. If clear (0), the battery voltage is low, and the time may be invalid.

The Control register has four bits which change how the RTC operates:

UTI if set (1), the update of the externally facing registers by the internal timers is inhibited. In order to read or write those registers, the program must first set UTI and then clear it when done.

STOP this bit allows for a battery saving feature. If it is clear (0) before the system is powered down, it will avoid draining the battery and may stop tracking the time. If it is set (1), it will keep using the battery as long as possible.

12/24 sets whether the RTC is using 12 or 24 hour accounting.

DSE if set (1), daylight savings is in effect.

The Rates register controls the watchdog timer and the periodic interrupt. The watchdog timer is not really relevant to the F256, but it monitors for activity and raises an interrupt if activity has not been seen within a certain amount of time (specified by the WD field). The periodic interrupt will be raised repeatedly, the period of which is set by the RS field (see table 10.3).

RS3	RS2	RS1	RS0	Period
0	0	0	0	None
0	0	0	1	30.5175 μs
0	0	1	0	61.035 μs
0	0	1	1	122.070 μs
0	1	0	0	244.141 μs
0	1	0	1	488.281 μs
0	1	1	0	976.5625 μs
0	1	1	1	1.95315 ms
1	0	0	0	3.90625 ms
1	0	0	1	7.8125 ms
1	0	1	0	15.625 ms
1	0	1	1	31.25 ms
1	1	0	0	62.5 ms
1	1	0	1	125 ms
1	1	1	0	250 ms
1	1	1	1	500 ms

Table 10.3: RTC Periodic Interrupt Rates

Example: Display the Time

In this example, we will read the time from the real time clock chip and print it out to the screen in *hh:mm:ss* format. The basic procedure is fairly simple: first the code disables the update of the transfer registers, then the code reads the hours and prints them, then the code reads the minutes and prints them, then the code fetches the seconds and prints them. Finally, the code re-enables the update of the transfer registers by dropping the UTI flag.

NOTE: This code resets the MMU I/O page to 0 before it tries to read from the clock chip. This is just to allow for the possibility of the kernel routines changing the I/O page without restoring it to 0.

```
ok_cint = $FF81       ; OpenKernal call to initialize the screen
ok_cout = $FFD2       ; OpenKernal call to print the character code in A
```

```
RTC_SECS = $D690      ; RTC Seconds register
RTC_MINS = $D692      ; RTC Minutes register
RTC_HOURS = $D694      ; RTC Hours register

RTC_CTRL = $D96E      ; RTC Control register
RTC_24HR = $02       ; 12/24 hour flag (1 = 24 Hr, 0 = 12 Hr)
RTC_STOP = $04       ; 0 = STOP when power off, 1 = run from battery when power off
RTC_UTI = $08       ; Update Transfer Inhibit

start:      jsr ok_cint                ; Initialize the text screen

            stz MMU_IO_CTRL            ; Make sure we're on I/O page 0

            lda RTC_CTRL               ; Stop the update of the RTC registers
            ora #RTC_UTI | RTC_24HR
            sta RTC_CTRL

            stz MMU_IO_CTRL    ; Make sure we're on I/O page 0

            lda RTC_HOURS              ; Print the hours
            jsr putbcd

            lda #':'
            jsr ok_cout

            stz MMU_IO_CTRL    ; Make sure we're on I/O page 0

            lda RTC_MINS               ; Print the minutes
            jsr putbcd

            lda #':'
            jsr ok_cout

            stz MMU_IO_CTRL    ; Make sure we're on I/O page 0

            lda RTC_SECS               ; Print the seconds
            jsr putbcd

            stz MMU_IO_CTRL    ; Make sure we're on I/O page 0

            lda RTC_CTRL               ; Reenable the update of the registers
            and #~RTC_UTI
            sta RTC_CTRL
```

Since the time registers of the clock chip are encoded in binary-coded-decimal, printing is relatively straightforward, and is handled by a simple `putbcd` subroutine:

```
;
; Print a BCD number to the screen
;
putbcd:     pha                         ; Save the number
            and #$F0                    ; Isolate the upper digit
            lsr a
            lsr a
            lsr a
            lsr a

            clc                         ; Convert to ASCII
            adc #'0'
            jsr ok_cout                 ; And print

            pla                         ; Get the full number back
            and #$0F                    ; Isolate the lower digit

            clc                         ; Convert to ASCII
            adc #'0'
            jsr ok_cout                 ; And print

            rts
```

11 Versatile Interface Adapter

The F256 includes a Western Design Center WDC65C22 versatile interface adapter or VIA. The VIA provides several useful features for I/O and timing:

- Two independent I/O ports of eight parallel bits (PA, and PB).
- Four handshake control lines (CA1, CA2, CB1, and CB2)
- Programmable serial register for serial I/O operations
- Two independent timer counters

On the F256jr, the VIA is connected to header which is compatible with the keyboard header on the Commodore VIC-20 and C-64. This means that a Commodore compatible keyboard could be connected to the F256 and used for keyboard input with appropriate programming. The VIA also provides access to the two Atari-style joystick ports. The pins could also be used for general purpose I/O, although the voltage levels are for 3.3 volt logic instead of the 5 volt logic used in older 8-bit machines. The internal circuitry of the VIA's port A and port B I/O pins is shown in figure 11.1.

> **Note:** While the F256jr has a single VIA, the F256k has two VIA chips. The second VIA chip is located at 0xDB00. The purpose of this second VIA is to manage the built-in keyboard of the F256k, while the first VIA is used solely for the joystick. On the F256jr, the single VIA handles either the joystick or the keyboard. See page 84 for a more complete description of the keyboard.

A complete description of the VIA would be rather long, so this guide will merely list out the register addresses and provide a quick break-down on the register functions. For a complete description, please see the data sheet from Western Design Center. See table 11.1 for a listing of all the VIA registers.

IORA Input/Output Register for Port A. The eight bits correspond to the eight pins on port A.

DDRA Data Direction Register for Port A. Each bit configures the corresponding pin to be input (0) or output (1).

IORB Input/Output Register for Port B. The eight bits correspond to the eight pins on port B.

Address	R/W	Name	Purpose
0xDC00	R/W	IORB	Port B data
0xDC01	R/W	IORA	Port A data
0xDC02	R/W	DDRB	Port B Data Direction Register
0xDC03	R/W	DDRA	Port A Data Direction Register
0xDC04	R/W	T1C_L	Timer 1 Counter Low
0xDC05	R/W	T1C_H	Timer 1 Counter High
0xDC06	R/W	T1L_L	Timer 1 Latch Low
0xDC07	R/W	T1L_H	Timer 1 Latch High
0xDC08	R/W	T2C_L	Timer 2 Counter Low
0xDC09	R/W	T2C_H	Timer 2 Counter High
0xDC0A	R/W	SDR	Serial Data Register
0xDC0B	R/W	ACR	Auxiliary Control Register
0xDC0C	R/W	PCR	Peripheral Control Register
0xDC0D	R/W	IFR	Interrupt Flag Register
0xDC0E	R/W	IER	Interrupt Enable Register
0xDC0F	R/W	IORA2	Port A data (no handshake)

Table 11.1: VIA Registers

Name	7	6	5	4	3	2	1	0
ACR	T1_CTRL		T2_CTRL	SR_CTRL			PBL_EN	PAL_EN
PCR	CB2_CTRL			CB1_CTRL	CA2_CTRL			CA1_CTRL
IFR	IRQF	T1F	T2F	CB1F	CB2F	SRF	CA1F	CA2F
IER	SET	T1E	T2E	CB1E	CB2E	SRE	CA1E	CA2E

Table 11.2: VIA Control Registers

DDRB Data Direction Register for Port B. Each bit configures the corresponding pin to be input (0) or output (1).

T1C_L, T1C_H Timer 1 counter value

T1L_L, T1L_H Timer 1 latch

T2C_L, T2C_H Timer 2 counter value

SDR is the shift register. Serial input may be read here, or data may be written here to be shifted out.

ACR Auxiliary Control Register. Contains fields to control the function of timer 1, timer 2, the shift register, and how Port A and Port B latch data. See table 11.2 for details.

PCR Peripheral Control Register. Contains fields to control how the CA1, CA2, CB1, and CB2 handshake pins are used. See table 11.2 for details.

IFR Interrupt Flag Register. Contains flags indicating which condition triggered an interrupt request. Possible conditions are timer 1, timer 2, CB1, CB2, CA1, CA2, and shift register complete. See table 11.2 for details.

IER Interrupt Enable Register. Contains flags to enable or disable interrupts based on the different possible conditions. See table 11.2 for details.

IORA2 Same as IOPA except that the built-in handshaking capability is not used.

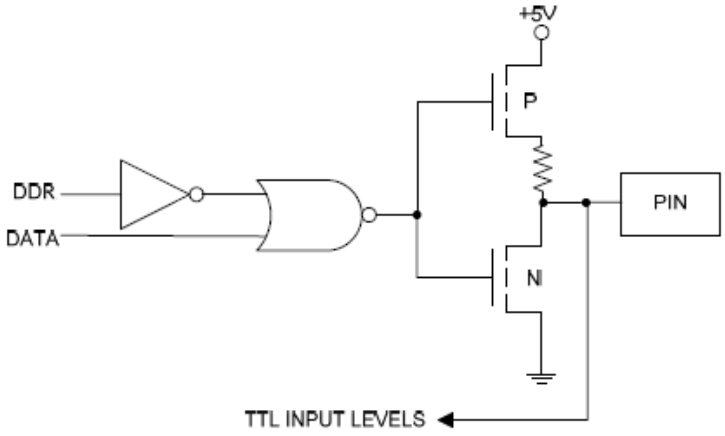


Figure 11.1: VIA Pin Internal Circuitry

12 Game Controllers

The F256 allows you to connect either Atari style joysticks or NES/SNES style gamepads as game controllers. The two different styles of controllers are used differently and are supported by different registers on the F256. Note: Atari style analog devices are not supported (*e.g.* paddles, analog joysticks, analog mice).

Atari Style Joysticks

The F256jr has two IDC headers that can be connected to a DE-9 socket to allow Atari style joysticks to be used (see figure: 12.1 for the pinouts). Joystick header 0 is wired to the pins of Port B of the VIA (see page 11), and joystick header 1 is connected to Port A. The various joystick switches are connected to the ports in same manner as on the C-64, with the exception that more buttons are supported (see table: 12.1). On the F256k, the DE-9 connectors are supplied on the board and are on the right-hand side of the case; there is no need for an adapter cable.

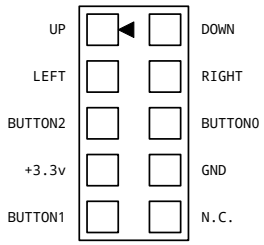


Figure 12.1: Joystick Port Pinouts

In order to use the joysticks, the DDR bits for the ports must be set to 0 for input. Then the input/output register for the port may be read. If a button or switch is closed on the joystick, the corresponding bit in the I/O register will be clear (0). If the button is not pressed, the bit will be set (1).

As a reminder: be aware that the WDC65C22 on the F256 is being used with a 3.3 volt supply. This means that any device plugged into the joystick ports should be 3.3 volt tolerant and should not raise any pin above 3.3 volts, otherwise damage could occur.

7	6	5	4	3	2	1	0
—	BUTTON2	BUTTON1	BUTTON0	RIGHT	LEFT	DOWN	UP

Table 12.1: Joystick Flags

Example: Displaying Joystick 1

In this example, we will poll joystick 1 and print out the state of all the buttons by printing the byte we read from the joystick port as a simple binary number. The example will try to be a little smart by only printing the value when the value has changed. NOTE: this example expects OpenKernal to be installed, and will call two of its routines for initializing the screen and printing a character.

First, we initialize the screen, the variable we use to track the old value of the joystick port, and the VIA (setting port A to be an input port):

```
ok_cint = $FF81                         ; OpenKernal: init the screen
ok_cout = $FFD2                         ; OpenKernal: print the character in A

; Variables

* = $0080

value:      .byte ?                     ; Variable for the old joystick value
prv:        .byte ?                     ; Copy of value for printing

* = $e000

start:      jsr ok_cint                 ; Set up the screen

            lda #$FF                    ; Set the previous value to $FF
            sta value

            stz MMU_IO_CTRL             ; Switch to I/O Page 0

            lda #$00                    ; Set VIA Port A to input
            sta VIA_DDRA
```

Next, we print the OpenKernal code to clear the screen, and we print out the byte in `value` as a binary number.

```
loop1:      lda #147                    ; Print the CBM clear screen code
            jsr ok_cout

            lda value                   ; Copy the value to prv
            sta prv

            ldx #8                      ; Loop for all eight bits
loop2:      asl prv                     ; Shift MSB into the carry
```

```
            bcc is0                     ; If it's 0, print '0'

            lda #'1'                    ; Otherwise, print '1'
            jsr ok_cout
            bra repeat                  ; And go to the next bit

is0:        lda #'0'                    ; Print '0'
            jsr ok_cout

repeat:     dex                         ; Count down
            bne loop2                   ; Repeat until we've done all 8 bits
```

Next, we read the value of port A. If it is different from `value`, we save it to `value` and go back to print the byte we read. Otherwise, we keep waiting and polling the joystick port.

```
            stz MMU_IO_CTRL             ; Switch to I/O Page 0

wait:       lda VIA_IORA                ; Get the status of port A
            cmp value                   ; Is it different from before?
            beq wait                    ; Yes: keep waiting

            sta value                   ; Save this value as the previous one
            bra loop1                   ; And go to print it
```

NES/SNES Gamepads

The F256 also provides support for NES/SNES compatible gamepads. NES gamepads work a little differently from Atari style joysticks. Where Atari style joysticks are directly readable through the VIA ports, NES gamepads communicate to the F256 through a serial interface. To read the gamepad, a program needs to first send the signal to the gamepad to capture the status of all the buttons, then the program needs to trigger the system to transfer the button status over the serial interface to the computer. This transfer takes a few clock cycles, since it is done serially, so the program needs to wait until the NES registers indicate that the transfer is done.

Before any transfer is done, the program must specify (by setting or clearing the MODE bit) whether the gamepad is NES compatible or SNES compatible. NES gamepads only send 8 bits of data, but SNES controllers send 12 bits. The NES/SNES data register are arranged differently depending on the value of MODE.

The process the program follows to read the state of the gamepad is:

1. Set NES_EN of NES_CTRL to enable the NES/SNES support (see table 12.2) and set or clear MODE, to choose between NES mode or SNES mode.

2. Set NES_TRIG of NES_CTRL to sample the buttons and transfer the data to the registers.

3. Read NES_STAT and wait until the DONE bit is set

4. Check the appropriate NES or SNES control registers (see table 12.3)

Address	R/W	Name	7	6	5	4	3	2	1	0
0xD880	W	NES_CTRL	NES_TRIG	—				MODE	—	NES_EN
0xD880	R	NES_STAT	NES_TRIG	DONE	—			MODE	—	NES_EN

Table 12.2: NES/SNES Gamepad Registers

5. Clear NES_TRIG

NES_CTRL Set (1) to start the process of sampling the buttons.

DONE If set (1), the gamepad status has been read into the registers, and is available for reading

MODE If set (1), the gamepad is expected to be an SNES compatible controller. If clear (0), the gamepad is expected to be an NES compatible controller.

NES_EN If set (1), enables NES/SNES controller support.

MODE	Address	Pad	7	6	5	4	3	2	1	0
0	0xD884	0	A	B	SELECT	START	UP	DOWN	LEFT	RIGHT
	0xD886	1	A	B	SELECT	START	UP	DOWN	LEFT	RIGHT
	0xD888	2	A	B	SELECT	START	UP	DOWN	LEFT	RIGHT
	0xD88A	3	A	B	SELECT	START	UP	DOWN	LEFT	RIGHT
1	0xD884	0	B	Y	SELECT	START	UP	DOWN	LEFT	RIGHT
	0xD885		—				A	X	L	R
	0xD886	1	B	Y	SELECT	START	UP	DOWN	LEFT	RIGHT
	0xD887		—				A	X	L	R
	0xD888	2	B	Y	SELECT	START	UP	DOWN	LEFT	RIGHT
	0xD889		—				A	X	L	R
	0xD88A	3	B	Y	SELECT	START	UP	DOWN	LEFT	RIGHT
	0xD88B		—				A	X	L	R

Table 12.3: NES/SNES Data Registers

NOTE: If you want to use NES/SNES controllers with the F256, you will need to add an adapter to convert the common port to NES and SNES connectors. The Wildbits shop has the adapter, which comes in one of two flavors: one for the F256 (which has an IDC pin header on the board for the NES/SNES controllers), and one for the F256k (which has a 9 pin mini-DIN connector on the back).

13 SD Card Interface

TinyVicky includes an interface to the external and internal SD card ports on F256. This interface provides access to the SPI bus interface SD cards support. This interface will allow a program to exchange bytes of data with an SD card using one of two clock speeds for the transfer rate (400 kHz or 12.5 MHz). Use of these registers requires an understanding of the SD card protocols and conventions, which are really outside the scope of this manual. So only the basic information about the control registers are provided here.

Address	R/W	7	6	5	4	3	2	1	0
0xFE90	RW	SPI_BUSY	—					SPI_CLK	CS_EN
0xFE91	RW	SPI_DATA							

Table 13.1: External SD Card Interface Registers

Address	R/W	7	6	5	4	3	2	1	0
0xFF00	RW	SPI_BUSY	—					SPI_CLK	CS_EN
0xFF01	RW	SPI_DATA							

Table 13.2: Internal SD Card Interface Registers

CS_EN This bit controls the chip select input on the SD card. If clear (0), the SD card is disabled. If set (1), the SD card is enabled.

SPI_CLK This bit controls the clock speed for the SPI interface to the SD card. If set (1), the clock speed is 400 kHz. If clear (0), the clock speed is 12.5 MHz.

SPI_BUSY This read only bit indicates if the SPI bus is busy exchanging bits with the SD card. The SPI_DATA register will not be ready for access while SPI_BUSY is set (1).

SPI_DATA this register is for the data to exchange with the SD card. A byte written to this register will be send to the SD card. The data read from this register are the bits received from the SD card. If SPI_BUSY is set, the program must way until SPI_BUSY is clear before reading or writing data to this register

NOTE: The system control registers have two bits relevant to the SD card interface: SD_WP, which indicates the write-protect status of the card, and SD_CD which indicates if a card is detected in the slot. See table 17.1 for details.

14 Keyboard and Mouse

The F256 provides a single PS/2 port for use with either a keyboard or a mouse. This port is accessed through five registers, which provide very simple access to a PS/2 device. The F256 does not have a full PS/2 controller, but instead provides mostly raw access to the data stream. It does make some attempt to translate set 2 scan codes to ASCII character code, although raw scan codes may be read instead. See table 14.1 for details.

Address	R/W	Name	7	6	5	4	3	2	1	0
0xFE50	R/W	PS2_CTRL	—		MCLR	KCLR	M_WR	—	K_WR	—
0xFE51	R/W	PS2_OUT	Data to send to keyboard							
0xFE52	R	KBD_IN	Data from the keyboard input FIFO							
0xFE53	R	MS_IN	Data from the mouse input FIFO							
0xFE54	R	PS2_STAT	K_AK	K_NK	M_AK	M_NK	—		MEMP	KEMP

Table 14.1: PS/2 Port Registers

K_WR set to 1 then 0 to send a byte written on PS2_OUT to the keyboard

M_WR set to 1 then 0 to send a byte written on PS2_OUT to the mouse

KCLR set to 1 then 0 to clear the keyboard input FIFO queue.

MCLR set to 1 then 0 to clear the mouse input FIFO queue.

K_AK when 1, the code sent to the keyboard has been acknowledged

K_NK when 1, the code sent to the keyboard has resulted in an error

M_AK when 1, the code sent to the mouse has been acknowledged

M_NK when 1, the code sent to the mouse has resulted in an error

KEMP when 1, the keyboard input FIFO is empty

MEMP when 1, the mouse input FIFO is empty

Mouse Support

The F256 provides special support for a PS/2 mouse, including support for a hardware mouse pointer.

Mouse Pointer

The F256 provides for a grayscale hardware mouse pointer. The pointer is a 16×16 grayscale image of 256 levels. Each pixel of the image is a single byte. The bitmap data is stored in the address range 0x18_0C00–0x18_0FFF (Block $C0 Offset $0C00).

The position of the mouse pointer is controlled in one of two ways. In the default approach (MODE = 0), the system software will monitor mouse movements, determine the mouse position programmatically, and set the TinyVicky mouse position registers directly. In the legacy approach (MODE = 1), the system software will receive the three byte PS/2 mouse data packet and set the TinyVicky mouse PS2_BYTE registers. In this legacy mode, TinyVicky will interpret the mouse packets and track the mouse position for the system. This approach is less work for the system software, but is less flexible.

Address	R/W	7	6	5	4	3	2	1	0
0xFEA0	W	—						MODE	EN
0xFEA2	RW	X15	X14	X13	X12	X11	X10	X9	X8
0xFEA3	RW	X7	X6	X5	X4	X3	X2	X1	X0
0xFEA4	RW	Y15	Y14	Y13	Y12	Y11	Y10	Y9	Y8
0xFEA5	RW	Y7	Y6	Y5	Y4	Y3	Y2	Y1	Y0
0xFEA6	W	PS2_BYTE_0							
0xFEA7	W	PS2_BYTE_1							
0xFEA8	W	PS2_BYTE_2							

Table 14.2: Mouse Pointer Registers

EN if set (1), the mouse pointer is displayed. If clear (0), the mouse pointer is not displayed

MODE if clear (0), the mouse position is specified by setting the X and Y registers. If set (1), the program must pass along the 3 byte PS/2 mouse packet to the packet registers (this is a legacy mode).

X this is the X coordinate of the mouse and both readable and writable if MODE is clear (0)

Y this is the Y coordinate of the mouse and both readable and writable if MODE is clear (0)

PS2_BYTE_0 the first byte of the PS/2 mouse message packet. Only used if MODE is set (1).

PS2_BYTE_1 the second byte of the PS/2 mouse message packet. Only used if MODE is set (1).

PS2_BYTE_2 the third byte of the PS/2 mouse message packet. Only used if MODE is set (1).

F256k Keyboard

Note: this section pertains to the F256k only.

The F256k includes a second WDC65C22 VIA chip at 0xFFB0 (see page 73 for details on the VIAs) which manages the keyboard input from the F256k's built-in keyboard. The F256K's keyboard is a matrix keyboard. Except for the RESTORE key, all keys on the keyboard are arranged in a matrix with the columns assigned to VIA1's PB pins and the rows assigned to VIA1's PA pins. VIA0's PB7 pin is also used for a column. See figure 14.1.

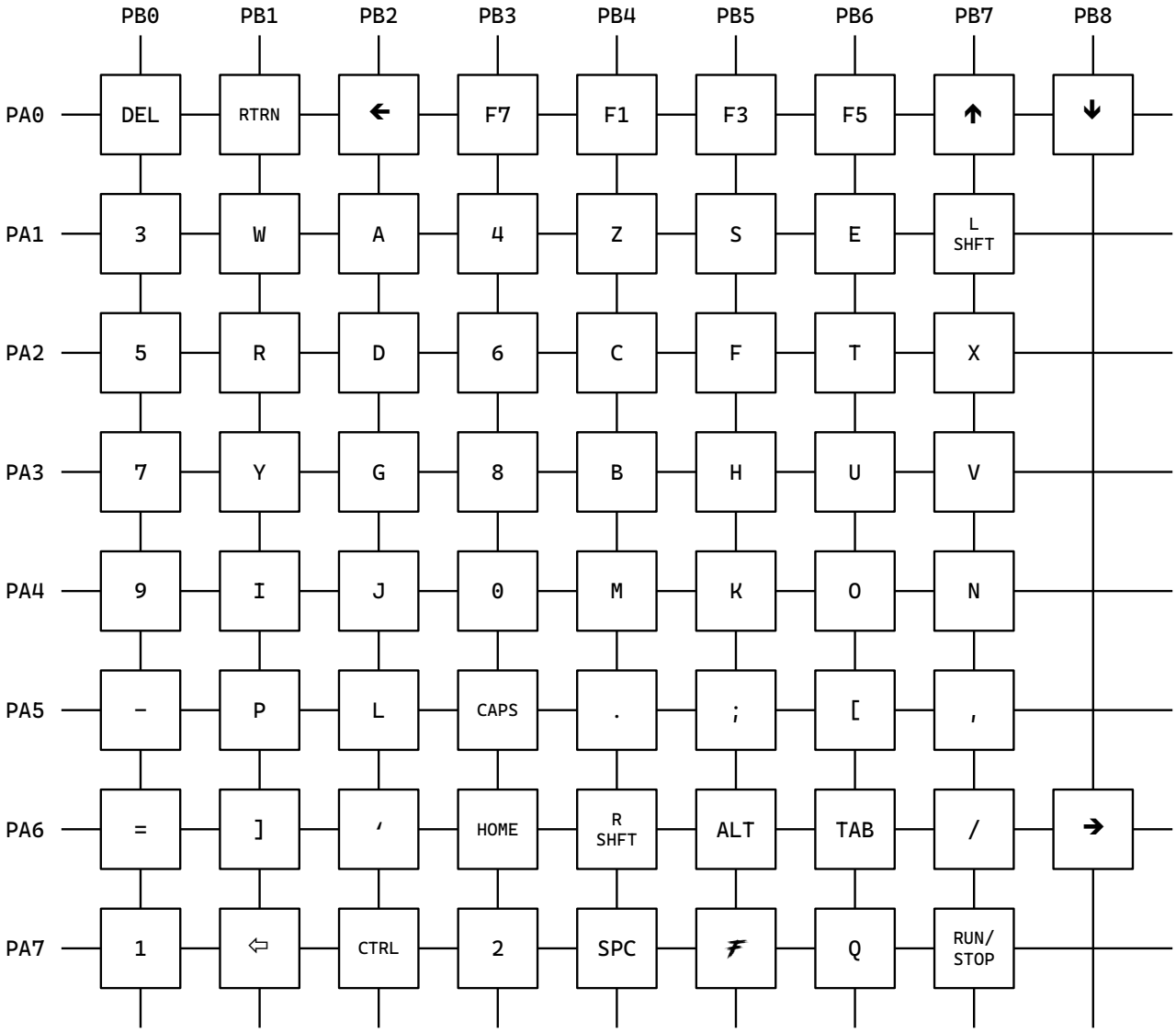


Figure 14.1: F256k Keyboard Matrix

Indicator	Address	R/W	Color
Power	0xD6A7	W	Blue
	0xD6A8	W	Green
	0xD6A9	W	Red
Media	0xD6AA	W	Blue
	0xD6AB	W	Green
	0xD6AC	W	Red
Shift	0xD6AD	W	Blue
	0xD6AE	W	Green
	0xD6AF	W	Red

Table 14.3: F256k Keyboard LEDs

The F256k keyboard includes three indicator RGB LEDs that can be set under program control. The three LEDs are for power, media access (SD and IEC), and the shift lock indicator. Each LED has three single byte registers to set the red, green, and blue component intensities for the desired colors. See table 14.3.

15 Serial and Wi-Fi Port

The F256 has a simple UART for serial communications. This UART can be used to provide an RS-232 serial connection (via an IDC header on the board compatible with IDC to DE-9 cables) or a Wi-Fi serial connection using an ESP8266 Feather adapter board. The UART is compatible with the standard 16750.

Address	R/W	Name	7	6	5	4	3	2	1	0
DLAB = 0										
0xD630	R	RXD	RX_DATA							
0xD630	W	TXR	TX_DATA							
0xD631	R/W	IER	—				STAT	ERR	TXE	RXA
0xD632	R	ISR	—				STAT	ERR	TXE	RXA
0xD632	W	FCR	RXT		FIFO64	—	—	TXR	RXR	FIFOE
0xD633	R/W	LCR	DLAB	—	PARITY			STOP	DATA	
0xD634	R/W	MCR	—			LOOP	OUT2	OUT1	RTS	DTR
0xD635	R	LSR	ERR	TEMT	THRE	BI	FE	PE	OE	DR
0xD636	R/W	MSR	DCD	RI	DSR	CTS	DDCD	TERI	DDSR	DCTS
0xD637	R	SPR	scratch data							
DLAB = 1										
0xD630	R/W	DLL	DIV7	DIV6	DIV5	DIV4	DIV3	DIV2	DIV1	DIV0
0xD631	R/W	DLH	DIV15	DIV14	DIV13	DIV12	DIV11	DIV10	DIV9	DIV8
0xD632	W	PSD	prescaler division							

Table 15.1: UART Registers

RXD (read only) register contains data from the receive FIFO

TXR (write only) writing a byte stores it in the transmission FIFO to be sent over the serial connection

IER this is the interrupt enable register. There are flags for each of the four conditions that the UART can use to trigger an interrupt

ISR this is the interrupt STAT register. There are flags for each of the four conditions that can trigger an interrupt

FCR FIFO control register. This register controls the FIFOs for transmission and receiving:

RXT sets the number of characters in the receive FIFO to trigger an interrupt. See table: 15.5.

FIFO64 enables the 64 byte FIFO

TXR if set, clear the transmition FIFO

RXR if set, clear the receive FIFO

FIFOE if set, the FIFOs are enabled. Otherwise, only a single character can be waiting to send or pending a read

LCR1	LCR0	Length
0	0	5
0	1	6
1	0	7
1	1	8

Table 15.2: UART Data Length

LCR2	Stop Bits
0	1
1	1.5 or 2

Table 15.3: UART Stop Bits

LCR5	LCR4	LCR3	Parity
—	—	0	NONE
0	0	1	ODD
0	1	1	EVEN
1	0	1	MARK
1	1	1	SPACE

Table 15.4: UART Parity

FCR7	FCR6	Trigger Level (bytes)
0	0	1
0	1	4
1	0	8
1	1	14

Table 15.5: UART RX FIFO Trigger

BPS	Divisor
300	5,244
600	2,622
1,200	1,311
1,800	874
2,000	786
2,400	655
3,600	437
4,800	327
9,600	163
19,200	81
38,400	40
57,600	27
115,200	13

Table 15.6: UART Divisors

16

Direct Memory Access

The DMA engine can either write a specific byte to RAM or copy a set of bytes from one location in RAM to another. The DMA engine can also treat memory as being arranged either linearly (that is, as a certain number of consecutive locations) or as a rectangle (the data is a rectangular area of an image).

Linear Data

Linear data (or "1D", if you prefer) is just a single block of sequential memory locations. When filling or copying data linearly, you need a destination address (and a source address if copying), and a count of bytes to copy. That is really all there is to it.

Rectangular Data

Rectangular data (or "2D") is a bit more complicated and is meant to be working with image data. With a bitmap, the pixel bytes are arranged in memory left to right and top to bottom. If the image starts at address a and is w pixels wide, then the pixel at (x, y) can be found at location $a + y \times w + x$. Rectangular fills and copies are meant to work on data that is arranged in this fashion. In this case, you can use DMA to fill or copy a rectangular area within that image. As with linear fills and copies, you will need a destination address (and source address if doing a copy), but instead of a count of bytes you need the width and height of the rectangular areas affected. But you need one other thing, too. You need to tell the DMA the geometry of the over-all image... you need to tell it the width of the image containing the rectangular areas. This is called the "stride" and effectively tells the DMA how many pixels to skip between lines when it finishes one line of the rectangle before getting to the next line.

START set to trigger the DMA

INT_EN enables triggering an interrupt when DMA is complete

FILL when set, DMA will write a specific byte to memory. When clear, DMA will copy data from a source address to the destination address

2D when set, DMA copies or fills a rectangular region of memory. When clear, DMA copies or fills a certain number of sequential bytes

Address	R/W	7	6	5	4	3	2	1	0
0xFEC0	R/W	START	—			INT_EN	FILL	2D	ENABLE
0xFEC1	W	FD7	FD6	FD5	FD4	FD3	FD2	FD1	FD0
0xFEC1	R	BUSY	—						
0xFEC4	R/W	—					SA18	SA17	SA16
0xFEC5	R/W	SA15	SA14	SA13	SA12	SA11	SA10	SA9	SA8
0xFEC6	R/W	SA7	SA6	SA5	SA4	SA3	SA2	SA1	SA0
0xFEC8	R/W	—					DA18	DA17	DA16
0xFEC9	R/W	DA15	DA14	DA13	DA12	DA11	DA10	DA9	DA8
0xFECA	R/W	DA7	DA6	DA5	DA4	DA3	DA2	DA1	DA0

Table 16.1: DMA Registers (Part 1)

Address	R/W	7	6	5	4	3	2	1	0
0xFECF	R/W	—					DZ18	DZ17	DZ16
0xFECE	R/W	DZ15	DZ14	DZ13	DZ12	DZ11	DZ10	DZ9	DZ8
0xFECD	R/W	DZ7	DZ6	DZ5	DZ4	DZ3	DZ2	DZ1	DZ0
0xFED0	R/W	W15	W14	W13	W12	W11	W10	W9	W8
0xFED1	R/W	W7	W6	W5	W4	W3	W2	W1	W0
0xFED2	R/W	H15	H14	H13	H12	H11	H10	H9	H8
0xFED3	R/W	H7	H6	H5	H4	H3	H2	H1	H0
0xFED4	R/W	SS15	SS14	SS13	SS12	SS11	SS10	SS9	SS8
0xFED5	R/W	SS7	SS6	SS5	SS4	SS3	SS2	SS1	SS0
0xFED6	R/W	SD15	SD14	SD13	SD12	SD11	SD10	SD9	SD8
0xFED7	R/W	SD7	SD6	SD5	SD4	SD3	SD2	SD1	SD0

Table 16.2: DMA Registers (Part 2)

ENABLE set to enable DMA

FD the byte to be written to memory when FILL is set

BUSY status bit set when DMA is busy copying data

SA the 19 bit source address (must be a location in the first 512KB of RAM). Only relevant when FILL is clear.

DA the 19 bit destination address (must be a location in the first 512KB of RAM)

DZ the number of bytes to copy for 1D operations

W the width of the rectangle of data to copy (only available when 2D is set)

H the height of the rectangle of data to copy (only available when 2D is set)

SS the width of the “stride” for the source bitmap (only available for 2D copy operations)

SD the width of the “stride” for the destination bitmap (only available when 2D is set)

The term “stride” might be a little confusing. It is a concept only relevant for 2D DMA operations. For 2D operations, the DMA engine is always copying or filling a rectangular area of a given width and height. The width of the rectangle need not be the full size of the overall bitmap image. If a program performs a DMA operation on a 32×32 area of the screen, the DMA engine will need to skip many pixels on each affected line. Thus, the program needs to inform the DMA engine how many pixels wide the bitmap is. For example, if a program is filling 32×32 rectangle in a 320×240 bitmap, it needs to tell the DMA engine that the width of the bitmap is 320. This number is the “stride” for the operation. The DMA engine will operate on 32 pixels, then skip $320 - 32$ pixels to get to the next set of 32 pixels to copy or fill. Figure 16.1 illustrates how the various addresses, sizes, and strides work in the case of a 2D copy operation.

For 2D fill operations, only the destination stride (SD) is needed. The destination stride specifies how wide the bitmap being altered is. For 2D copy operations, both the destination stride and the source stride (SS) are needed. The source stride specifies how wide the bitmap is that provides the source data. Why would you ever have a source and destination stride be different? As a simple example, let’s say a program needs to copy graphical font data to a bitmap for the screen in 320×240 mode. The characters are in cells that are 8 pixels wide by 16 pixels deep, and that the characters are arranged vertically. So the over all image for the characters is 8 pixels wide by 4,096 pixels high. In that case, the source stride is 8, but the destination stride is 320 (since it is the bitmap for the full screen).

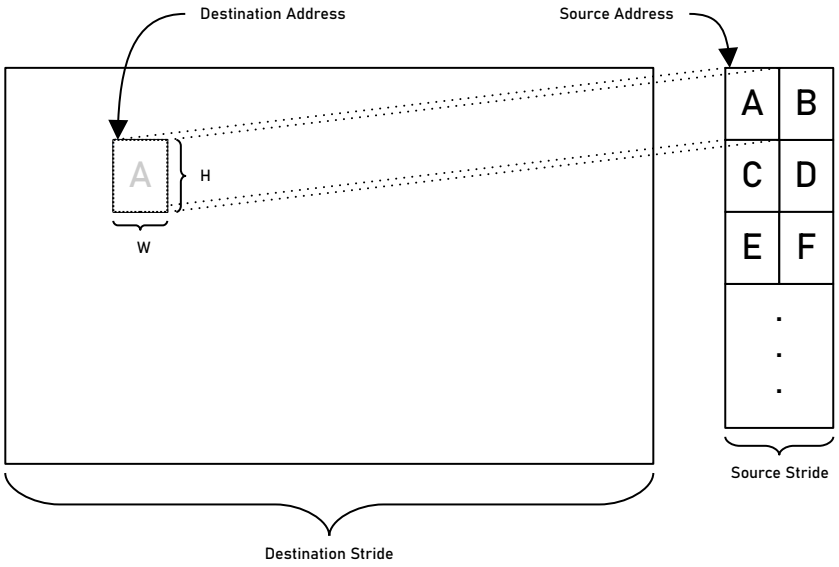


Figure 16.1: A 2D Copy DMA Operation

Example: Using DMA to Fill a Bitmap

In this example, we will use the 1D fill operation to set the bitmap shown on the screen to a set color. Note that the full example code includes the various setup operations for the bitmap mode and the graphics CLUTs, which are exactly the same as were used in the bitmap example (see page 4).

```
DMA_CTRL = $DF00                ; DMA Control Register
```

```
DMA_CTRL_START = $80            ; Start the DMA operation
DMA_CTRL_FILL = $04             ; DMA is a fill operation
DMA_CTRL_ENABLE = $01           ; DMA engine is enabled

DMA_STATUS = $DF01              ; DMA status register (Read Only)
DMA_STAT_BUSY = $80             ; DMA engine is busy with an operation

DMA_FILL_VAL = $DF01            ; Byte value to use for fill operations

DMA_DST_ADDR = $DF08            ; Destination address (system bus)
DMA_COUNT = $DF0C               ; Number of bytes to fill or copy

bitmap_base = $10000            ; The base address of our bitmap

bitmap_width = 320              ; The size of our bitmap
bitmap_height = 240
bitmap_size = bitmap_width*bitmap_height
```

First, we need to enable the DMA engine and set it up for a fill operation:

```
        lda #DMA_CTRL_FILL | DMA_CTRL_ENABLE
        sta DMA_CTRL
```

Next, we provide the value to fill:

```
        lda #$ff
        sta DMA_FILL_VAL    ; We will fill the screen with $FF
```

Then we need to provide the destination address:

```
        lda #<bitmap_base   ; Our bitmap will be the destination
        sta DMA_DST_ADDR
        lda #>bitmap_base
        sta DMA_DST_ADDR+1
        lda #‘bitmap_base
        and #$03
        sta DMA_DST_ADDR+2
```

Next, we provide the number of bytes to write:

```
        lda #<bitmap_size   ; We will write 320*240 bytes
        sta DMA_COUNT
        lda #>bitmap_size
        sta DMA_COUNT+1
        lda #‘bitmap_size
        sta DMA_COUNT+2
```

Finally, we flip the START flag to trigger the DMA operation and wait for it to complete:

```
            lda DMA_CTRL
            ora #DMA_CTRL_START
            sta DMA_CTRL

wait_dma:   lda DMA_STATUS      ; Wait until DMA is not busy
            and #DMA_STAT_BUSY
            cmp #DMA_STAT_BUSY
            beq wait_dma

            stz DMA_CTRL        ; Turn off the DMA engine
```

Example: Using DMA to Fill a Rectangle

We can extend our 1D DMA example to draw a 2D rectangle at a given coordinate on the screen. In this example, we will calculate the starting address based on coordinates using the assembler's built-in math functions for constants. In general, a program will need to calculate these values at runtime and could use the built-in integer math coprocessor.

We will need to use some different registers to specify the size and layout of the rectangular area. These registers overlap the registers used in the 1D case. The first two replace the `DMA_COUNT` register to give the width and height of the rectangle. The second specifies the stride for the destination bitmap (for copying, there is a source stride as well that can be different from the destination stride):

```
DMA_WIDTH = $DF0C               ; Width of rectangle (16-bit)
DMA_HEIGHT = $DF0E              ; Height of rectangle (16-bit)
DMA_STRIDE_DST = $DF12          ; Stride of the destination (16-bit)
```

We start off very similar to the 1D fill and just turn on the flag for 2D DMA:

```
lda #DMA_CTRL_FILL | DMA_CTRL_2D | DMA_CTRL_ENABLE
sta DMA_CTRL
```

We can then calculate the starting address, using the base address of the bitmap and a starting coordinate of $(100, 40)$:

```
lda #<(bitmap_base + 320 * 40 + 100)
sta DMA_DST_ADDR
lda #>(bitmap_base + 320 * 40 + 100)
sta DMA_DST_ADDR+1
lda #`(bitmap_base + 320 * 40 + 100)
sta DMA_DST_ADDR+2
```

Next, we set the value to write into the bitmap fill area:

```
lda #$30
sta DMA_FILL_VAL    ; We will fill the screen with $30
```

A difference from 1D is that the size of the fill is a width and a height, so we provide the width and height of the rectangle we want to draw. In this case, we are setting it to $(100, 30)$:

```
            lda #100            ; Size of rectangle is (100,30)
            sta DMA_WIDTH
            stz DMA_WIDTH+1
            lda #30
            sta DMA_HEIGHT
            stz DMA_HEIGHT+1
```

For 2D DMA operations, we need to provide the “stride”. This is the width of the overall bitmap into which the DMA operation is writing. In this case, we are updating a bitmap that is the full screen, so we set the destination stride to 320:

```
            lda #<bitmap_width  ; Set the width of the destination bitmap for 2D DMA
            sta DMA_STRIDE_DST
            lda #>bitmap_width
            sta DMA_STRIDE_DST+1
```

And then we can start the DMA operation and wait for it to complete:

```
            lda DMA_CTRL
            ora #DMA_CTRL_START
            sta DMA_CTRL

wait_dma2d: lda DMA_STATUS      ; Wait until DMA is not busy
            and #DMA_STAT_BUSY
            cmp #DMA_STAT_BUSY
            beq wait_dma2d
```

17

System Control Registers

The Buzzer and Status LEDs

The F256 has several software-controllable LEDs. There are the SD card access LED and the power LED, but there are also two status LEDs on the board which may be controlled either manually or set to flash automatically. All the LEDs under "manual" control can be controlled by setting or clearing their relevant flags in the SYS0 register (0xD6A0) (see table: 17.1). The power LED is controlled by PWR_LED. The SD card LED is controlled by SD_LED.

Address	R/W	Name	7	6	5	4	3	2	1	0
0xD6A0	W	SYS0	RESET	—	CAP_EN	BUZZ	L1	L0	SD_L	PWR_L
0xD6A0	R	SYS0	—	SD_WP	SD_CD	BUZZ	L1	L0	SD_L	PWR_L
0xD6A1	R/W	SYS1	L1_RATE		L0_RATE		SID_ST	PSG_ST	L1_MN	L0_MN

Table 17.1: System Control Registers

The two status LEDs on the board are a little more complex. They may be in manual or automatic mode. The two flags L0_MN and L1_MN in SYS1 control which mode they are in. If an LED's flag is set (1), then the LED is under manual control and its equivalent flag in SYS0 controls whether the LED is on or off. If the flag is clear, then the LED is set to flash automatically, and the LED's flashing rate will be set by pair of bits L0_RATE or L1_RATE according to table 17.2.

The flag PSG_ST controls how the output of the PSG sound chips are mixed. If clear (0), the PSGs will be mixed for monaural output (both will go to both left and right channels). If set (1), the left PSG will go to the left channel, and the right PSG will go to the right channel. This allows a program to use the PSGs as either independent 4 voice channels in stereo or as a monaural 8 voice channels.

The flag SID_ST controls how the output of the SID devices are mixed. If clear (0), the SIDs will be mixed for monaural output (both will go to both left and right channels). If set (1), the left SID will go to the left channel, and the right SID will go to the right channel. This allows a program to use the SIDs as either independent 3 voice channels in stereo or as a monaural 6 voice channels.

For the PC speaker, there is the BUZZ flag. By toggling BUZZ, a program can tweak the speaker and make a noise.

> **Note:** CAP_EN is only on the F256k. If set, it enables the RGB LED for the shift lock key. It is ignored on the F256. Likewise, the F256k does not have a built-in buzzer, so BUZZ is ignored on the F256k. The SID_ST flag is also available on the F256k only. The same functionality is provided by jumpers on the F256jr board.

RATE1	RATE0	Rate
0	0	1s
0	1	0.5s
1	0	0.4s
1	1	0.2s

Table 17.2: LED Flash Rates

Software Reset

A program can trigger a system reset. This can be done by writing the value `0xDE` to `0xD6A2` and the value `AD` to `0xD6A3` to validate that a reset is really intended (see table: 17.3), setting the most significant bit (RESET) of `0xD6A0`, and then clearing the RESET bit to actually trigger the reset.

Address	R/W	Name	7	6	5	4	3	2	1	0
0xD6A2	R/W	RST0	Set to 0xDE to enable software reset							
0xD6A3	R/W	RST1	Set to 0xAD to enable software reset							

Table 17.3: System Reset

Random Numbers

The F256 has a built-in pseudo-random number generator that produces 16-bit random numbers (see table: 17.4). To use the random number generator, a program just sets the enable flag and then reads the random numbers from RNDL and RNDH (0xD6A4 and 0xD6A5). The program can set the seed value to better randomize the numbers by storing a seed value in those same locations and then toggling SEED_LD (set to load the seed value then reclear).

ENABLE set to turn on the random number generator

SEED_LD set to load a value stored in SEEDL and SEEDH as the seed value for the random number generator

RNDL and RNDH read 16-bit random numbers from these registers when the random number generator is enabled

Address	R/W	Name	7	6	5	4	3	2	1	0
0xD6A4	W	SEEDL	SEED[7…0]							
0xD6A4	R	RNDL	RND[7…0]							
0xD6A5	W	SEEDH	SEED[15…0]							
0xD6A5	R	RNDH	RND[15…0]							
0xD6A6	W	RND_CTRL	—						SEED_LD	ENABLE
0xD6A6	R	RND_STAT	DONE	—						

Table 17.4: Random Number Generator

Machine ID and Version Information

Nine registers are set aside to identify the machine, the version of the printed circuit board, and the version of the FPGA. See table 17.5 for the various registers. All the registers are read-only, and only the chip information will change over the course of the machine's life span. The machine ID contains a six-bit code that is common between all the C256 and F256 family machines (see table 17.6).

For the F256, the machine ID will be 0x02. For the F256k, the machine ID will be 0x12.

Address	R/W	Name	7	6	5	4	3	2	1	0
0xD6A7	R	MID	—			ID				
0xD6A8	R	PCBID0	ASCII character 0: “B”							
0xD6A9	R	PCBID1	ASCII character 1: “0”							
0xD6AA	R	CHSV0	TinyVicky subversion in BCD (low)							
0xD6AB	R	CHSV1	TinyVicky subversion in BCD (high)							
0xD6AC	R	CHV0	TinyVicky version in BCD (low)							
0xD6AD	R	CHV1	TinyVicky version in BCD (high)							
0xD6AE	R	CHN0	TinyVicky number in BCD (low)							
0xD6AF	R	CHN1	TinyVicky number in BCD (high)							
0xD6EB	R	PCBMA	PCB Major Rev (ASCII)							
0xD6EC	R	PCBMI	PCB Minor Rev (ASCII)							
0xD6ED	R	PCBD	PCB Day (BCD)							
0xD6EE	R	PCBM	PCB Month (BCD)							
0xD6EF	R	PCBY	PCB Year (BCD)							

Table 17.5: Machine ID and Versions

MID5	MID4	MID3	MID2	MID1	MID0	Hex using 0x1F Mask	CPU	Machine
0	0	0	0	1	0	0x02	6502/65816/6809	F256 Jr. (mmu)
0	1	0	0	1	1	0x12	6502/65816/6809	F256K (mmu)
0	1	1	0	1	0	0x1A	6809	F256 Jr.Jr.
0	1	0	1	1	0	0x16	6809	F256K2
0	0	0	0	0	0	0x00	65816	C256 FMX
0	0	0	0	0	1	0x01	65816	C256 U
0	0	0	0	1	0	0x02	6502/65816/6809	F256 Jr. (mmu)
0	0	0	0	1	1	0x03	65816	F256 Jr. (ext)
1	0	0	0	1	1	0x02	65816	F256 Jr.Jr. (mmu)
1	0	0	0	1	1	0x03	65816	F256 Jr.Jr. (ext)
0	0	0	1	0	0	0x04	65816	Gen X
0	0	0	1	0	1	0x05	65816	C256 U+ (4M SRAM)
0	0	0	1	1	0	0x06	Reserved	Reserved
0	0	0	1	1	1	0x07	Reserved	Reserved
0	0	1	0	0	0	0x08	68xx0xx	A2560 X/GenX
0	0	1	0	0	1	0x09	68EC000	A2560 U+
0	0	1	0	1	0	0x0A	68LC060	A2560 M
0	0	1	0	1	1	0x0B	68040RC25V	A2560 K (classic)
0	0	1	1	0	0	0x0C	68040FE33V	A2560 K40
0	0	1	1	0	1	0x0D	68LC060	A2560 K60
0	0	1	1	1	0	0x0E	Undefined	Undefined
0	0	1	1	1	1	0x0F	Undefined	Undefined
0	1	0	0	0	0	0x10	65816	F256P
0	1	0	0	0	1	0x11	65816	F256K2 (mmu)
0	1	0	0	1	0	0x12	6502/65816/6809	F256K (mmu)
0	1	0	0	1	1	0x13	6502/65816	F256K (ext)
0	1	0	1	0	0	0x14	65816	F256K2 (ext)
0	1	0	1	0	1	0x15	Reserved	Reserved
0	1	0	1	1	0	0x16	6809	F256K2
0	1	0	1	1	1	0x17	Reserved	Reserved
0	1	1	0	0	0	0x18	68000	F256K2
0	1	1	0	0	1	0x19	Reserved	Reserved
0	1	1	0	1	0	0x1A	6809	F256 Jr.Jr. (mmu)

Table 17.6: Machine IDs

18

IEC Serial Port

The F256 has an IEC serial port included (this is the Commodore serial port variation of the IEEE-488 interface). There are two registers supporting the IEC port. There is a read-only register that shows the current state of the individual lines on the serial bus, and there is a read/write register that can be used to control the various lines as well as how IEC interrupts are handled.

Address	R/W	Name	7	6	5	4	3	2	1	0
0xD680	R	IEC_I	SRQ_i	—		ATN_i	—		CLK_i	DAT_i
0xD681	R/W	IEC_O	SRQ_o	RST_o	NMI_EN	ATN_o	—		CLK_o	DAT_o

Table 18.1: IEC Registers

DAT_i Reflects the current state of the DATA line on the IEC bus.

CLK_i Reflects the current state of the CLK line on the IEC bus.

ATN_i Reflects the current state of the ATN line on the IEC bus.

SRQ_i Reflects the current state of the SREQ line on the IEC bus.

DAT_o Sets the DATA line on the IEC bus.

CLK_o Sets the CLK line on the IEC bus.

ATN_o Sets the ATN line on the IEC bus.

SRQ_o Sets the SREQ line on the IEC bus.

RST_o Resets the IEC bus (and any installed SIDs on the F256jr).

NMI_EN If set (1), the IEC interrupts will trigger an NMI interrupt. If clear (0), the IEC interrupts will trigger an IRQ interrupt.

19 Integer Math Coprocessor

The F256 includes a built-in math coprocessor for integer math. This coprocessor provides fast 16-bit unsigned multiplication and division. As well as a 32-bit adder. The use of this coprocessor is straightforward: both operands are written to the appropriate registers and then the result is read for the corresponding answer register. The math units are completely separate blocks using separate registers, so they function independently of each other.

Address	R/W	Name	Data
0xFEE0	R/W	MULU_A_H	Unsigned A High Byte
0xFEE1	R/W	MULU_A_L	Unsigned A Low Byte
0xFEE2	R/W	MULU_B_H	Unsigned B High Byte
0xFEE3	R/W	MULU_B_L	Unsigned B Low Byte
0xFEF0	R	MULU_HH	$A \times B$ (unsigned) byte 3
0xFEF1	R	MULU_HL	$A \times B$ (unsigned) byte 2
0xFEF2	R	MULU_LH	$A \times B$ (unsigned) byte 1
0xFEF3	R	MULU_LL	$A \times B$ (unsigned) byte 0

Table 19.1: Unsigned Multiplication Registers

Address	R/W	Name	Data
0xFEE4	R/W	DIVU_DEN_H	Unsigned Denominator High Byte
0xFEE5	R/W	DIVU_DEN_L	Unsigned Denominator Low Byte
0xFEE6	R/W	DIVU_NUM_H	Unsigned Numerator High Byte
0xFEE7	R/W	DIVU_NUM_L	Unsigned Numerator Low Byte
0xFEF4	R	QUOU_LH	Quotient of NUM/DEN (unsigned) high byte
0xFEF5	R	QUOU_LL	Quotient of NUM/DEN (unsigned) low byte
0xFEF6	R	REMU_HH	Remainder of NUM/DEN (unsigned) high byte
0xFEF7	R	REMU_HL	Remainder of NUM/DEN (unsigned) low byte

Table 19.2: Unsigned Division Registers

Address	R/W	Name	Data
0xFEE8	R/W	ADD_A_HH	Unsigned A byte 3
0xFEE9	R/W	ADD_A_HL	Unsigned A byte 2
0xFEEA	R/W	ADD_A_LH	Unsigned A byte 1
0xFEEB	R/W	ADD_A_LL	Unsigned A byte 0
0xFEEC	R/W	ADD_B_HH	Unsigned B byte 3
0xFEED	R/W	ADD_B_HL	Unsigned B byte 2
0xFEEE	R/W	ADD_B_LH	Unsigned B byte 1
0xFEEF	R/W	ADD_B_LL	Unsigned B byte 0
0xFEF8	R	ADD_R_HH	$A + B$ (unsigned) byte 3
0xFEF9	R	ADD_R_HL	$A + B$ (unsigned) byte 2
0xFEFA	R	ADD_R_LH	$A + B$ (unsigned) byte 1
0xFEFB	R	ADD_R_LL	$A + B$ (unsigned) byte 0

Table 19.3: Unsigned 32-bit Addition Registers

20 Using the Debug Port

One of the ways to get software and data onto the F256 is through the USB debug port. The debug port uses a USB serial protocol to allow a host computer to issue commands to the F256. These commands allow the host computer to stop and start the CPU, write to memory, read from memory, erase the flash memory, and reprogram the flash memory. With this port, it is possible to load a program and its data directly into the F256's memory and start it running. It is also possible to examine the F256's memory to see what state a program has left it in.

Three are three main tools available to provide user access to the debug port:

Foenix IDE A full-featured emulator and development tool for the Foenix line of computers. Among the many tools provided by the IDE is a built-in GUI tool to upload and download data to the F256 and program the flash. The main limitation of the IDE is that it was written in .NET and uses features that are available under the Windows API.

Foenix Uploader Tool A stand-alone version of just the uploader tool from the Foenix IDE. This tool is more limited (it may only support binary files) and is tailored to specific machines.

FoenixMgr A script written in Python 3 which provides command line access on the host computer to the debug port. It supports files in Intel HEX, Motorola SREC, raw binary, PGX, and PGZ files. It should run on any computer or operating system that can run Python 3 and provide sufficient access to USB serial interfaces. It runs under Windows and Linux definitely and may be able to run under Mac OS X eventually.

Debug Protocol

The USB debug port is accessed over the USB Serial protocol. Data is sent from the host computer to the F256 using data packets, each one of which is a command. The general process is:

1. Host PC sends the command to enter debug mode
2. The F256 replies
3. Host PC sends a command packet
4. The F256 replies
5. The host repeats from step 3 until finished

6. Host PC send the command to exit debug mode

7. The F256 replies and sends a reset signal to the CPU

Command Packets

The commands sent from the host PC are in the form of command packets show in table 20.1. The command codes themselves are listed in table 20.3. The F256 will respond to each command packet with a response packet as shown in table 20.2. The size of a packet can vary depending on the command. Some commands and responses include no actual data payload bytes. Others will transfer actual data and will include however many bytes of payload are needed.

Each command and response packet includes an LRC check byte, which is simply the exclusive or of all the bytes in the packet, except for the LRC value itself. This provides only rudimentary error checking, but the connection itself is generally pretty reliable, so more sophisticated error checking is really not needed.

Offset	Size	Name
0	1	Command sync byte
1	1	Command byte
2	3	Address
5	2	Length
7	n	Payload
$7 + n$	1	LRC check byte

Table 20.1: USB Debug Port Command Packet

Command sync byte This is always `0x55` and signals the start of a command packet

Command byte This byte specifies what command is being sent (see table: 20.3)

Address This is a three byte, big-endian integer that provides the address relevant to the command. For a write command, it is the address of the first block of memory to receive data. For a read command, it is the address of the first byte of memory to read. For the program flash command, it is the address of the first byte of data to write to flash.

Length This is the number of bytes to transfer. For a write command, it is the number of bytes to be sent to the F256 and will be control the size of the payload section of the write command packet. For the read command, it is the number of bytes to read from the F256 and will control the size of the payload section of the response packet (the payload section of the read command packet is empty).

Payload This is an option section of the packet that contains the actual data to transfer between the host PC and the F256.

LRC check byte This byte provides for simple error checking on the packet transmission.

Offset	Size	Purpose
0	1	Response sync byte (0xAA)
1	2	Status bytes
3	m	Payload
$3 + m$	1	LRC check byte

Table 20.2: USB Debug Port Reponse Packet

Response Packets

Response packets follow the same general structure as a command packet (see table 20.2):

Response sync byte This is always 0xAA and signals the start of a response packet

Status bytes These two bytes contain the status codes for the success or failure of the command

Payload This is an option section of the packet that contains the actual data to transfer between the host PC and the F256.

LRC check byte This byte provides for simple error checking on the packet transmission.

Command	Purpose
0x80	Enter debug mode
0x81	Exit debug mode (resets CPU)
0x00	Read a block of data from the F256 to the host PC
0x01	Write a block of data to RAM on the F256
0x10	Program flash memory from data in F256's RAM
0x11	Erase flash memory
0x12	Erase flash sector
0x13	Program flash sector
0x90	Set the MMU to boot in RAM (F256jr Rev A, or F256k only)
0x91	Set the MMU to boot in flash (F256jr Rev A, or F256k only)
0xFE	Fetch the revision number of the debug interface

Table 20.3: USB Debug Port Commands

Flash Sectors

Individual blocks or sectors of flash may be erased or programmed without affecting the rest of flash memory. This can be done through the commands 0x12 to erase flash sectors and 0x13 to program them from RAM. The packets for sectors are a little different from the others. The main difference is that third byte of the packet (ordinarily the high byte of the address) is the number of the sector to program, and addresses are limited to 16-bits. Each sector is a 4KB block, with 0 being the first 4KB of flash, 1 being the second 4KB of flash, and so on.

The flash of the F256 has a limitation that the smallest block of flash that can be erased is 8KB, so when erasing sectors, two sectors must be erased, not just one. And the sector pairs must be aligned to 8KB. So sector 0 and sector 1 would be erased together, but not sector 1 and sector 2 (although sectors 0 – 3 would be fine).

Programming flash sectors has no such limitation (it is fine to flash just a 4KB block). However, for simplicity's sake, it would probably be best for any program directly accessing the debug port to limit erasing and programming to 8KB blocks. Programming the flash sectors does have a limitation: since the address is limited to 16-bits, the data can only be stored in the first 64KB of the 512KB system RAM.

21 Expansion Connector

The F256 includes a PCIe x1 style expansion connector. This expansion connector is designed to support both RAM expansion and game style ROM cartridges. The pin assignments for the expansion connector can be seen in figure 21.1, while the size information for the card itself can be seen in figure 21.2.

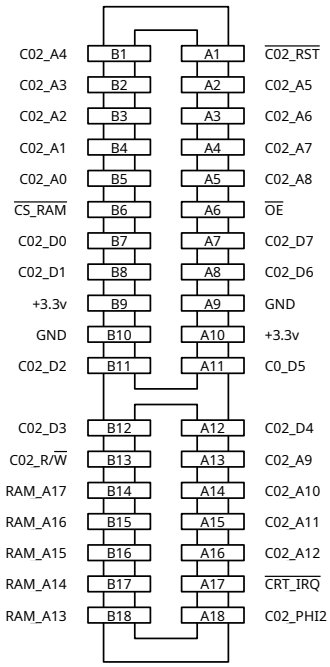


Figure 21.1: Expansion Port

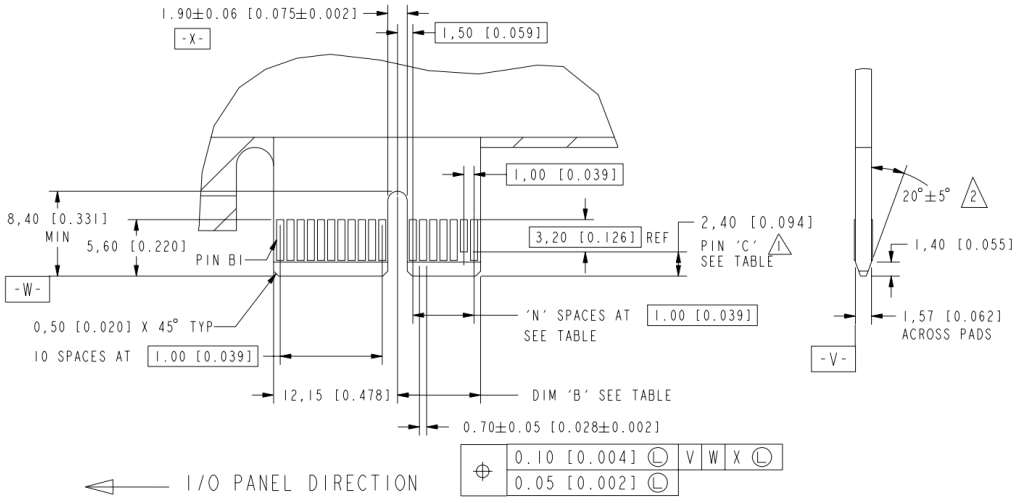


Figure 21.2: Expansion Port Physical Size

22

F256jr I/O Connectors and Jumpers

Note: This chapter describes the various jumpers and connectors on the F256jr board and is not relevant to the F256k.

Connectors

The F256jr has several connectors on its board for I/O devices beyond the standard connections on the back. Some of these connectors are the only way to access that particular I/O device, but some are auxiliary connectors that provide alternate forms of access. All are IDC header pins. The diagrams that follow show the pin assignments for these connectors. In these diagrams, the views are top-down onto the board, with the board arranged so that the main connectors are towards the top and the power connector is towards the bottom.

Game Connectors

There are two connectors for game controllers. There are two connectors for Atari style joysticks (figure 22.1). And two for NES or Super NES gamepads (figure 22.2).

For the Atari style joysticks, an adapter cable will be needed to convert from IDC to the standard DE-9 connector. There are two key differences between the F256jr and Atari or Commodore devices: first, the F256jr provides 3.3 volts instead of 5 volts, and second there are no paddle inputs supported on the F256jr.

For the NES and SNES style gamepads, an adapter box will be needed to provide the correct interfacing for a controller. This adapter should be available from Wildbits Computing in the near future (at the time of writing this manual).

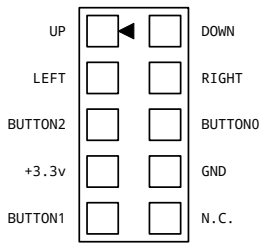


Figure 22.1: Joystick Port Pinouts

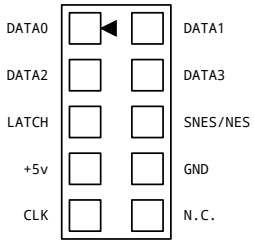


Figure 22.2: NES/SNES Gamepad Port Pinouts

PS/2 Port

An internal connector is included for the PS/2 mouse and keyboard port (figure 22.3). This connector just provides an alternate way of accessing the PS/2 signals. It could be useful in building an integrated case for the F256jr that includes a PS/2 keyboard.

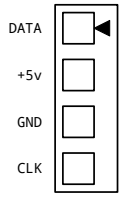


Figure 22.3: Auxiliary PS/2 Pinouts

UART

This connector provides access to the serial in and out signals for the F256jr's UART (figure 22.4). The TxD and RxD signals are compatible with standard ±12 volt RS-232 signals. The signals on this connector can be brought out to a DE-9 connector to provide a standard 3 wire RS-232 serial port.

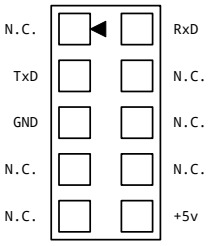


Figure 22.4: UART Pinouts

An ESP32 Feather board can be installed on the F256jr to provide access to Wi-Fi, but using this board will take over the supplied UART. Therefore, the F256jr can provide either an RS-232 serial port or access to Wi-Fi, but not both at the same time. A jumper on the board selects which is active.

USB Debug Port

While there is a USB Mini-B connector (figure: 22.5) on the port to access the USB debug interface, there is an IDC header connector on the board to provide access to this for the case USB connector, if desired. This is not compatible with all USB case connectors, as many of them are dual connectors and require pins for two USB ports. It should be compatible with cases with single connectors or with panel mounted USB cables.

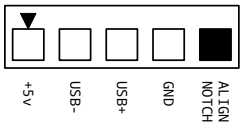


Figure 22.5: USB Debug Pinouts

Case Connectors

There are two connectors for the usual PC case connections. There is one for the headphone jack (figure: 22.6), and one for the various buttons, LEDs, and case speaker (figure: 22.7): power, reset, hard drive/SD card activity. Note that while the speaker and buttons are not polarized, the power LED and hard-drive activity LED are and must be wired in the correct orientation.

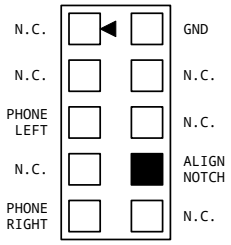


Figure 22.6: Case Audio Pinouts

Not shown is a small secondary connector for an SPST switch for power (the connector is located at the front-right corner of the F256jr RevB board). This connector provides an alternative to the usual case power push button. The two buttons cannot be used together. Either the case push button shown in 22.7 is used, or the other SPST switch is used, but not both.

Jumpers

There are several IDC header pins with jumpers to configure various options on the F256jr. All the jumper headers are three pin headers, where the center pin is the common. The header is used to select the appropriate routing of a signal or voltage level. So the jumper is always used to connect the center common pin to either the left or right pins (or the top or bottom, depending on the orientation of the header).

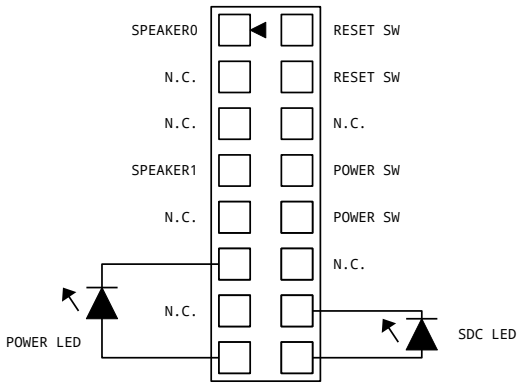


Figure 22.7: Case Button and LED Pinouts

SID Jumpers

The SID chips have two sets of jumpers each. The first pair are the voltage selection jumpers. They can be used to select the appropriate voltage for the SID chip: 12 volts for the original 6581, and 9 volts for the later 8580 (see figure 22.8). If you are using a modern replacement, check the instructions as to which voltage to use: some replacements work with either voltage.

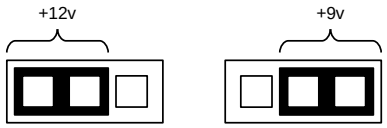


Figure 22.8: SID Voltage Jumper

The second pair of SID jumpers are the channel selectors (see figure 22.9). These jumpers select the source of the left and right channels for the CODEC. With each one, you can select either the right or the left SID as an input to the CODEC for that stereo channel. If a F256jr is using both SIDs, the left channel should select the left SID, and the right channel select the right SID. But if only a single SID is used, both channels can be set to that SID. For instance, if the SID is in the left socket, both left and right channels can select the left SID as the source to get a balanced monaural sound.

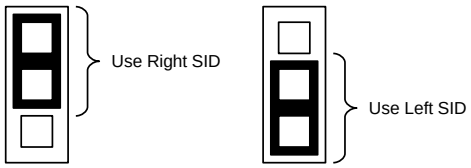


Figure 22.9: SID Stereo Channel Source Jumper

Boot Source

There is a jumper to select boot source (see figure 22.10). With the jumper in the left position, the F256jr will boot off the flash, using the last 8KB bank of flash memory as the last 8KB of CPU address space. With the jumper in the right position, the F256jr will boot using RAM. See page 14 for a more complete description of boot sources.

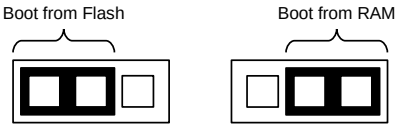


Figure 22.10: Boot Source Jumper

UART Configuration

There are two headers for selecting how the UART is used—selecting between the serial port or the ESP32 Wi-Fi module (see figure 22.11). One jumper controls the routing of the TxD line, while the other controls the routing of the RxD line. With the jumper positioned across the “back” pair of pins, the signal is routed to the Wi-Fi module. With the jumper positioned across the “front” pair of pins, the signal is routed to the DE-9 port connected to the UART connector. Note that both the TxD and RxD jumpers should be in the same relative position so that they both use the same device.



Figure 22.11: UART Device Jumpers

23 Memory Maps

Address	Purpose
0x000000	System RAM for programs, data, and graphics (512 KB)
0x07FFFF	
0x080000	Flash memory (512 KB)
0x0FFFFF	
0x100000	Expansion RAM for programs, and data (256 KB)
0x13FFFF	
0x140000	Reserved
0x1FFFFF	

Table 23.1: System Memory Map for the F256

Bank	Address	Purpose
0	0x0000	MMU Memory Control Register
	0x0001	MMU I/O Control Register
	0x0002	RAM or Flash
	0x0007	
	0x0008	RAM, Flash, or MMU LUT Registers
	0x000F	
	0x0010	65C02 Page Zero
	0x00FF	
	0x0100	65C02 Stack
	0x01FF	
	0x0200	RAM or Flash
	0x1FFF	
1	0x2000	RAM or Flash
	0x3FFF	
2	0x4000	RAM or Flash
	0x5FFF	
3	0x6000	RAM or Flash
	0x7FFF	

Table 23.2: CPU Memory Map for the F256 (Banks 0–3)

Bank	Address	Purpose
4	0x8000	RAM or Flash
	0x9FFF	
5	0xA000	RAM or Flash
	0xBFFF	
6	0xC000	RAM, Flash, I/O, Text mode character, or color data
	0xDFFF	
7	0xE000	RAM or Flash
	0xFFFA	
	0xFFFA	65C02 NMI Vector
	0xFFFB	
	0xFFFC	65C02 Reset Vector
	0xFFFD	
	0xFFFE	65C02 IRQ Vector
	0xFFFF	

Table 23.3: CPU Memory Map for the F256 (Banks 4–7)

Start	End	Purpose	Page
0xC000	0xC3FF	Gamma Table Blue	53
0xC400	0xC7FF	Gamma Table Green	
0xC800	0xCBFF	Gamma Table Red	
0xCC00	0xCFFF	Reserved	
0xD000	0xD0FF	VICKY Master Control Registers	20
0xD100	0xD1FF	VICKY Bitmap Control Registers	31
0xD200	0xD2FF	VICKY Tile Control Registers	41
0xD300	0xD3FF	Reserved	
0xD400	0xD4FF	SID Left	59
0xD500	0xD57F	SID Right	
0xD580	0xD583	OPL3 (F256k only)	61
0xD600	0xD607	PSG Left	56
0xD608	0xD60F	PSG Left and Right	
0xD610	0xD61F	PSG Right	
0xD620	0xD62F	CODEC	55
0xD630	0xD63F	UART	86
0xD640	0xD64F	PS/2 Interface	82
0xD650	0xD65F	Timers	67
0xD660	0xD66F	Interrupt Controller	62
0xD670	0xD67F	DIP Switch	47

Table 23.4: I/O Page 0 Addresses (0xC000–0xD67F)

Start	End	Purpose	Page
0xD680	0xD68F	IEC Controller	74
0xD690	0xD69F	Real Time Clock	69
0xD6A0	0xD6AF	System Control Registers	95
0xD6A7	0xD6AF	F256k LEDs (F256k only and write-only)	85
0xD6B0	0xD6DF	Reserved	
0xD6E0	0xD6EA	Mouse Pointer Registers	83
0xD6EB	0xD7EF	PCB Version Register	97
0xD6EF	0xD7FF	Reserved	
0xD800	0xD83F	Text Foreground Color LUT	19
0xD840	0xD87F	Text Background Color LUT	
0xD880	0xD8FF	Reserved	
0xD900	0xDAFF	VICKY Sprite Control Registers	34
0xDB00	0xDBFF	Reserved	
0xDC00	0xDCFF	65C22 VIA Control Registers	74
0xDD00	0xDDFF	SD Card Controller	82
0xDE00	0xDE1F	Integer Math Coprocessor	100
0xDE20	0xDEFF	Reserved	
0xDF00	0xDF13	DMA Controller	90

Table 23.5: I/O Page 0 Addresses (0xD680–0xDF13)

Start	End	Purpose	Page
0xC000	0xC7FF	Text Mode Font Set 0 Memory	21
0xC800	0xCFFF	Text Mode Font Set 1 Memory	21
0xD000	0xD3FF	Graphics Color LUT 0	27
0xD400	0xD7FF	Graphics Color LUT 1	
0xD800	0xDBFF	Graphics Color LUT 2	
0xDC00	0xDFFF	Graphics Color LUT 3	

Table 23.6: Memory Map for I/O Page 1

24 References

It is not possible to cover all the details of all the chips that are part of the F256 in this one reference manual. This chapter lists links to all the data sheets for each chip used or implemented as well as some other useful websites. These chips include the 65C02 (CPU), 65C22 (VIA), WM8776 (sound CODEC), bq4802ly (real time clock), 6581 (SID), and the SN76489 (PSG).

CPU https://www.westerndesigncenter.com/wdc/documentation/w65c02s.pdf

VIA https://www.westerndesigncenter.com/wdc/documentation/w65c22.pdf

CODEC https://web.archive.org/web/20140801092610/http://www.wolfsonmicro.com/media/76476/WM8776.pdf

RTC https://www.ti.com/lit/ds/symlink/bq4802y.pdf

SID http://archive.6502.org/datasheets/mos_6581_sid.pdf

PSG http://www.vgmpf.com/Wiki/images/7/78/SN76489AN_-_Manual.pdf

OPL3 http://www.bitsavers.org/components/yamaha/YMF262_199110.pdf (F256K only)

The website for all Wildbits information is https://wildbitscomputing.com, and the community wiki is at https://f256wiki.wildbitscomputing.com. The latest electronic copy of this manual is at https://github.com/jfed6000/Wildbits-F256-6809-Manual

25 Copyright

This work is licensed under the Creative Commons Attribution-ShareAlike 4.0 International License. To view a copy of this license, visit http://creativecommons.org/licenses/by-sa/4.0/ or send a letter to Creative Commons, PO Box 1866, Mountain View, CA 94042, USA.